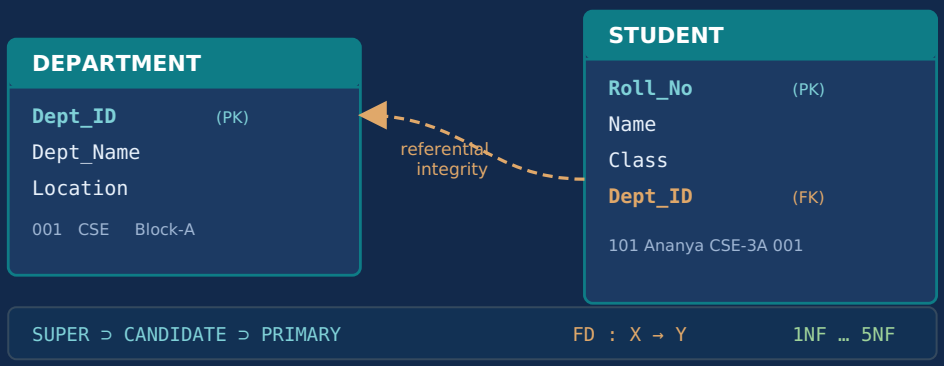
CHANDIGARH UNIVERSITY • UIE • DEPARTMENT OF CSE • UNIT 2

Database Management Systems

Complete Exam-Ready Notes
Concepts, Keys, Functional Dependencies, Normalization, Database Security, PL/SQL, Cursors, and SQL Views



Build from: Chandigarh University DBMS lecture material, UNIT 2 (Chapters 2.1.1–2.1.7 and 2.3.1–2.3.5)
Inside: 8 parts • 36 chapters • 40+ original diagrams • 30+ worked tables and dry runs
For: university theory papers, SQL / PL-SQL practicals, output prediction, normalization problems and viva

Database Management System (24CSH-204 / 24ITH-204) • Academic Session 2026-27, Odd Semester (Jul–Dec 2026)
Course instructor of the source material: **Dr. Dinesh Singh Dhakar**, Assistant Professor, Computer Science and Engineering, University Institute of Engineering, Chandigarh University — Post Doctorate (Canada) | Ph.D. | M.Tech. | CDAC, Pune | B.Tech. • Discover · Learn · Empower

How to Use These Notes

Front matter • reading paths, conventions and the visual key

These notes replace slide-by-slide study with a single continuous explanation. Each topic is introduced with the problem it solves, then the rule, then a worked example on small tables that you can check by hand. Nothing important from the source has been dropped; the source's fragments have been re-ordered into a sequence where each idea rests on the one before it.

Three reading paths

Table 0.1 — Choose the path that matches the time you have.

Path	Read	Best for
First-time study	Parts 1–7 in order, working every example on paper	Learning the subject properly
Exam revision (one week)	Part 8 first (comparison tables), then the chapter summaries of Parts 1–7	Consolidating before a paper
Night before	Chapter 35 (last-minute sheet) plus the question bank in Chapter 32	Fast recall

The visual key

Colour is used with a fixed meaning throughout, exactly as a technical figure should. Learn the key once and every later diagram reads faster.

DEFINITION
Blue panels carry a formal definition you should be able to reproduce almost word for word.

EXPLANATION
Teal panels add reasoning, background or a bridge to the next idea.

WORKED STEP
Amber panels show intermediate reasoning: a dry run, a probe sequence, a closure computation.

COMMON MISTAKE
Red panels flag a genuine error or a trap that costs marks.

CORRECT RESULT
Green panels mark a verified output, a successful decomposition, or a fact worth memorising.

FROM THE SOURCE
Violet notes record where the source material is ambiguous, incomplete or internally inconsistent, and say plainly what has been done about it.

Diagram colour code

- Primary key / unique identity
- Foreign key / dependency
- Correct, lossless, successful path
- Anomaly, violation, error path
- Structure, headings, containers

Conventions used in the code

All database code in these notes is **Oracle SQL and Oracle PL/SQL**, the dialect of the source material. SQL keywords appear in upper case, identifiers in lower case, and every PL/SQL block ends with a slash (`/`) because that is how the blocks are submitted at the SQL*Plus prompt. Output blocks are shown immediately after the program that produces them, and were derived by tracing the program statement by statement. Where a program uses a substitution variable (`&num`), the value typed at the prompt is shown in the transcript exactly as SQL*Plus echoes it.

> **A WORD ON THE SOURCE**
>
> The source material is a set of lecture notes and slides: it is accurate in substance but fragmented, sometimes repeats a definition twice with different wording, and occasionally leaves a table image described only by a caption. These notes keep the source's terminology and examples, supply the missing tables explicitly, and add new illustrative examples only where the source gives none — such additions are always labelled **“illustrative example”**. Chapter 36 lists exactly what was covered and what was not.

Source, Scope and Coverage Statement

Front matter • what came from where, and what these notes deliberately do not contain

The course the source belongs to

The supplied material is the UNIT 2 lecture set of **Database Management System (24CSH-204 / 24ITH-204)**, University Institute of Engineering, Department of Computer Science & Engineering, Chandigarh University, Academic Session 2026-27 (Odd Semester, Jul–Dec 2026), prepared by Dr. Dinesh Singh Dhakar. The source's own chapter numbers are preserved in the mapping table below so you can cross-check against your slides.

Table 0.2 — Source chapter to these notes.

Source chapter	Source title	Covered in
2.1.1	Functional dependencies	Part 2 (Ch. 3–5)
2.1.2	Introduction to Normalization and 1NF	Part 3 (Ch. 6)
2.1.3	Second Normal Form (2NF)	Part 3 (Ch. 7)
2.1.4	Third Normal Form (3NF)	Part 3 (Ch. 8)
2.1.5	BCNF and 4NF	Part 3 (Ch. 9–10)
2.1.6	5NF and Denormalization	Part 3 (Ch. 11, 14)
2.1.7	Keys in detail	Part 1 (Ch. 1–2)
–	Database Security (slides)	Part 4 (Ch. 15–22)
2.3.1	Conditional Control Structures	Part 5 (Ch. 24)
2.3.2	Iterative Control Structures	Part 5 (Ch. 25)
2.3.3	Sequential Control Structures	Part 5 (Ch. 26)
2.3.4	PL/SQL Cursors	Part 6 (Ch. 27–28)
2.3.5	SQL Views	Part 7 (Ch. 29–30)

Course outcomes as stated in the source

Table 0.3 — Course outcomes (verbatim intent from the source slides).

CO	Title	Bloom level
CO1	Understand database system concepts and design databases for different applications	B1, B2, B3
CO2	Identify different types of DDL, DML, DCL and TCL commands and their usage	B3, B4
CO3	Classify normalization, dependencies and denormalization along with their requirements	B6
CO4	Analyse and compare transaction processing techniques and recovery techniques	B5, B6
CO5	Evaluate database performance after implementing triggers, procedures, packages, cursors and views	B6

HONEST SCOPE LIMIT — PLEASE READ

The course outcome list mentions **transaction processing**, **recovery techniques**, **triggers**, **procedures** and **packages**. The supplied PDF contains no lecture content for those topics: it covers only keys, functional dependencies, normalization, denormalization, database security, PL/SQL control structures, cursors and views. Those topics are therefore **not** written as chapters here, because doing so would mean inventing material and presenting it as if it came from your course. They are listed in Chapter 36 as a documented coverage gap.

What these notes add beyond the source

Three things, all clearly marked. First, **diagrams**: the source's figures exist only as captions such as “Output:”, so every table, relationship, dependency and flow has been redrawn as a vector figure with a number and caption. Second, **explicit tables**: where the source says “the above table” without showing it, the rows have been reconstructed from the surrounding text and are labelled as reconstructed. Third, a **question bank** (Chapters 32–34) built strictly from the source's own examples. No new DBMS theory has been imported from outside the supplied material.

Contents

A complete DBMS chapter built from the supplied Chandigarh University lecture material (UNIT 2): relational keys, functional dependencies, normalization, database security, PL/SQL control structures, cursors and SQL views — taught from first principles, then revised for exams, practicals and viva.

How to Use These Notes 2

Source, Scope and Coverage Statement 4

PART 1 — Relational Foundations and Database Keys 11

1. Relational Model Foundations and the Idea of a Key 12
   - 1.1 Relation, Tuple, Attribute and Domain 12
   - 1.2 Why a Database Needs Keys 12
   - 1.3 The Vocabulary Trap: Relation = Table, Tuple = Record 13
2. The Ten Keys in Detail 14
   - 2.1 Super Key 14
   - 2.2 Candidate Key 15
   - 2.3 Primary Key 16
   - 2.4 Alternate Key 16
   - 2.5 Foreign Key and Referential Integrity 17
   - 2.6 Partial Key 17
   - 2.7 Composite Key and Compound Key 18
   - 2.8 Unique Key 18
   - 2.9 Surrogate Key 19
   - 2.10 Secondary Key 19
   - 2.11 Key Classification at a Glance 20
   - 2.12 Commonly Confused Pairs 20
   - 2.13 Key Identification Exercises 21

PART 2 — Functional Dependencies 22

3. Functional Dependencies: Meaning and Notation 23
   - 3.1 What a Functional Dependency Says 23
   - 3.2 Determinant and Dependent 24
   - 3.3 Reading a Dependency Correctly 24
   - 3.4 Finding Dependencies in a Table 24
   - 3.5 Trivial, Non-Trivial and Completely Non-Trivial 25
   - 3.6 Why Functional Dependencies Matter 26
4. Armstrong's Axioms and Reasoning with Dependencies 27
   - 4.1 The Three Primary Axioms 27
   - 4.2 Derived Rules 27
   - 4.3 Closure of a Set of Attributes 28
   - 4.4 Worked Closure Computations 28
   - 4.5 Equivalence of Dependency Sets 29
5. Special Dependencies: Transitive, Multivalued and Join 30
   - 5.1 Transitive Dependency 30
   - 5.2 Multivalued Dependency 31
   - 5.3 Join Dependency 32
   - 5.4 Dependency Types and the Normal Forms They Drive 33

PART 3 — Normalization 34

6. Normalization Fundamentals and First Normal Form 35
   - 6.1 What Normalization Is 35
   - 6.2 The Three Anomalies 36

6.3 The Ladder of Normal Forms ... 37
6.4 First Normal Form (1NF) ... 37
6.5 Worked 1NF Example ... 37
7. Second Normal Form (2NF) ... 39
7.1 Prime and Non-Prime Attributes ... 39
7.2 The 2NF Rule: Full Functional Dependency ... 39
7.3 Partial Dependency and How to Remove It ... 39
7.4 Worked 2NF Example ... 41
8. Third Normal Form (3NF) ... 43
8.1 The 3NF Rule ... 43
8.2 Transitive Dependency in a Real Relation ... 43
8.3 Worked 3NF Example ... 44
9. Boyce-Codd Normal Form (BCNF) ... 46
9.1 Why 3NF Is Not Always Enough ... 46
9.2 The BCNF Rule ... 46
9.3 Worked BCNF Example: Employees and Departments ... 47
9.4 BCNF and Lossless Join ... 48
10. Fourth Normal Form (4NF) ... 49
10.1 Multivalued Dependency Revisited ... 49
10.2 The 4NF Rule ... 49
10.3 Worked 4NF Example: Students ... 50
10.4 Worked 4NF Example: Subject, Lecturer, Books ... 51
11. Fifth Normal Form (5NF) and Join Dependency ... 53
11.1 The 5NF Rule ... 53
11.2 Worked 5NF Example: Agent, Company, Product ... 53
11.3 Why 5NF Is Largely of Academic Interest ... 55
12. Lossless Decomposition ... 57
12.1 What Lossless Means ... 57
12.2 The Lossless-Join Test ... 57
12.3 Worked Test on a Student Relation ... 58
12.4 Dependency Preservation ... 58
13. A Complete Worked Normalization Case Study ... 60
13.1 The Unnormalized Relation ... 60
13.2 Step 1: To 1NF ... 61
13.3 Step 2: To 2NF ... 61
13.4 Step 3: To 3NF ... 62
13.5 Step 4: To BCNF ... 63
13.6 Step 5: To 4NF ... 63
13.7 The Final Schema ... 63
14. Denormalization ... 65
14.1 Definition and Motivation ... 65
14.2 A Concrete Example ... 65
14.3 Benefits and Trade-Offs ... 66

PART 4 — Database Security ... 68
15. Database Security: Meaning, Importance and Scope ... 69
15.1 Definition ... 69
15.2 The Five Things Security Must Protect ... 69
15.3 The Usability Paradox ... 70
15.4 Why Database Security Matters ... 70
16. Threats and the CIA Triad ... 72
16.1 What a Threat Is ... 72
16.2 Loss of Integrity, Availability and Confidentiality ... 72

16.3 Threat Categories ... 73
17. Levels of Data Security, Physical and OS Security ... 75
17.1 Types of Security Issues ... 75
17.2 The Six Levels of Data Security ... 75
17.3 Physical-Level Security ... 76
17.4 Operating-System-Level Security ... 76
18. Encryption and Key Management ... 78
18.1 Why Encrypt ... 78
18.2 Storage-Level Encryption and Key Management ... 78
18.3 Practical Guidance from the Source ... 79
19. Authentication, Authorization and Access-Control Models ... 80
19.1 Authentication versus Authorization ... 80
19.2 Discretionary Access Control (DAC) ... 81
19.3 Mandatory Access Control (MAC) ... 81
19.4 Role-Based Access Control (RBAC) ... 82
19.5 Comparing the Three Models ... 82
20. Network and Human-Level Security ... 83
20.1 Network-Level Threats ... 83
20.2 Passwords ... 84
20.3 Smartcards ... 84
21. Control Methods of Database Security ... 86
21.1 The Six Control Measures ... 86
21.2 Authentication ... 86
21.3 Access Control ... 87
21.4 Inference Control ... 87
21.5 Flow Control and Covert Channels ... 88
21.6 Statistical Database Security ... 88
21.7 Encryption ... 88
21.8 Threat-to-Control Mapping ... 89
22. The DBA, Privileges and Statistical Database Security ... 90
22.1 Who the DBA Is ... 90
22.2 DBA Responsibilities ... 90
22.3 The DBA / Superuser Account ... 90

PART 5 — PL/SQL Control Structures ... 92
23. PL/SQL Control Structures: Overview ... 93
23.1 What Control Structures Are For ... 93
23.2 The Three Categories ... 93
23.3 The Anonymous Block ... 93
23.4 SET SERVEROUTPUT and the Substitution Variable ... 94
24. Conditional Control: IF and CASE ... 96
24.1 IF-THEN ... 96
24.2 IF-THEN-ELSE ... 96
24.3 IF-THEN-ELSIF ... 96
24.4 IF-THEN-ELSIF-ELSE ... 97
24.5 The CASE Statement ... 98
24.6 Worked Conditional Programs ... 98
25. Iterative Control: LOOP, WHILE and FOR ... 103
25.1 The Basic LOOP ... 103
25.2 EXIT and EXIT-WHEN ... 104
25.3 Loop Labels and Nested Loops ... 105
25.4 WHILE-LOOP ... 106
25.5 Building a Loop-That-Runs-At-Least-Once ... 106

25.6 FOR-LOOP and REVERSE 107
25.7 Worked Iterative Programs 108
26. Sequential Control: GOTO and NULL 112
26.1 The GOTO Statement 112
26.2 Worked GOTO Programs 112
26.3 Restrictions on GOTO 114
26.4 The NULL Statement 115

PART 6 — PL/SQL Cursors 118
27. Cursors: Concept and Implicit Cursors 119
27.1 What a Cursor Is 119
27.2 The Context Area and the Active Set 119
27.3 Implicit Cursors 120
27.4 Implicit Cursor Attributes 120
27.5 Worked Implicit Cursor Program 121
28. Explicit Cursors and Cursor Loops 123
28.1 Declaring, Opening, Fetching and Closing 123
28.2 The Cursor Life Cycle 124
28.3 Worked Explicit Cursor Program 124
28.4 Implicit versus Explicit Cursors 125
28.5 Cursor Errors and Exam Traps 126

PART 7 — SQL Views 128
29. SQL Views: Concept and Creation 129
29.1 Definition and Base Tables 129
29.2 Benefits of Views 130
29.3 The Sample Tables Used Throughout 130
29.4 Creating a View from a Single Table 131
29.5 Creating a View from Multiple Tables 132
29.6 Querying a View 133
30. Modifying, Dropping and Updating Views 135
30.1 Dropping a View 135
30.2 Creating or Replacing a View 135
30.3 Conditions Required to Update a View 136
30.4 Inserting a Row Through a View 137
30.5 Deleting a Row Through a View 138
30.6 WITH CHECK OPTION 139
30.7 Uses of a View 140

PART 8 — Revision and Examination 142
31. Consolidated Comparison Tables and Formula Sheet 143
31.1 Key Comparison Tables 143
31.2 Normal-Form Summary Table 144
31.3 Dependency Rules and Axioms 144
31.4 PL/SQL Control-Structure Summary 145
31.5 Cursor and View Summary 146
31.6 Security Summary 147
32. Question Bank with Answers 149
32.1 Two-Mark Questions 149
32.2 Five-Mark Questions 152
32.3 Ten-Mark / Descriptive Questions 154
32.4 Output-Prediction Questions 155
32.5 Debugging Questions 158
32.6 Normalization Problems with Step-by-Step Solutions 160

33. MCQ Bank with Answer Key 163
34. Viva Questions 167
35. Last-Minute Revision Sheet 171
36. Source Coverage Checklist and References 175

Note: page numbers in the Contents above are printed page numbers. The two front-matter pages that precede it are labelled **i** and **ii**; the cover and the eight Part-divider pages carry no printed number, so the printed sequence 1–182 skips them.

PART 1

Relational Foundations and Database Keys

Relations, tuples and attributes · why a database needs keys · the ten keys of the source, one at a time · classification, confusions and exercises

Source: UNIT 2, Chapter 2.1.7 (Keys in detail) — all ten key types, the Student and Employee schemas, and the Department / Employee foreign-key example.

1. Relational Model Foundations and the Idea of a Key

Part 1 • the vocabulary you must not get wrong

Every later topic in these notes — dependency, normalization, join, view — is built on one small piece of vocabulary. Learn it precisely here and the rest of the book reads easily. The source opens the key chapter with a note that is worth taking seriously: “The terms ‘relation’ and ‘table’ are used interchangeably; the terms ‘tuple’ and ‘record’ are used interchangeably. So, don't get confused!”

1.1 Relation, Tuple, Attribute and Domain

A **relation** is a table of values arranged in rows and columns, where the order of the rows and the order of the columns do not carry meaning. Each row is a **tuple** (a record). Each column is an **attribute** (a field), and each attribute draws its values from a permitted set called its **domain**.

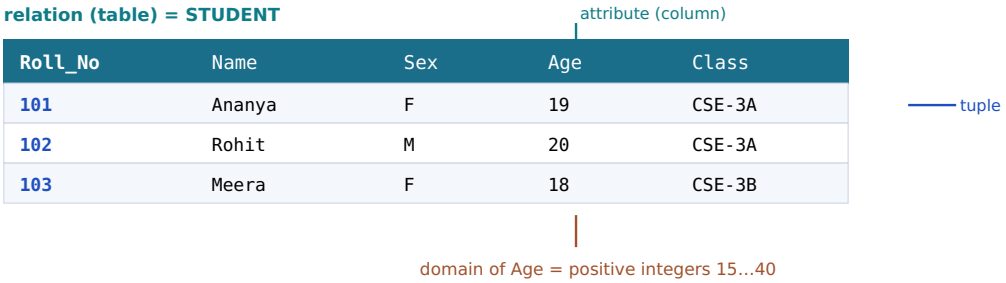


Figure 1.1 — A relation is a set of tuples; each column is an attribute with a domain. The row order and column order are irrelevant to the meaning.

DEFINITION

A **relation** is a set of tuples over a fixed set of attributes. A **tuple** is one row. An **attribute** is one column name. The **degree** of a relation is its number of attributes; its **cardinality** is its number of tuples.

1.2 Why a Database Needs Keys

The source gives three reasons, and they are the right three. A real table may hold thousands of rows, and some of those rows may look identical in every column a user cares about. A key is what makes a row addressable anyway.

WHY KEYS EXIST (SOURCE REASONS)

1. Unique identification. Keys let you identify any row of data in a table even when records appear duplicated in ordinary columns.

2. Relationship building. Keys allow you to establish and identify the relation between two tables.

3. Identity and integrity. Keys help you enforce identity and integrity in those relationships.

1.3 The Vocabulary Trap: Relation = Table, Tuple = Record

Because the source's slides mix the formal and informal words in the same paragraph, the following table is worth keeping beside you while reading Part 1.

Table 1.1 — Formal and informal vocabulary, as used interchangeably in the source.

Formal term	Informal term	Meaning
Relation	Table	A named set of tuples over fixed attributes
Tuple	Row / record	One entry of the relation
Attribute	Column / field	One named property
Domain	Data type / value set	The permitted values of an attribute
Degree	Number of columns	Count of attributes
Cardinality	Number of rows	Count of tuples

COMMON MISTAKE

Writing “relation” to mean “relationship”. In the relational model a **relation** is a table. The link between two tables is a **relationship**, and it is implemented by a **foreign key**. Mixing the two words in an answer costs marks in viva.

2. The Ten Keys in Detail

Part 1 • the source's ten keys, each defined, illustrated and compared

The source lists ten keys in DBMS. Two of them — **compound key** and **composite key** — are so close in meaning that the source itself defines "composite key" twice, once using the wording that belongs to a surrogate key. Section 2.7 untangles this and flags it plainly rather than quietly rewriting it.

Table 2.1 — The ten keys of the source, in the order the source presents them.

#	Key	One-line job
1	Super key	Any set of attributes that identifies a tuple uniquely
2	Candidate key	A minimal super key
3	Primary key	The candidate key the designer chooses
4	Alternate key	A candidate key that was not chosen as primary
5	Foreign key	An attribute whose values must exist as a key elsewhere
6	Partial key	Identifies a group of related tuples, not one tuple
7	Composite key	A primary key made of two or more attributes
8	Unique key	Unique and non-updatable, but may be NULL
9	Surrogate key	A system-generated artificial identifier
10	Secondary key	Used for indexing and faster searching

Throughout this chapter the notes use two running schemas. Learn them once and every example below is immediately readable.

Table 2.2 — The two running schemas (source: Student schema and Employee schema).

`Student ( roll, name, sex, age, address, class, section )`
`Emp ( Emp_ID, Emp_SSN, Emp_Num, Emp_Name, Dept_ID, ... )`

2.1 Super Key

> **DEFINITION**
>
> A **super key** is a set of attributes that can identify each tuple uniquely in the given relation. A super key is not restricted to any particular number of attributes — it may consist of any number of attributes, and it may carry extra attributes that are not needed for unique identification.

The source's own caution is the key sentence: all the attributes in a super key are sufficient to identify each tuple uniquely, but all of them may not be necessary. A super key may therefore contain a smaller set that would already have done the job.

Example 1 — the Student schema

Given `Student ( roll, name, sex, age, address, class, section )`, each of the following sets is a super key, because each set uniquely identifies every student:

- `( roll, name, sex, age, address, class, section )` — the whole tuple
- `( class, section, roll )` — a minimal one
- `( class, section, roll, sex )` — minimal set plus an extra attribute
- `( name, address )` — unique in this schema

Example 2 — the Employee schema

In the source's Employee example, `Emp_SSN` and `Emp_Num` are each super keys.

> **COMMON MISTAKE**
>
> Assuming that every combination of columns is a super key. Only sets that actually guarantee uniqueness qualify. In the Student schema, `( class, section )` alone is **not** a super key, because many students share a class and section.

2.2 Candidate Key

> **DEFINITION**
>
> A **candidate key** is a **minimal super key** — a set of the fewest possible attributes that can identify each tuple uniquely. Equivalently: a super key from which no attribute can be removed without destroying uniqueness.

From the same Student schema, the candidate keys are `( class, section, roll )` and `( name, address )`. The set `( class, section, roll, sex )` is a super key but **not** a candidate key, because sex can be removed and the set still identifies a student.

Properties of a candidate key (from the source)

Property	Consequence
Must contain unique values	No two tuples share a candidate-key value
May have multiple attributes	It can be composite
Must not contain NULL	A NULL would be “unknown”, so it cannot identify
Should contain the minimum fields needed	Minimality is what separates it from a super key
Uniquely identifies each record	That is its whole purpose

> **EXPLANATION — WHY MINIMALITY MATTERS**
>
> Minimality is not fussiness. A candidate key is the smallest piece of information you must supply to pin down exactly one row. If you store a larger set as “the” key, you force users to supply redundant data and you allow inconsistencies: the extra columns could be changed to contradict the identifying ones.

Example 2 — a table with three candidate keys

In the source's student table, `Stud_ID`, `Roll_No` and `Email` are all candidate keys; each one independently identifies a student record.

FACTS TO MEMORISE

Every table has **at least one** candidate key. A table may have **several** candidate keys but only **one** primary key. Every attribute that appears in some candidate key is called a **prime attribute**; all other attributes are **non-prime**. This prime/non-prime split is the language of 2NF and 3NF in Part 3.

2.3 Primary Key

DEFINITION

A **primary key** is the candidate key that the database designer selects and implements. It is the official identifier of the relation.

Rules for defining a primary key (from the source)

1. Two rows cannot have the same primary-key value — it must be unique.
2. Every row must have a primary-key value — it cannot be NULL.
3. The value in a primary-key column can never be modified or updated while any foreign key refers to it.
4. A relation is allowed to have only one primary key.

COMMON MISTAKE

"A table can have two primary keys." No. A table can have several candidate keys; exactly one of them becomes the primary key, and the rest become alternate keys. Two separate primary keys would give two conflicting notions of identity.

2.4 Alternate Key

DEFINITION

An **alternate key** is a candidate key that is left unimplemented after the primary key has been chosen. All the candidate keys that are not the primary key are alternate keys.

In the source's example, `Stud_ID`, `Roll_No` and `Email` are all qualified to be primary key. Because `Stud_ID` is chosen, `Roll_No` and `Email` become the alternate keys.



Figure 2.1 — Three candidate keys; one is implemented as the primary key and the remaining two become alternate keys.

2.5 Foreign Key and Referential Integrity

DEFINITION

An attribute **X** is a **foreign key** referring to attribute **Y** when the values of X must come from the values that actually exist in Y. The relation containing Y is the **referenced relation** (also called the master or primary table); the relation containing X is the **referencing relation** (also called the foreign table). X may have a name different from Y.

The source's example uses two relations:

Employee (Emp_ID, Emp_Name, …, Dept_ID)	Department (Dept_ID, Dept_Name)
t_dept takes only values present in **dept_no** of Department	Dept_ID is the primary key here

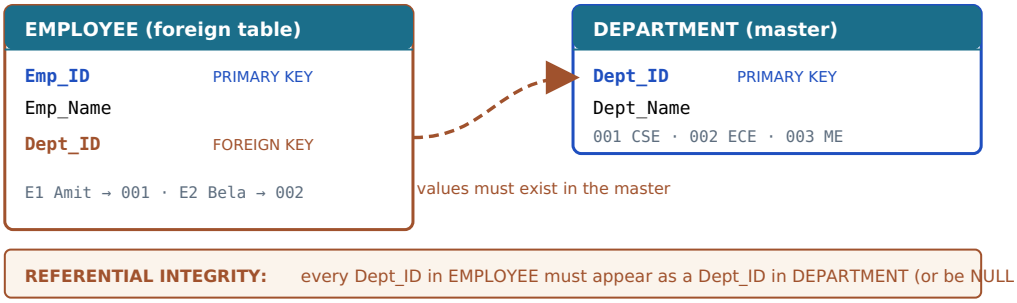


Figure 2.2 — A foreign key creates the relationship between two tables and is the mechanism of referential integrity.

Notes on the foreign key (from the source)

- A foreign key references the primary key of the referenced table.
- It can take only those values that are present in the primary key of the referenced relation.
- It may have a name different from that of the primary key it references.
- It **can** take the NULL value (meaning “no department assigned yet”).
- There is no restriction that a foreign key be unique — in fact it is usually not unique.

2.6 Partial Key

DEFINITION

A **partial key** is a key that cannot identify all the records of a table uniquely. It can, however, select a bunch of related tuples from the table.

The source's example makes this concrete:

Table 2.3 — Department (Emp_no, Dependent_name, Relation).

Emp_no	Dependent_name	Relation
E1	Suman	Mother
E1	Ajay	Father
E2	Vijay	Father
E2	Ankush	Son

Using `Emp_no` alone we cannot identify a single tuple — E1 appears twice — but we can select the group of tuples belonging to E1. `Emp_no` is therefore a partial key. In entity–relationship modelling this is exactly the "partial key" of a weak entity; the full identifier is the partial key together with the owner's key.

2.7 Composite Key and Compound Key

DEFINITION

A **composite key** is a primary key made up of two or more attributes rather than a single attribute. The combination guarantees uniqueness even though each individual column may repeat values.

Table 2.4 — A composite key in action. Neither column is unique alone; the pair is.

Student_Name	Course_Code
Rahul	CS101
Priya	CS101
Rahul	MA201
Sana	CS101

Student_Name repeats ("Rahul" twice) and Course_Code repeats ("CS101" three times), but the pair `( Student_Name, Course_Code )` is unique, because a student cannot enrol in the same course twice. That pair is the composite primary key.

Composite versus compound — as the source draws the line

The source states the difference explicitly: any part of a **compound** key can be a foreign key, but the **composite** key may or may not be part of a foreign key. In other words, a compound key is a composite key in which each component is itself a foreign key to a different table. A many-to-many junction table such as `Enrolment ( Student_ID, Course_ID )` has a compound key, because both components are foreign keys.

FROM THE SOURCE — AN INCONSISTENCY HANDLED HONESTLY

The source's summary list defines **Composite Key** with the wording "An artificial key which aims to uniquely identify each record is called a surrogate key…" — that is the surrogate key definition pasted under the wrong heading, because the two sentences below it define "Surrogate Key" with the same words. Later in the same chapter, "Composite Key" is correctly defined as a primary key of multiple attributes. These notes follow the **later, correct** definition and treat the earlier line as a copy error in the source. Nothing else in the chapter depends on it.

2.8 Unique Key

DEFINITION

A **unique key** is unique for all records of the table, is non-updatable once assigned, and **may** contain a NULL value.

The source's example is the Aadhaar card number, and it is a good one because it exercises all three properties at once:

- It is **unique** for all citizens (tuples) of India (the table).
- If the card is lost and a duplicate copy is issued, the duplicate always carries the **same number** — so the value is **non-updatable**.
- Some citizens have no Aadhaar card yet, so for those rows the value is **NULL**.

EXPLANATION — UNIQUE KEY VERSUS PRIMARY KEY

The single practical difference is the NULL rule. A primary key forbids NULL and permits exactly one per table. A unique key permits NULL and a table may declare several of them. Both enforce uniqueness on the values that are present.

2.9 Surrogate Key

DEFINITION

A **surrogate key** is an artificial key created to identify each record uniquely. It is unique because it is generated when no natural primary key exists. It lends no meaning to the data, is usually an integer, and is generated just before the record is inserted.

The source's example is the mobile number of students in a class where every student owns a mobile phone. The source also lists the two situations in which surrogate keys are allowed:

- When no existing property has the characteristics required of a primary key.
- When the natural primary key is too big or too complicated.

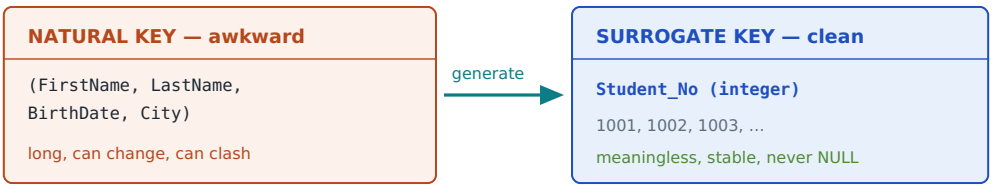


Figure 2.3 — A surrogate key replaces a complex or absent natural key with a generated integer. The original attributes are still stored, but no longer carry identity.

COMMON MISTAKE

Confusing a surrogate key with a unique key. A surrogate key is **artificial and updatable**; a unique key is typically a **natural, non-updatable** attribute (Aadhaar number, passport number). The source's own definitions of the two make this distinction by listing “updatable / not updatable” as the separating property.

2.10 Secondary Key

DEFINITION

A **secondary key** is a key required for indexing, so that searching is better and faster.

Unlike the previous nine keys, a secondary key says nothing about uniqueness. It is a performance device: it is the attribute you commonly search on, and an index is built over it so the DBMS can avoid a full table scan. In the Student schema, if the query “find all students of class CSE-3A” is common, `class` would be a secondary key.

2.11 Key Classification at a Glance

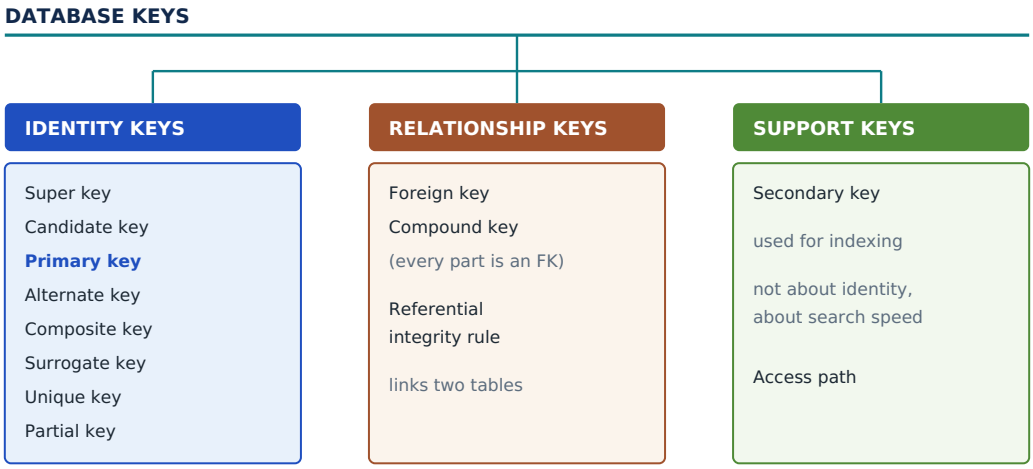


Figure 2.4 — The ten source keys grouped by the job they do. Only the first group says “this row is unique”; the second says “this row belongs to”; the third says “search here fast”.

2.12 Commonly Confused Pairs

Table 2.5 — Side-by-side comparison of the pairs students most often mix up.

Pair	First	Second	The decisive test
Super key / Candidate key	Any set that identifies uniquely; may contain redundant attributes	A super key with no removable attribute	Can you delete a column and keep uniqueness? If yes it was only a super key.
Candidate key / Primary key	Every minimal identifier that exists	The one minimal identifier that was chosen	Count: a table may have many candidate keys, exactly one primary key.
Alternate key / Secondary key	A candidate key not chosen as primary	Any attribute used for indexing	Is it a minimal identifier? Alternate yes; secondary not necessarily.
Composite key / Surrogate key	Two or more real attributes combined	One artificial generated attribute	Does the key carry meaning from the data? Composite yes; surrogate no.
Primary key / Unique key	Unique, NOT NULL, one per table	Unique, NULL allowed, many per table	May it be NULL? Unique yes; primary no.
Foreign key / Primary key	Values must exist in another table; may repeat; may be NULL	Values define identity; never repeat; never NULL	Which table “owns” the value?

2.13 Key Identification Exercises

EXERCISE 1

Schema: `Student ( roll, name, sex, age, address, class, section )`. Assume no two students share a name, and a name is unique across the table.

(a) List all candidate keys. **(b)** Is `( class, section, roll, sex )` a candidate key? **(c)** Name two super keys that are not candidate keys.

Answer. (a) `( class, section, roll )` and `( name, address )` — both are minimal. (b) No: it is a super key, but sex is removable, so it is not minimal. (c) The whole tuple, and `( class, section, roll, sex )`.

EXERCISE 2

Schema: `Employee ( Emp_ID, Emp_Name, Passport_Number, License_Number, Dept_ID )` and `Department ( Dept_ID, Dept_Name )`.

(a) Identify the primary key, all candidate keys and all alternate keys of Employee. **(b)** Identify the foreign key. **(c)** Give one composite key and one super key that is not a candidate key.

Answer. (a) Primary key `Emp_ID`; candidate keys `Emp_ID`, `Passport_Number`, `License_Number`; alternate keys `Passport_Number` and `License_Number`. (b) `Dept_ID` in Employee references `Dept_ID` in Department. (c) Composite: `( Emp_Name, Emp_ID )` as the source gives it. Super key but not candidate: `( Emp_ID, Emp_Name )`.

EXERCISE 3

Schema: `Enrolment ( Student_ID, Course_ID, Semester )`, where a student takes each course once, and both `Student_ID` and `Course_ID` reference other tables.

Question. Is `( Student_ID, Course_ID )` a composite key or a compound key? Justify.

Answer. It is a **compound** key, because each of its parts is itself a foreign key to a different table. Every compound key is a composite key, but the reverse is not true: a composite key whose parts are ordinary attributes (not foreign keys) is composite only.

Chapter summary

SUMMARY — PART 1

- A **super key** identifies uniquely but may carry extra attributes; a **candidate key** is a minimal super key; the chosen candidate key is the **primary key**; the rest are **alternate keys**.
- A **foreign key** enforces referential integrity between a referencing and a referenced relation; it may repeat and may be NULL.
- A **partial key** identifies a group, not a tuple. A **composite key** is a multi-attribute primary key; a **compound key** is a composite key whose parts are all foreign keys.
- A **unique key** allows NULL; a **surrogate key** is artificial and generated; a **secondary key** exists for indexing, not identity.
- Prime attributes are those appearing in some candidate key — the vocabulary that drives 2NF and 3NF.

PART 2

Functional Dependencies

What a dependency asserts · determinants and dependents · trivial, non-trivial and completely non-trivial · Armstrong's axioms and closure · transitive, multivalued and join dependencies

Source: UNIT 2, Chapter 2.1.1 (Functional dependencies) — the Department example, the three forms of dependency, Armstrong's axioms, the Book/Author example, and the Employee join-dependency and Student multivalued-dependency tables.

3. Functional Dependencies: Meaning and Notation

Part 2 • the single idea that makes normalization possible

Normalization is not a set of arbitrary rules. It is a set of tests applied to functional dependencies. If you cannot see the dependencies hiding in a table, you cannot normalize it — you can only memorise the steps. This chapter builds the ability to see them.

3.1 What a Functional Dependency Says

DEFINITION

A **functional dependency** (FD) is a constraint between two sets of attributes of a relation. If X and Y are sets of attributes of relation R, then **X → Y** ("X determines Y", or "Y is functionally dependent on X") means that any two tuples that agree on X must also agree on Y.

Formally, as the source states it: for any two tuples t1 and t2 in R, if t1[X] = t2[X] then t1[Y] = t2[Y].

Read that sentence slowly, because it is the whole subject. The arrow does **not** mean "causes", it does not mean "is calculated from", and it does not mean "appears to the left". It means: in every legal state of this relation, the same X value cannot be paired with two different Y values.

The source's own example is the smallest possible one. Consider a `Department` relation with attributes `DeptId` and `DeptName`, where `DeptId` is the primary key:

Table 3.1 — The source's Department relation.

DeptId	DeptName
001	Finance
002	Marketing
003	HR

Because `DeptId` uniquely identifies the department, it also uniquely determines the department name. The dependency is

`DeptId → DeptName`

The source phrases the justification memorably: if you want to know the department name, then at first you need to have the DeptId. That "you must know X first" test is a reliable practical way to spot a dependency in a table.

FROM THE SOURCE — A WORDING SLIP

The source then writes "DeptId is functionally dependent on DeptName" and still labels the dependency `DeptId → DeptName`. The sentence is backwards: in `DeptId → DeptName` it is **DeptName** that is functionally dependent on **DeptId**, not the other way round. The notation in the source is correct and is the one used throughout these notes; only that one explanatory sentence is inverted.

3.2 Determinant and Dependent

Every dependency has two named parts, and using the names correctly makes dependency reasoning far easier.

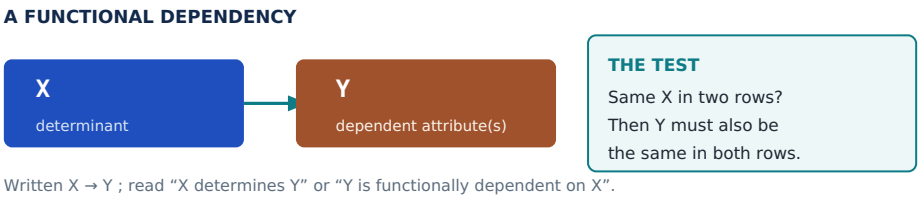


Figure 3.1 — X is the determinant, Y is the determined set. The dependency is a statement about every legal state of the relation, not about the rows you happen to see.

Table 3.2 — The vocabulary of dependencies.

Term	Meaning	In DeptId → DeptName
Determinant	The left-hand set X	`DeptId`
Dependent	The right-hand set Y	`DeptName`
Functional dependency	The constraint X → Y	DeptId determines DeptName
Determines	Verb for the left-hand set	DeptId determines DeptName
Is determined by	Verb for the right-hand set	DeptName is determined by DeptId

3.3 Reading a Dependency Correctly

COMMON MISTAKE

Reading the arrow as a causal or computational relation. A dependency is a **constraint on allowed data**. The table below shows sentences people write, and which of them are correct.

Table 3.3 — Correct and incorrect readings of X → Y.

Sentence	Verdict	Why
“X determines Y.”	**Correct**	Standard reading of the arrow.
“Y is functionally dependent on X.”	**Correct**	Standard reading of the arrow.
“X depends on Y.”	**Wrong**	Reverses the direction. X is the determinant.
“Y causes X.”	**Wrong**	The arrow is a data constraint, not causation.
“X is computed from Y.”	**Wrong**	Computation is irrelevant to the constraint.

3.4 Finding Dependencies in a Table

Given only a table of rows, a dependency can be **disproved** but never **proved** — the rows you can see are a sample of all legal states. The practical procedure is to look for two rows with the same X and different Y; if they exist, X → Y is false.

WORKED STEP — DISPROVING A DEPENDENCY

Take the source's Book relation (used again in Section 5.1):

Book	Author	Author_age
Game of Thrones	George R. R. Martin	66
Harry Potter	J. K. Rowling	49
Dying of the Light	George R. R. Martin	66

Claim: `Author → Book`. **Test:** look for two rows with the same Author and different Book. Rows 1 and 3 both have author "George R. R. Martin" but different books. The claim is **false**. This is exactly the step that makes `Book → Author_age` a transitive dependency later in Section 5.1, and transitive dependencies are what 3NF removes.

3.5 Trivial, Non-Trivial and Completely Non-Trivial

The source gives three forms of functional dependency. The distinction looks pedantic until you write an exam answer about BCNF, where the words "trivial" and "non-trivial" decide whether a dependency violates the form.

DEFINITIONS — THE THREE FORMS

Trivial: for `A → B`, B is a **subset of** A. The dependency tells you nothing new.

Non-trivial: for `A → B`, B is **not a subset of** A.

Completely non-trivial: for `A → B`, A ∩ B is **empty** (the two sides share no attribute at all).

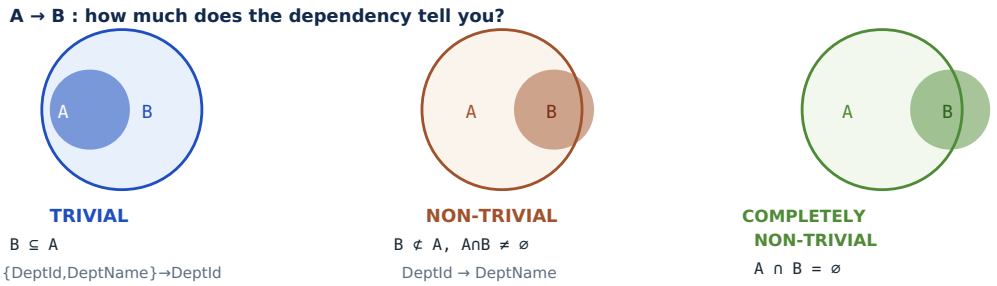


Figure 3.2 — The three forms of functional dependency, drawn as set relationships. Note that every completely non-trivial dependency is also non-trivial.

Source examples, checked

Table 3.4 — The source's examples for each form.

Form	Example	Why it has that form
Trivial	`{ DeptId, DeptName } → DeptId`	The right side DeptId is a subset of the left side, so it is trivially satisfied.
Non-trivial	`DeptId → DeptName`	DeptName is not a subset of DeptId, so the dependency genuinely constrains the data.
Completely non-trivial	`DeptId → DeptName`	The same dependency also has empty intersection between its sides, so it is completely non-trivial as well.

EXPLANATION — THE NESTING TO REMEMBER

The three classes are nested, not separate categories. The example `DeptId → DeptName` is simultaneously non-trivial and completely non-trivial, because its two sides share no attribute. A dependency such as `{A, B} → {B, C}` is non-trivial (C is not in the left side) but **not** completely non-trivial, because the left and right sides share B.

3.6 Why Functional Dependencies Matter

The source lists five advantages of knowing about functional dependencies, and each one points at a later chapter of these notes.

Table 3.5 — Advantages of functional dependencies, with the chapter where each is used.

Advantage (source)	What it means in practice	Used in
Avoids data redundancy	The same fact is not repeated at multiple locations	Ch. 6, 8
Maintains data quality	Constraints prevent contradictory states	Ch. 6
Defines meaning and constraints	The FDs are the business rules of the schema	Ch. 4
Identifies bad designs	Anomaly-prone tables reveal themselves as odd FDs	Ch. 7, 9
Reveals facts about the design	Closure and keys can be derived, not guessed	Ch. 4

4. Armstrong's Axioms and Reasoning with Dependencies

Part 2 • a small set of rules from which every valid dependency can be derived

A set F of functional dependencies that you write down for a schema is never the whole truth: it implies others. Armstrong's axioms are the inference rules that let you compute those implied dependencies. They were developed by **William Armstrong in 1974**, as the source notes.

4.1 The Three Primary Axioms

ARMSTRONG'S THREE AXIOMS

Reflexivity. If Y is a subset of X, then X → Y. The source states it as "A → B, if B is a subset of A". This is the rule that makes trivial dependencies true.

Augmentation. If X → Y, then XZ → YZ for any set of attributes Z. The source writes it as "AC → BC, if A → B". Adding the same attributes to both sides preserves the dependency.

Transitivity. If X → Y and Y → Z, then X → Z. The source gives it as "If A → B and B → C, then A → C, i.e. a transitive relation".

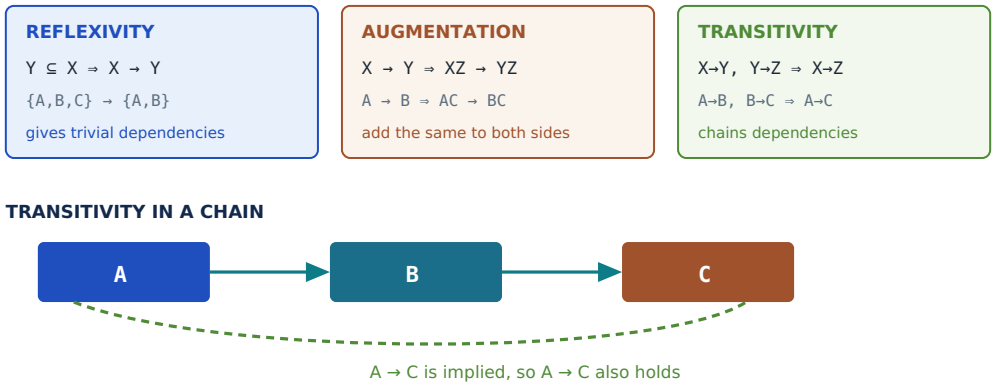


Figure 4.1 — Armstrong's three primary axioms. From these three, all further rules (Section 4.2) can be derived.

4.2 Derived Rules

Three further rules are commonly used because they shorten derivations. They are derived, not assumed: each can be proved from the three axioms.

Table 4.1 — Derived rules, all provable from reflexivity, augmentation and transitivity.

Rule	Statement	Derivation sketch
Union (additivity)	If X → Y and X → Z, then X → YZ	Augment X → Y to X → XY; augment X → Z to XY → YZ; transitivity gives X → YZ.
Decomposition (projectivity)	If X → YZ, then X → Y and X → Z	Y ⊆ YZ, so by reflexivity YZ → Y; transitivity with X → YZ gives X → Y.
Pseudotransitivity	If X → Y and WY → Z, then WX → Z	Augment X → Y to WX → WY; transitivity with WY → Z gives WX → Z.

COMMON MISTAKE — DECOMPOSITION IS ONE-WAY

Decomposition works on the right side, never on the left. From `A → BC` you may conclude `A → B` and `A → C`. But from `AB → C` you may **not** conclude `A → C` or `B → C`. Splitting the left side is an invalid inference, and it is one of the most frequent mistakes in normalization answers.

4.3 Closure of a Set of Attributes

To decide whether a given set of attributes is a candidate key, you need to know everything it can determine. That is its **closure**.

DEFINITION

The **closure** of a set of attributes X under a set of dependencies F, written $\mathbf{X^+}$, is the set of all attributes that can be functionally determined by X using the axioms. If X^+ contains every attribute of the relation, then X is a super key; if X is also minimal, X is a candidate key.

ALGORITHM — COMPUTING X^+

1. Start with the result set = X.
2. Scan the dependencies in F. If the left side of a dependency is already contained in the result set, add its right side to the result set.
3. Repeat step 2 until no new attribute is added.
4. The final result set is X^+.

4.4 Worked Closure Computations

WORKED STEP — A FULL CLOSURE

Relation R(A, B, C, D, E) with dependencies:

```
F = { A → B,  B → C,  CD → E }
```

Task: compute A^+ and decide whether A is a candidate key.

Pass	Result set	Dependency used	Reason
start	{A}	—	Initialise with A
1	{A, B}	A → B	Left side {A} ⊆ result
2	{A, B, C}	B → C	Left side {B} ⊆ result
3	{A, B, C}	CD → E	Left side {C,D} ⊄ result (D missing)
stop	{A, B, C}	—	No new attribute added

Result: A^+ = {A, B, C}. Since D and E are missing, **A is not a super key** and therefore not a candidate key.

WORKED STEP — FINDING A CANDIDATE KEY BY CLOSURE

Same relation R(A, B, C, D, E) with F = { A → B, B → C, CD → E }.

Task: test `{A, D}`.

Pass	Result set	Dependency used
start	`{A, D}`	—
1	`{A, D, B}`	A → B
2	`{A, D, B, C}`	B → C
3	`{A, D, B, C, E}`	CD → E

Result: $(AD)^+ = \{A, B, C, D, E\}$ — every attribute. So `{A, D}` is a super key. Is it minimal? Test the proper subsets: $A^+ = \{A, B, C\}$ (missing D, E) and $D^+ = \{D\}$ (missing A, B, C, E). Neither subset determines everything, so `{A, D}` is a **candidate key**.

CORRECT RESULT — THE SHORTCUT

Attributes that appear on the **right side only** can never be part of a candidate key. Attributes that appear on the **left side** (or nowhere) must be part of every candidate key. Here E appears only on the right, so E is excluded from keys; A and D appear on the left, so both must be present. Starting with {A, D} is therefore the right first guess — and the closure above confirms it.

4.5 Equivalence of Dependency Sets

Two dependency sets F and G are **equivalent** when every dependency in F is implied by G and every dependency in G is implied by F. Equivalence is what lets a designer replace a messy set of dependencies with a cleaner, smaller one without changing the meaning of the schema — and it is the theoretical justification for a lossless decomposition, covered in Chapter 12.

EXPLANATION — WHY THIS IS WORTH KNOWING

When you decompose a table in Part 3, the projections of the original dependencies onto the new tables form a new set G. If F and G are equivalent, the decomposition has preserved every constraint. If they are not, the decomposition has quietly lost a business rule — a fact worth stating in an exam answer.

5. Special Dependencies: Transitive, Multivalued and Join

Part 2 • the three dependency types that drive 3NF, 4NF and 5NF

Ordinary functional dependencies are not the only constraint a table can violate. Three richer kinds of dependency each have a normal form aimed at them. Understanding them here makes Part 3 read as one connected argument rather than six unrelated definitions.

5.1 Transitive Dependency

DEFINITION

A functional dependency is **transitive** when it is formed indirectly by two other dependencies. The source's precise statement: `X → Z` is transitive if all three of the following hold:

- `X → Y`
- `Y does not → X` (so Y is not itself a determinant of X)
- `Y → Z`

The source adds a useful scope note: a transitive dependency can only occur in a relation of three or more attributes, and it is the dependency that 3NF removes.

The source's Book example

Table 5.1 — The source's transitive-dependency example.

Book	Author	Author_age
Game of Thrones	George R. R. Martin	66
Harry Potter	J. K. Rowling	49
Dying of the Light	George R. R. Martin	66

Check the three conditions:

1. `{Book} → {Author}` — knowing the book tells you the author. ✔
2. `{Author} does not → {Book}` — an author may write many books. ✔
3. `{Author} → {Author_age}` — knowing the author tells you the age. ✔

Therefore, by transitivity, `{Book} → {Author_age}` holds: if you know the book name, you can know the author's age. That indirect dependency is a transitive dependency, and it is exactly the kind of dependency 3NF breaks.

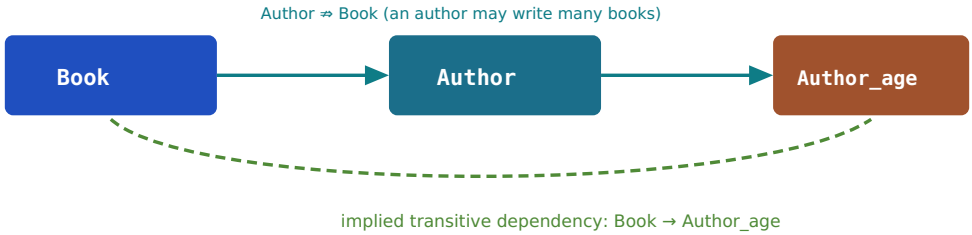


Figure 5.1 — A transitive dependency is a two-hop chain. It is harmless informationally but dangerous in a stored table, because the same author's age is repeated for every book.

COMMON MISTAKE

Calling any two-dependency chain "transitive". The middle condition matters. If `Y → X` also held, then X and Y would determine each other and the chain would simply be a pair of equivalent determinants, not a transitive dependency in the 3NF sense.

5.2 Multivalued Dependency

DEFINITION

A **multivalued dependency** (MVD) occurs when the existence of one or more rows in a table implies the existence of one or more other rows in the same table. If a table has attributes P, Q and R, and Q and R are two independent multivalued facts about P, we write `P ↠ Q` and `P ↠ R`.

The source states the three conditions for a table to have a multivalued dependency, and they are worth listing exactly as given:

1. For a dependency `A → B`, if for a single value of A multiple values of B exist, then the table may have a multivalued dependency.
2. The table must have **at least three columns** for a multivalued dependency to exist.
3. For a relation R(A, B, C), if there is a multivalued dependency between A and B, then **B and C must be independent of each other**.

The source's Student example

Table 5.2 — The source's multivalued-dependency example (as given).

StudentName	CourseDiscipline	Activities
Amit	Mathematics	Singing
Amit	Mathematics	Dancing
Yuvraj	Computers	Cricket
Akash	Literature	Dancing
Akash	Literature	Cricket
Akash	Literature	Singing

Amit takes Mathematics and has two activities; Akash takes Literature and has three. The courses and the activities are two independent lists about the same student — a course choice does not depend on an activity choice. So `StudentName ↠ CourseDiscipline` and `StudentName ↠ Activities` both hold. This

relation violates 4NF, and the fix (decomposing into `StudentCourse` and `StudentActivities`) is worked in Section 10.3.

FROM THE SOURCE — A REPEATED-ROW DEFECT

In the source's corrected tables, the row "Amit / Mathematics" is listed twice in `StudentCourse`, and "Akash / Literature" appears three times. In a true relation the rows of a table form a **set**, so duplicate tuples cannot exist: the correct decomposition is `StudentCourse = { (Amit, Mathematics), (Yuvraj, Computers), (Akash, Literature) }` and `StudentActivities = { (Amit, Singing), (Amit, Dancing), (Yuvraj, Cricket), (Akash, Dancing), (Akash, Cricket), (Akash, Singing) }`. The duplicates in the source appear to be a display artefact of the slide. The corrected version is used in Chapter 10.

5.3 Join Dependency

DEFINITION

A table is in **join dependency** if it can be recreated by joining multiple tables, each of which holds a subset of its attributes. A join dependency is a generalisation of a multivalued dependency, and it is the dependency that 5NF addresses: a relation is in 5NF only if it is already in 4NF and cannot be decomposed further without losing information.

The source's Employee example

Table 5.3 — The source's join-dependency example.

EmpName	EmpSkills	EmpJob (Assigned Work)
Tom	Networking	EJ001
Harry	Web Development	EJ002
Katie	Programming	EJ002

The source shows that this table can be decomposed into three tables — `EmployeeSkills` (EmpName, EmpSkills), `EmployeeJob` (EmpName, EmpJob) and `JobSkills` (EmpSkills, EmpJob) — and that joining those three reproduces the original table. The join dependency is

`{ (EmpName, EmpSkills), (EmpName, EmpJob), (EmpSkills, EmpJob) }`

FROM THE SOURCE — AN IMPORTANT QUALIFICATION

The source says of this three-table decomposition "therefore it is not in 5NF". Taken literally that is too strong: the three-table decomposition of this particular data happens to join back losslessly, but a table is only genuinely in violation of 5NF when the join dependency is **not** implied by the candidate keys — that is, when the constraint is a real business rule of the schema, not an accident of the rows shown. The source itself makes exactly this point in its 5NF chapter, where it distinguishes the version of the table that obeyed the tuple-formation rule (which was in 5NF) from the version that did not (which was not). These notes present the source's example as an illustration of what a join dependency is, and explain the key-implied test properly in Section 11.1.

5.4 Dependency Types and the Normal Forms They Drive

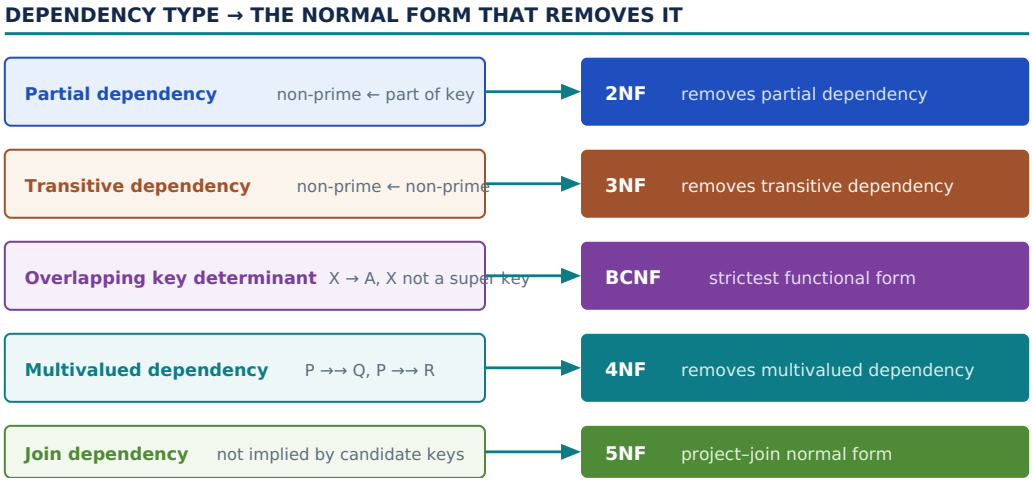


Figure 5.2 — Each normal form exists to remove one specific kind of dependency. The ladder is cumulative: 3NF requires 2NF, BCNF requires 3NF, and so on.

Chapter summary

SUMMARY — PART 2

- $X \to Y$ means equal X forces equal Y, in every legal state. X is the determinant, Y is the dependent set.
- Trivial: $B \subseteq A$. Non-trivial: $B \not\subset A$. Completely non-trivial: $A \cap B = \varnothing$.
- Armstrong's three axioms are reflexivity, augmentation and transitivity; union, decomposition and pseudotransitivity are derived from them.
- Closure X^+ is computed by repeatedly adding right sides whose left sides are already contained; if X^+ is everything and X is minimal, X is a candidate key.
- A **transitive** dependency is a chain $X \to Y \to Z$ where $Y \not\to X$. It is removed by 3NF.
- A **multivalued** dependency $P \twoheadrightarrow Q$ exists when two independent multi-valued facts sit in one table. It is removed by 4NF.
- A **join** dependency exists when a table can be losslessly reconstructed from projections. It is removed by 5NF when it is not implied by the candidate keys.

PART 3

Normalization

Redundancy and the three anomalies · 1NF to 5NF, one form at a time · before-and-after tables and worked decompositions · lossless join and dependency preservation · a complete case study · denormalization

Source: UNIT 2, Chapters 2.1.2–2.1.6 (Normalization, 1NF, 2NF, 3NF, BCNF, 4NF, 5NF, Denormalization). Every table below is the source's own example, made explicit.

6. Normalization Fundamentals and First Normal Form

Part 3 • why we decompose at all

Before learning six normal forms, it helps to know exactly what disease they are treating. The disease is **redundancy**: the same fact stored in more than one place. Redundancy is not merely wasteful; it is the direct cause of three specific failures called **anomalies**. Normalization is the systematic removal of redundancy by decomposition, and each normal form removes one more kind of redundancy.

6.1 What Normalization Is

> **DEFINITION (SOURCE)**
>
> **Normalization** is a process of designing a consistent database by minimising redundancy and ensuring data integrity through decomposition which is lossless. It is a systematic, step-by-step decomposition of complex records into simple records.

The source lists several characterisations, all of which are worth knowing because exam questions often ask for two or three of them.

Table 6.1 — How the source describes normalization.

Description	What it adds
A process of organizing data in the database	The general aim
A systematic approach of decomposing tables to eliminate data redundancy	The method: decomposition
Developed by E. F. Codd	Provenance — the relational model's inventor
A multi-step process placing data in tabular form by removing duplicate data	The mechanism: remove duplicates
A step-by-step decomposition of complex records into simple records	The direction: big table → small tables
Also called Canonical Synthesis	An alternative name
A technique of building database structures to store data	The purpose: schema design

Features of normalization (source list)

- It avoids data redundancy.
- It is a formal process of developing data structures.
- It promotes data integrity.
- It ensures data dependencies make sense — that is, data is logically stored.
- It eliminates undesirable characteristics.

How normalization works, in the source's words

Normalization designs the schema by modifying the existing schema, reducing both redundancy and dependency of the data. Unwanted duplication is removed along with the anomalies. In an insert an-

omaly, values such as NULL are forced where they should not be; in an update anomaly, data cannot be updated correctly because the same value occurs multiple times in a column; in a delete anomaly, deleting one record removes facts that belong to other records. The aim is therefore to remove redundant data and store only related data in each table, which decreases database size and makes the data logically stored.

6.2 The Three Anomalies

The source defines the three anomalies and, crucially, the definition of each is best understood on a single example. Use this one table throughout the section.

Table 6.2 — An unnormalized relation used to demonstrate all three anomalies. The primary key is (Stu_ID, Course).

Stu_ID	Stu_Name	Dept	Course	Instructor
101	Ananya	CSE	DBMS	Dr. Dhakar
101	Ananya	CSE	OS	Dr. Verma
102	Rohit	CSE	DBMS	Dr. Dhakar
103	Meera	ECE	Signals	Dr. Rao

THE THREE ANOMALIES

Update anomaly. If data items are scattered and not linked properly, updating one fact leaves copies elsewhere unchanged. Here “Ananya's department is CSE” appears twice; changing it in one row and not the other leaves the database inconsistent.

Insertion anomaly. Data cannot be inserted because some other, unrelated fact is missing. Here we cannot record that a new course “Networks” is taught by Dr. Nair until at least one student enrols, because the primary key requires a Stu_ID — and we cannot invent one.

Deletion anomaly. Deleting a record removes facts that belong to other records. If Meera is the only student taking Signals, deleting Meera also deletes the fact that Dr. Rao teaches Signals.

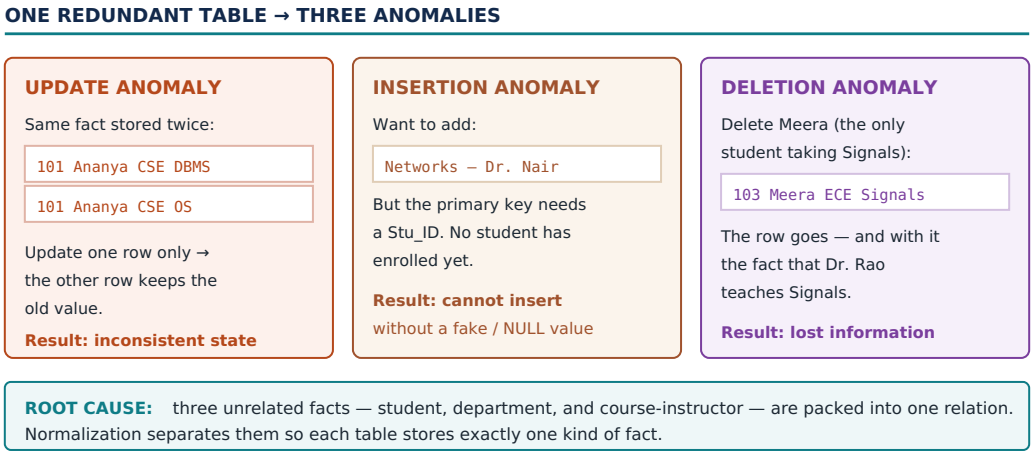


Figure 6.1 — All three anomalies come from the same cause: unrelated facts sharing one table. Removing the cause removes all three symptoms.

6.3 The Ladder of Normal Forms

The source lists six normal forms in order. Each is a stricter condition than the one before it, and each removes one more class of redundancy.

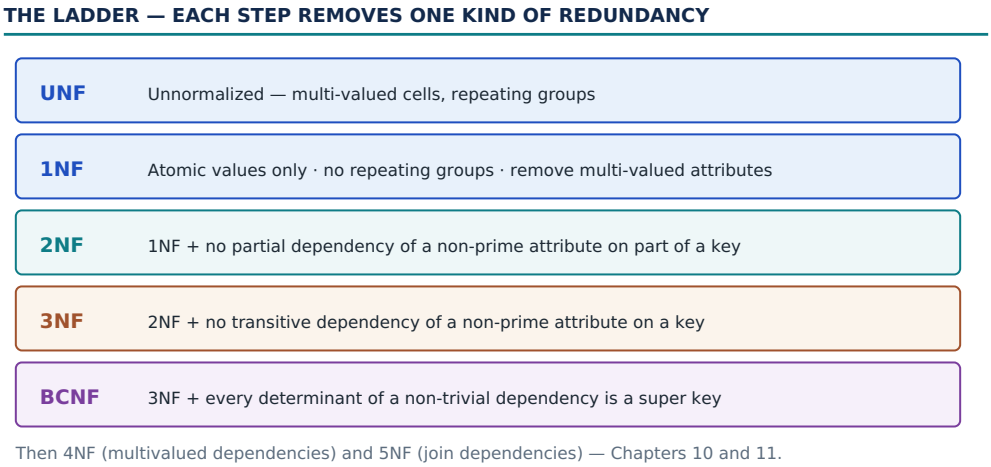


Figure 6.2 — The ladder. A relation that satisfies 3NF automatically satisfies 2NF and 1NF; the forms are cumulative, not alternatives.

6.4 First Normal Form (1NF)

> **DEFINITION**
>
> A relation is in **First Normal Form** if every attribute contains an **atomic** value. An attribute of a table cannot hold multiple values; it must hold only single-valued attributes. 1NF therefore disallows multi-valued attributes, composite attributes, and their combinations.

The source gives a second, equivalent statement that is worth remembering: 1NF is defined in the definition of relations themselves — all attributes in a relation must have atomic domains, and the values in an atomic domain are indivisible units. Each attribute must contain only a single value from its predefined domain.

6.5 Worked 1NF Example

Source Example 1 — the EMPLOYEE table

The source's first example is an EMPLOYEE relation that is **not** in 1NF because `EMP_PHONE` is multi-valued: one employee row lists several phone numbers in a single cell.

Table 6.3 — Before: EMPLOYEE violates 1NF — EMP_PHONE holds several values in one cell.

Emp_ID	Emp_Name	EMP_PHONE
E1	Amit	9876543210, 9812345678
E2	Bhavna	9900112233
E3	Chirag	9123456780, 9000111222, 9811122233

To reach 1NF the multi-valued cell is split so that each phone number occupies its own row. The primary key becomes the pair `( Emp_ID, EMP_PHONE )`, because one employee may have several numbers.

Table 6.4 — After: the same information in 1NF. Each cell is atomic; the key is now composite.

Emp_ID	Emp_Name	EMP_PHONE
E1	Amit	9876543210
E1	Amit	9812345678
E2	Bhavna	9900112233
E3	Chirag	9123456780
E3	Chirag	9000111222
E3	Chirag	9811122233

EXPLANATION — THE PRICE OF 1NF

Notice what happened: the fix introduced **repetition** of Emp_ID and Emp_Name. That repetition is precisely what 2NF will now attack. This is the pattern of the whole ladder — each normal form removes one kind of redundancy, and sometimes exposes the next kind.

Source Example 2 — a course-enrolment relation

The source's second example is a relation in which a single student cell or course cell holds several values. Rearranging it so that each attribute holds a single value from its domain converts it to 1NF, exactly as above. The same principle applies: **one cell, one value**.

COMMON MISTAKE

Believing that 1NF is about the number of columns. It is not. 1NF is about the **contents of one cell**. A table with 200 columns is perfectly capable of being in 1NF, provided every cell holds exactly one indivisible value. Equally, a two-column table can violate 1NF if a cell contains a list.

QUICK TEST FOR 1NF

Ask of every column: "Can this cell ever need to hold two values?" A comma-separated list, a set of tags, an array, or a repeating group in a cell all mean the answer is yes, and the relation is not in 1NF. The fix is always the same: move the repeated values into their own rows.

Chapter summary

SUMMARY — CHAPTER 6

- Normalization is lossless decomposition to remove redundancy and anomalies; it was introduced by E. F. Codd and is also called canonical synthesis.
- The three anomalies are update, insertion and deletion; all three come from unrelated facts sharing one table.
- **1NF** requires atomic values: no multi-valued attributes, no composite attributes, no repeating groups inside a cell.
- The ladder runs 1NF → 2NF → 3NF → BCNF → 4NF → 5NF and is cumulative.

7. Second Normal Form (2NF)

Part 3 • removing partial dependencies

2NF is the first form that depends on which attributes are prime. Before stating the rule, the source defines the two terms the rule needs.

7.1 Prime and Non-Prime Attributes

DEFINITIONS (SOURCE)

Prime attribute. An attribute that is part of a candidate key.

Non-prime attribute. An attribute that is not part of any candidate key.

The source's note in the keys chapter says the same thing in one line: those attributes which appear in some candidate key are called prime attributes. Note that a prime attribute must be part of **some** candidate key — not necessarily of the primary key you happened to choose.

7.2 The 2NF Rule: Full Functional Dependency

DEFINITION

If we follow Second Normal Form, then **every non-prime attribute must be fully functionally dependent on the primary key**. That is: if X → A holds, there must be **no proper subset Y of X** for which Y → A also holds.

In the source's formulation, a table is in 2NF when two conditions are met:

1. The table is in the first normal form.
2. The primary key of the table is composed of exactly one column.

EXPLANATION — THE TWO STATEMENTS ARE THE SAME RULE

The second condition looks like a different rule, and students often wonder why a single-column key automatically gives 2NF. The reason is that a single-column key has **no proper subset** other than the empty set, so a partial dependency is impossible. Conversely, when the key has two or more columns, a non-prime attribute might depend on only one of them — and that is exactly the violation. So “no proper subset determines the attribute” and “the key is a single column” describe the same requirement from two angles.

7.3 Partial Dependency and How to Remove It

DEFINITION

A **partial dependency** exists when a non-prime attribute depends on **part** of a composite candidate key rather than on the whole key.

The source's Student_Project relation is the canonical example. Its prime key attributes are `Stu_ID` and `Proj_ID`. The rule says the non-key attributes `Stu_Name` and `Proj_Name` must depend on **both**, not on

either individually. But `Stu_Name` can be identified by `Stu_ID` alone, and `Proj_Name` by `Proj_ID` alone. Those are partial dependencies, and 2NF forbids them.

Table 7.1 — Before: Student_Project violates 2NF. Key = (Stu_ID, Proj_ID).

Stu_ID	Proj_ID	Stu_Name	Proj_Name
S1	P1	Ananya	Library System
S1	P2	Ananya	Hostel Portal
S2	P1	Rohit	Library System

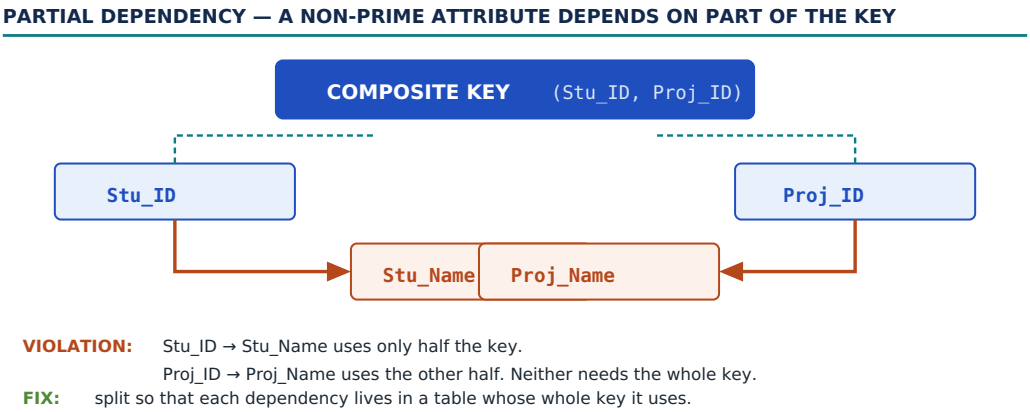


Figure 7.1 — A partial dependency: each non-prime attribute uses only one half of the composite key. 2NF removes it by splitting along the dependency lines.

Decomposing along those lines gives two relations, and neither contains a partial dependency:

Table 7.2 — After (relation 1): Stu_ID → Stu_Name, key Stu_ID.

Stu_ID	Stu_Name
S1	Ananya
S2	Rohit

Table 7.3 — After (relation 2): Proj_ID → Proj_Name, key Proj_ID.

Proj_ID	Proj_Name
P1	Library System
P2	Hostel Portal

IMPORTANT — DO NOT LOSE THE RELATIONSHIP

The two tables above remove the redundancy, but they also remove the fact that S1 works on P1. In a real schema you keep a third, junction relation `Works_On ( Stu_ID, Proj_ID )` whose whole composite key represents that many-to-many relationship. The source's slide shows only the two projections because it is illustrating the dependency, not designing the final schema. Chapter 13 builds the complete schema properly.

7.4 Worked 2NF Example

The source's enrolment table

The source's second example is a student-enrolment table. Its first column is the student name, its second the course code.

Table 7.4 — Before: enrolment is not in 2NF.

Student Name	Course Code
Rahul	CS101
Priya	CS101
Rahul	MA201
Sana	CS101

Neither column is unique — "Rahul" appears in rows 1 and 3, "CS101" appears in rows 2 and 4 — but the pair `( Student Name, Course Code )` is unique, because a student cannot enrol in the same course more than once. So the pair is the primary key.

As per the 2NF definition this table is **not** in 2NF. In this particular table both columns are prime, so there is no non-prime attribute to violate the rule literally; the source's real point is the structural one: a two-column key is a table in which partial dependencies can arise, and the standard treatment is to separate the entity from the relationship. The source does exactly that, by introducing an enrolment number as a single-column key.

Table 7.5 — After (step 1): an enrolment number becomes a single-column key.

Enrolment No	Student Name	Course Code
1	Rahul	CS101
2	Priya	CS101
3	Rahul	MA201
4	Sana	CS101

Table 7.6 — After (step 2): the courses are held separately, so each course fact is stored once.

Course Code	Course Name
CS101	Programming Fundamentals
MA201	Discrete Mathematics

The source's closing remark is exactly right: these two tables together provide the exact same information as the original table. That is the definition of a lossless decomposition, formalised in Chapter 12.

> **EXPLANATION — THE SHAPE OF A 2NF FIX**
>
> Every 2NF decomposition has the same shape. You identify a composite key; you notice that some non-prime attribute depends on only one part of it; you move that attribute into a new table whose key is that one part. The junction table keeps only the two key parts, and it is what preserves the original many-to-many fact.

QUICK TEST FOR 2NF

Look at the primary key. If it has **one** column, the relation is automatically in 2NF (given 1NF). If it has two or more columns, check every non-prime attribute: does it need the whole key, or would one part of the key do? If one part would do, the relation is not in 2NF.

Chapter summary

SUMMARY — CHAPTER 7

- A **prime attribute** belongs to some candidate key; a **non-prime attribute** belongs to none.
- **2NF** requires 1NF plus the rule that every non-prime attribute is fully functionally dependent on the key — no proper subset of the key determines it.
- A **partial dependency** is a non-prime attribute depending on part of a composite key.
- The fix is to split the relation so each non-prime attribute sits with the smallest key that determines it, keeping a junction relation to preserve the original relationship.

8. Third Normal Form (3NF)

Part 3 • removing transitive dependencies

2NF removed dependencies on part of a key. 3NF removes dependencies that pass through a non-prime attribute.

8.1 The 3NF Rule

DEFINITION

For a relation to be in **Third Normal Form** it must be in Second Normal Form and the following must hold: **no non-prime attribute is transitively dependent on a prime key attribute.**

Equivalently, for any non-trivial functional dependency `X → A`, either:

- X is a **super key**, or
- A is a **prime attribute**.

The source gives a second formulation worth keeping beside the first: the relation should comply with 2NF, and any non-prime attribute should not have a transitive functional dependency on the super key.

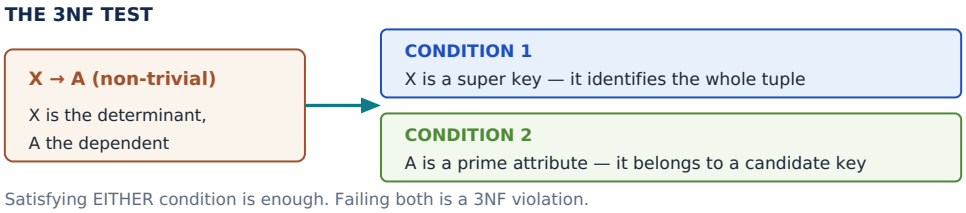


Figure 8.1 — The 3NF test in its two-condition form. The “A is prime” escape clause is what distinguishes 3NF from BCNF (Chapter 9).

8.2 Transitive Dependency in a Real Relation

The source's running example for 3NF is the Student_detail relation with attributes `Stu_ID`, `Stu_Name`, `Zip` and `City`. `Stu_ID` is the key and the only prime attribute. `City` can be identified by `Stu_ID`, but it can also be identified by `Zip` itself. Since neither `Zip` is a super key nor `City` a prime attribute, and since `Stu_ID → Zip → City`, there is a transitive dependency.

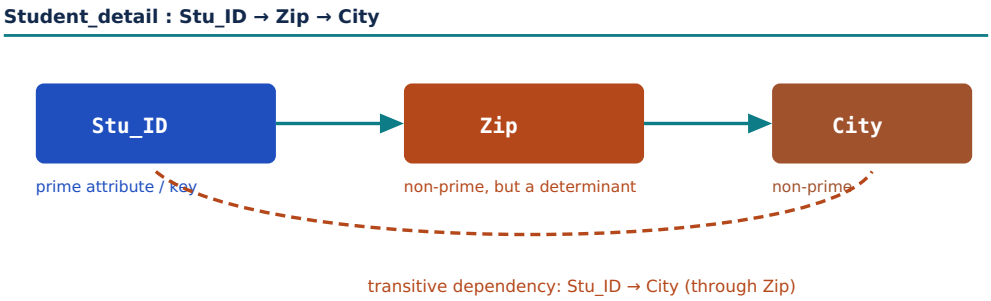


Figure 8.2 — Zip is not a super key, yet it determines City. That is the transitive dependency 3NF removes.

8.3 Worked 3NF Example

The source's Employee relation

The source's larger example is an Employee relation with attributes `Emp_ID`, `Emp_Name`, `Emp_Zip`, `Emp_State`, `Emp_City` and `Emp_District`. The super keys include `{Emp_ID}`, `{Emp_ID, Emp_Name}`, `{Emp_ID, Emp_Name, Emp_Zip}` and many others; the candidate key is `{Emp_ID}`. So `Emp_ID` is the prime attribute and all the others are non-prime.

Table 8.1 — Before: Employee violates 3NF.

Emp_ID	Emp_Name	Emp_Zip	Emp_State	Emp_City	Emp_District
E1	Amit	160017	Chandigarh	Chandigarh	Sector 17
E2	Bhavna	160017	Chandigarh	Chandigarh	Sector 17
E3	Chirag	140301	Punjab	Mohali	Kharar

The three attributes `Emp_State`, `Emp_City` and `Emp_District` are functionally dependent on `Emp_Zip`. We can find the zip number using `Emp_ID`, so `Emp_Zip` is dependent on `Emp_ID`. But the three location attributes are **non-prime** and are only indirectly dependent on `Emp_ID` — through `Emp_Zip`. That indirectness breaks the 3NF rule.

WORKED STEP — FINDING THE VIOLATION

Dependencies present:

```
Emp_ID   → Emp_Name, Emp_Zip, Emp_State, Emp_City, Emp_District   (the key determines all)
Emp_Zip  → Emp_State, Emp_City, Emp_District                      (zip determines location)
```

Apply the 3NF test to `Emp_Zip → Emp_State`: is `Emp_Zip` a super key? No — many employees share a zip. Is `Emp_State` prime? No — the only prime attribute is `Emp_ID`. Both conditions fail, so this dependency violates 3NF. The same reasoning applies to `Emp_Zip → Emp_City` and `Emp_Zip → Emp_District`.

The fix is to split the relation so that the location attributes sit with the determinant that governs them:

Table 8.2 — After (relation 1): Employee_ID — key Emp_ID, no transitive dependency.

Emp_ID	Emp_Name	Emp_Zip
E1	Amit	160017
E2	Bhavna	160017
E3	Chirag	140301

Table 8.3 — After (relation 2): Employee_Zip — key Emp_Zip, location stored once per zip.

Emp_Zip	Emp_State	Emp_City	Emp_District
160017	Chandigarh	Chandigarh	Sector 17
140301	Punjab	Mohali	Kharar

Both resulting tables are in 3NF. In the first, every non-prime attribute depends on the whole key and on nothing else. In the second, the key `Emp_Zip` determines the three location attributes directly, with no intermediate non-prime determinant. Notice the immediate practical benefit: the fact “160017 is in

Chandigarh" is now stored **once** instead of once per employee, so changing it cannot leave the database inconsistent.

QUICK TEST FOR 3NF

For every non-trivial dependency `X → A`, ask: is X a super key? If yes, fine. If no, ask: is A a prime attribute? If yes, fine. If the answer to both is no, the relation is not in 3NF, and that dependency is the transitive one to break.

Chapter summary

SUMMARY — CHAPTER 8

- **3NF** requires 2NF plus: no non-prime attribute is transitively dependent on a key.
- Test form: for every non-trivial X → A, either X is a super key **or** A is prime.
- A transitive dependency is a chain key → non-prime → non-prime.
- The fix moves the second-hop attributes into a table keyed by the non-prime determinant.

9. Boyce-Codd Normal Form (BCNF)

Part 3 • 3.5NF — the strictest functional form

BCNF is 3NF with the escape clause removed. Where 3NF allowed “A is prime” as an excuse, BCNF does not.

9.1 Why 3NF Is Not Always Enough

3NF permits a dependency `X → A` to stand even when X is not a super key, provided A is a prime attribute. That exception is usually harmless, but in one situation it leaves redundancy in place: when a **non-super-key determinant** determines a **prime attribute** that belongs to a different candidate key. BCNF closes that gap.

EXPLANATION — THE SINGLE DIFFERENCE

3NF: for every non-trivial X → A, **either** X is a super key **or** A is prime.
BCNF: for every non-trivial X → A, X **must** be a super key. (No second option.)

Because BCNF is stricter, every relation in BCNF is automatically in 3NF, but not every 3NF relation is in BCNF. The source's own framing is exactly this: BCNF is “an extension of Third Normal Form on strict terms”.

9.2 The BCNF Rule

DEFINITION

Boyce-Codd Normal Form states that for any non-trivial functional dependency `X → A`, **X must be a super key**.

The source gives two further statements of the same rule, both worth recognising:

- A relation is in BCNF if it holds the following conditions: the relation must be in third normal form, and for any functional dependency `A → B` in the relation, A should be the super key.
- If there is a functional dependency `A → B`, then either A is a super key, or the dependency is **trivial** — that is, all the columns of B are contained in the columns of A.

EXPLANATION — WHAT THE TRIVIAL CLAUSE MEANS

The source illustrates the trivial clause with `(course code, professor name) → (course code)`. This is trivially true because when you know the course code and the professor name, you obviously know the course code. Such a dependency cannot violate anything, so BCNF ignores it. Only **non-trivial** dependencies are tested.

The source explains the reasoning behind the rule in two sentences worth reproducing, because they show why the rule exists rather than merely what it says:

1. **A is a super key:** only on a super key column should it be the case that there is a dependency of other columns. If a set of columns B can be determined by knowing some other set of columns A, then A should be a super key — a super key determines each row uniquely.
2. **It is a trivial functional dependency:** there should be no non-trivial dependency of the kind that causes trouble. The source's example is that a professor's department depended on the

professor's name; that creates integrity issues, because someone may edit the professor's name without changing the department, leaving the database inconsistent.

9.3 Worked BCNF Example: Employees and Departments

The source's BCNF example is an Employee relation with attributes `Emp_ID`, `Emp_Nationality`, `Emp_Department`, `Dept_Type` and `No_of_Employees`.

Table 9.1 — Before: Employee violates BCNF.

Emp_ID	Emp_Nationality	Emp_Department	Dept_Type	No_of_Employees
E1	Indian	CSE	Academic	40
E2	Indian	CSE	Academic	40
E3	British	HR	Administrative	12

The candidate key for this relation is `{Emp_ID, Emp_Dept}`. The source explains why the relation fails BCNF: neither `Emp_ID` alone can provide the department information, nor can `Emp_Dept` alone determine the employee information. Yet there are dependencies whose determinants are not super keys:

```
Emp_ID     → Emp_Nationality                (determinant is only part of the key)
Emp_Dept   → Dept_Type, No_of_Employees     (determinant is only part of the key)
```

Each of those determinants is only part of the candidate key, so neither is a super key. The relation therefore does not comply with BCNF. The source's fix is to divide it into three tables:

Table 9.2 — After (relation 1): Employee_Nationality — key Emp_ID.

Emp_ID	Emp_Nationality
E1	Indian
E2	Indian
E3	British

Table 9.3 — After (relation 2): Employee_Department — key Emp_Dept.

Emp_Dept	Dept_Type	No_of_Employees
CSE	Academic	40
HR	Administrative	12

Table 9.4 — After (relation 3): Employee_ID_Dept — key (Emp_ID, Emp_Dept), the junction that preserves the assignment.

Emp_ID	Emp_Dept
E1	CSE
E2	CSE

Emp_ID	Emp_Dept
E3	HR

In relation 1, `Emp_ID` is the candidate key and the only dependency is `Emp_ID → Emp_Nationality`, whose determinant is a super key — so it is in BCNF. In relation 2, `Emp_Dept` is the candidate key and the dependency is `Emp_Dept → Dept_Type, No_of_Employees`, again with a super-key determinant — so it is in BCNF too. Relation 3 has no non-key attributes, so it is trivially in BCNF. The source's conclusion is exactly this: “both the relations are in BCNF”.

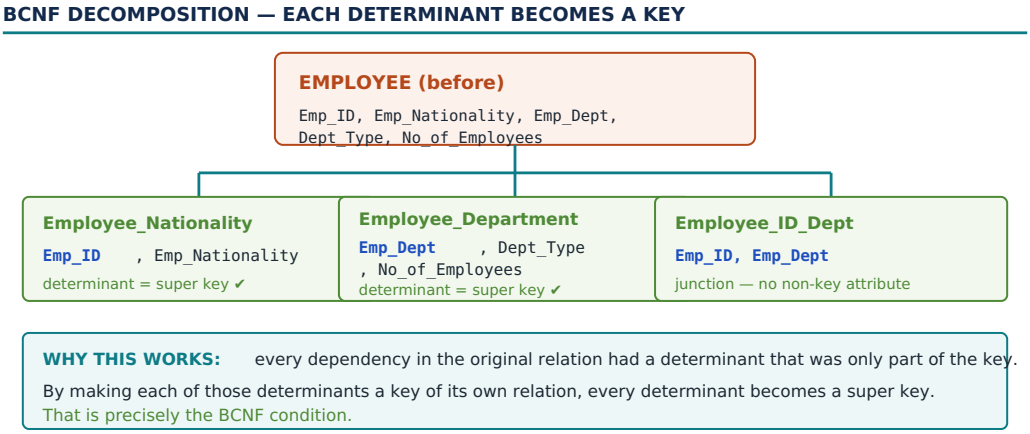


Figure 9.1 — The BCNF fix makes every determinant of every non-trivial dependency a super key, by giving each determinant its own relation.

9.4 BCNF and Lossless Join

One caution belongs here, because it is a favourite viva question. A BCNF decomposition is always **lossless**, but it is not always **dependency-preserving**: it is possible that some functional dependency of the original schema cannot be checked on any single resulting table. 3NF is the highest form that can always be achieved while preserving all dependencies. Chapter 12 defines both properties precisely and tests the example above for both.

Chapter summary

SUMMARY — CHAPTER 9

- **BCNF** is 3NF without the “A is prime” escape clause: every determinant of a non-trivial dependency must be a super key.
- It is also called 3.5NF, and it is strictly stronger than 3NF.
- Trivial dependencies are ignored by the test.
- The fix gives each non-super-key determinant its own relation, keeping a junction table for the original relationship.
- BCNF guarantees lossless join but not always dependency preservation.

10. Fourth Normal Form (4NF)

Part 3 • removing multivalued dependencies

Up to BCNF every rule has been about functional dependencies — “one value of X forces one value of Y”. 4NF is the first form that deals with a different situation: one value of X forcing many values of Y, independently of many values of Z.

10.1 Multivalued Dependency Revisited

Section 5.2 defined the multivalued dependency. Restate it here in the form the 4NF rule needs, because the rule is stated in terms of it.

DEFINITION

For a relation R(A, B, C), a **multivalued dependency** `A ↠ B` holds when, for each value of A, the set of B values is independent of the set of C values. In other words, A determines a set of B values and a set of C values, and the two sets have no relationship to each other. The source's three conditions for a multivalued dependency are:

1. For a dependency A → B, if for a single value of A multiple values of B exist, the table may have a multivalued dependency.
2. The table must have at least three columns.
3. For R(A, B, C), if there is a multivalued dependency between A and B, then B and C must be independent of each other.

10.2 The 4NF Rule

DEFINITION

A relation is in **Fourth Normal Form** if it holds all of the following conditions:

- The relation complies with **Boyce-Codd Normal Form**.
- There is **no multivalued dependency** between the table's attributes.

The source gives two further statements of the same rule, both useful for an exam answer:

- For a table to satisfy 4NF: it should be in BCNF, and the table should not have any multivalued dependency.
- In the fourth normal form: it should meet all the requirements of 3NF, and an attribute of one or more rows in the table should not result in more than one row of the same table, leading to multivalued dependencies.

COMMON MISTAKE

Thinking that 4NF replaces BCNF. It does not — 4NF includes BCNF. A relation that is not in BCNF cannot be in 4NF, no matter how many multivalued dependencies you remove. State both conditions when answering.

10.3 Worked 4NF Example: Students

The source's first 4NF example is the Students relation with three attributes: `Stud_ID`, `Stud_Course` and `Stud_Hobby`.

Table 10.1 — Before: Students violates 4NF. Stud_Course and Stud_Hobby are independent facts about Stud_ID.

Stud_ID	Stud_Course	Stud_Hobby
S1	DBMS	Cricket
S1	DBMS	Music
S1	OS	Cricket
S1	OS	Music

The relation is not in 4NF because it has multivalued dependencies: both `Stud_Course` and `Stud_Hobby` depend on `Stud_ID` in a multi-valued way, producing the cross-product pattern visible above — two courses × two hobbies = four rows. To make the relation 4NF, break it into two relations:

Table 10.2 — After (relation 1): StudentCourse.

Stud_ID	Stud_Course
S1	DBMS
S1	OS

Table 10.3 — After (relation 2): StudentHobby.

Stud_ID	Stud_Hobby
S1	Cricket
S1	Music

The two relations together carry exactly the same information as the original, but each now stores a single multi-valued fact about the student. The cross-product rows are gone.

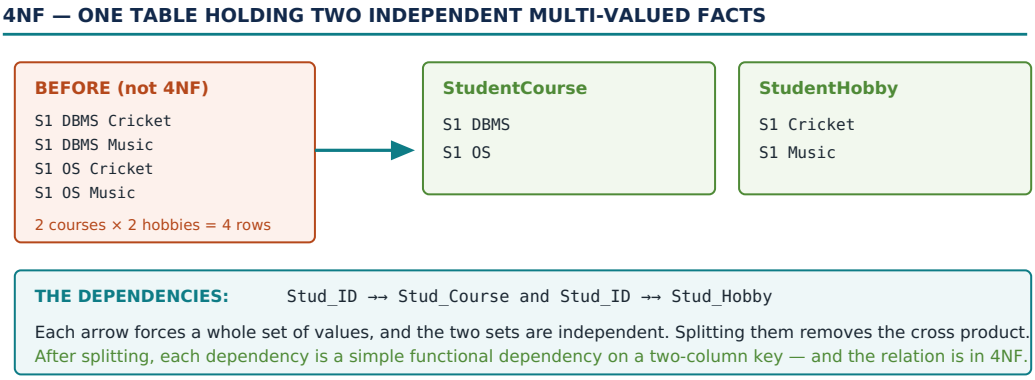


Figure 10.1 — A multivalued dependency makes one table hold a cross product of two independent facts. 4NF splits them into one table per fact.

10.4 Worked 4NF Example: Subject, Lecturer, Books

The source's second 4NF example is a Course relation with a SUBJECT, the LECTURER who teaches it, and the recommended BOOKS for it.

Table 10.4 — Before: the Course relation. LECTURER and BOOKS are independent of each other.

SUBJECT	LECTURER	BOOKS
Mathematics	Alex	Maths Book1
Mathematics	Alex	Maths Book2
Mathematics	Bosco	Maths Book1
Mathematics	Bosco	Maths Book2

The source's analysis is precise and worth following step by step. The data satisfies 3NF. But LECTURER and BOOKS are two independent entities here: there is no relationship between a lecturer and a book. Either Alex or Bosco can teach Mathematics, and for Mathematics a student can refer to either Maths Book1 or Maths Book2. So:

```
SUBJECT →→ LECTURER
SUBJECT →→ BOOKS
```

This is a multivalued dependency on SUBJECT. The source then makes the crucial observation: if you select both the lecturer and the books recommended for a subject, the query returns a (lecturer, books) combination, which wrongly implies that a particular lecturer recommends a particular book. That implication is not in the data at all — it is an artefact of storing the two facts together.

WORKED STEP — THE MISLEADING QUERY

```
SELECT c.LECTURER, c.BOOKS
FROM   COURSE c
WHERE  SUBJECT = 'Mathematics';
```

This returns four rows: (Alex, Maths Book1), (Alex, Maths Book2), (Bosco, Maths Book1), (Bosco, Maths Book2). Reading the result, a user could conclude that Bosco recommends Maths Book2. Nothing in the data supports that conclusion — the two facts are simply independent.

The source's remedy is to split the single table into two, and to query them independently:

```
-- Select the lecturer names
SELECT c.SUBJECT, c.LECTURER
FROM   COURSE c
WHERE  c.SUBJECT = 'Mathematics';

-- Select the recommended book names
SELECT c.SUBJECT, c.BOOKS
FROM   COURSE c
WHERE  c.SUBJECT = 'Mathematics';
```

Firing two independent queries removes the multivalued dependency and the confusion around the data, and the table is then in 4NF. This is the same fix as Section 10.3 in a different setting: **one table, one multi-valued fact**.

QUICK TEST FOR 4NF

Look for the cross-product pattern. If a table has three columns A, B, C and, for a single A value, every B value is paired with every C value, then B and C are independent multi-valued facts and the table is not in 4NF. The fix is always to split B and C into two tables, each keyed by A.

Chapter summary

SUMMARY — CHAPTER 10

- **4NF** requires BCNF plus: no multivalued dependency among the attributes.
- A multivalued dependency `A ↠ B` exists when, for each A, the B values and the C values are independent sets. It needs at least three columns.
- Its signature is a cross product in the data, and its cost is a misleading join.
- The fix splits the table into one table per multi-valued fact, each keyed by the determinant.

11. Fifth Normal Form (5NF) and Join Dependency

Part 3 • project–join normal form

5NF is also called the **Projected Normal Form** or **Project–Join Normal Form (PJNF)**. It is the last of the six forms, and the one the source describes as "largely of academic interest" — and it explains exactly why, which is worth understanding.

11.1 The 5NF Rule

> **DEFINITION**
> A relation R is in **Fifth Normal Form** if and only if **every join dependency in R is implied by the candidate keys of R**.

The source gives the equivalent property list, and it is the clearest way to remember the rule:

1. R should already be in 4NF.
2. R cannot be further non-loss decomposed — that is, it has no join dependency beyond those implied by its keys.

The source also states it in the "table" form: a table T is in fifth normal form if every join dependency in T is a consequence only of the candidate keys of T. And in plain language: a table is in 5NF if it is in 4NF and it cannot be subdivided into any smaller tables without losing some form of information.

> **EXPLANATION — THE PHRASE THAT MATTERS**
> The whole of 5NF turns on one phrase: **"implied by the candidate keys"**. A table can always be split into projections and joined back — the question is whether the split is forced by the keys or whether it depends on an extra business rule. If the split is forced by the keys, the join dependency carries no new information and the table is in 5NF. If the split depends on a separate rule, that rule is a genuine constraint and the table is not in 5NF until it is decomposed.

11.2 Worked 5NF Example: Agent, Company, Product

The source's 5NF example is the table it calls `agent_company product`, and its explanation of the two versions of that table is the most important passage in the chapter.

The table records which agents sell which products for which companies. Suppose the business rule is:

```
If an agent sells a product, and the agent works for a company,
and the company makes that product — then the agent sells that product for that company.
```

That rule is a **tuple-formation rule**. It is not a key; it is an extra constraint on which rows may exist. Because such a rule exists, the join dependency

```
*(agent, company, product_name)
```

is not a consequence only of the primary key — it is also a consequence of that extra rule. That is exactly the condition the source names for a 5NF violation.

FROM THE SOURCE — THE TWO VERSIONS OF THE TABLE

The source distinguishes two versions clearly, and the distinction is the whole point:

Version 1 — no tuple-formation rule applied. In the first example of the `agent_company Product` table there was no application of this rule, hence no join dependency other than that on the primary key. **That table was in 5NF.**

Version 2 — the rule applied. The second version of the same table violated 5NF, because the join dependency *(agent, company, product_name) was not a consequence only of the primary key, but also a consequence of the tuple-formation rule given earlier.

The lesson is precise: **the same three columns can be in 5NF or not, depending on whether a tuple-formation rule exists.** 5NF is about business rules, not about column lists.

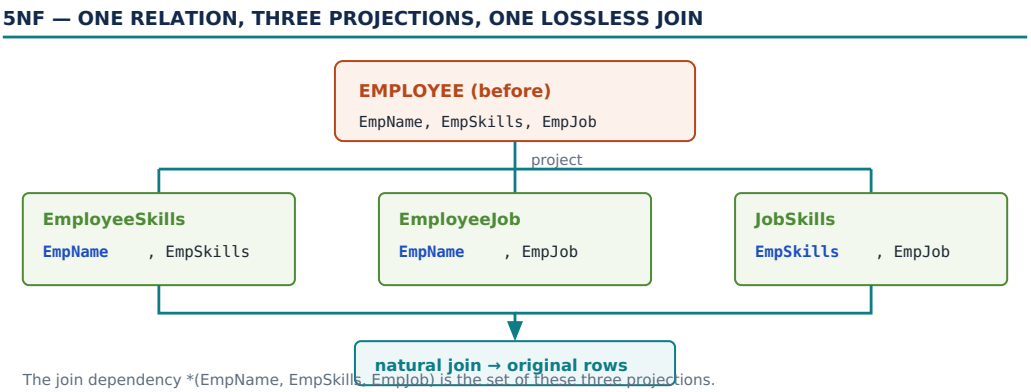


Figure 11.1 — A join dependency: the relation is reconstructed by joining three projections. 5NF asks whether that dependency follows from the keys alone.

Illustrating the decomposition

Section 5.3 showed how such a table is decomposed into three projections. Reproduced here with each relation's own key marked, which is what makes the “implied by the candidate keys” test applicable.

Table 11.1 — The three-column relation, before decomposition.

EmpName	EmpSkills	EmpJob
Tom	Networking	EJ001
Harry	Web Development	EJ002
Katie	Programming	EJ002

Table 11.2 — Projection 1: EmployeeSkills.

EmpName	EmpSkills
Tom	Networking
Harry	Web Development
Katie	Programming

Table 11.3 — Projection 2: EmployeeJob.

EmpName	EmpJob
Tom	EJ001
Harry	EJ002
Katie	EJ002

Table 11.4 — Projection 3: JobSkills.

EmpSkills	EmpJob
Networking	EJ001
Web Development	EJ002
Programming	EJ002

The join dependency is the set of three projections:

```
{ (EmpName, EmpSkills), (EmpName, EmpJob), (EmpSkills, EmpJob) }
```

and the source's conclusion is that a join of the three relations above is equal to the original relation. The rows of Tables 11.2–11.4 are the source's rows.

11.3 Why 5NF Is Largely of Academic Interest

The source's closing assessment is refreshingly honest and should be quoted in any exam answer on 5NF:

> **THE SOURCE'S OWN ASSESSMENT**
>
> “It can be shown that if a table is in 5NF, then, because join dependencies are the ‘ultimate’ form of dependency, it must also be in 4NF and thus conform to all the further normal forms. The problem with this is that **detecting join dependencies is, in practice, very difficult**. For this reason, 5NF is largely of academic interest.”

Two consequences follow. First, 5NF sits at the top of the ladder: 5NF ⟹ 4NF ⟹ BCNF ⟹ 3NF ⟹ 2NF ⟹ 1NF. Second, in ordinary application design, reaching BCNF or 3NF is usually enough; 4NF matters when independent multi-valued facts are present, and 5NF matters only when a tuple-formation rule of the kind above is in force. The source's own summary of the whole normalization chapter makes the same practical point:

> **SOURCE SUMMARY OF NORMALIZATION**
>
> “The various forms of database normalization are useful while designing the schema of a database in such a way that there is no data replication which may possibly lead to inconsistencies. While designing the schema for applications, we should always think about how we can make use of these forms.”

Chapter summary

SUMMARY — CHAPTER 11

- **5NF** (also projected normal form / PJNF): every join dependency must be implied by the candidate keys.
- It requires 4NF, and it means the relation cannot be non-loss decomposed further.
- The same three columns can be in 5NF or not, depending on whether a tuple-formation rule exists — that is why the source's two versions of the agent/company/product table differ.
- Detecting join dependencies is hard in practice, so 5NF is mainly of academic interest.

12. Lossless Decomposition

Part 3 • the property that makes decomposition safe

Every normal form in Part 3 has been reached by splitting a relation. That raises an obvious danger: if you split a table badly, you can lose information, and no amount of tidiness in the resulting schema will compensate. The property that rules this out is **lossless decomposition**, and the source names it in its own definition of normalization — “decomposition which is lossless”.

12.1 What Lossless Means

DEFINITION

A decomposition of relation R into relations R1, R2, …, Rn is **lossless** (more precisely, lossless-join) if joining the decomposed relations back together reproduces **exactly** the original relation — no rows lost, and no spurious rows created.

The word “lossless” is slightly misleading, because the danger is not only losing rows. A bad decomposition can also invent rows: the natural join of the pieces produces combinations that never existed in the original. The source's phrasing of the goal captures both directions: a relation decomposed into relations must have the lossless-join property, which ensures that no spurious or extra tuples are generated when the relations are reunited through a natural join.

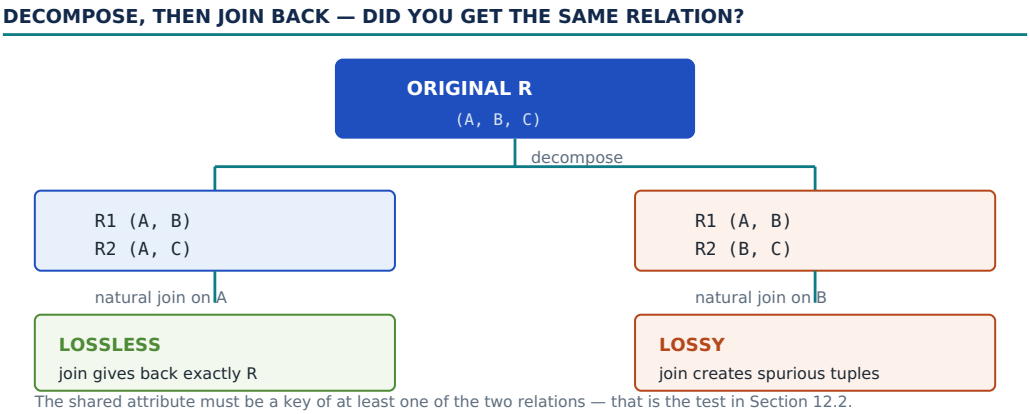


Figure 12.1 — A decomposition is lossless when the shared attribute is a key of one of the pieces. If it is not, the join invents rows.

12.2 The Lossless-Join Test

THE TEST FOR A BINARY DECOMPOSITION

Let R be decomposed into R1 and R2. The decomposition is **lossless** if and only if the **common attributes** R1 ∩ R2 form a **super key** of at least one of R1 or R2.

That is: (R1 ∩ R2) → R1 or (R1 ∩ R2) → R2.

The intuition is worth stating plainly. The join can only match rows on the shared attributes. If those shared attributes identify a row uniquely in at least one of the pieces, the join cannot create a false match, so no spurious tuple can appear. If the shared attributes identify nothing, several rows can match each other and the cross product appears.

12.3 Worked Test on a Student Relation

WORKED STEP — TESTING TWO DECOMPOSITIONS

Relation: `R ( Stu_ID, Stu_Name, Zip, City )`, with dependencies `Stu_ID → Stu_Name, Zip` and `Zip → City`.

Decomposition A (the correct 3NF split): `R1 ( Stu_ID, Stu_Name, Zip )` and `R2 ( Zip, City )`.

Step	Check	Result
1	Common attributes R1 ∩ R2	`{ Zip }`
2	Is Zip a super key of R2?	`Yes — Zip → City, and R2 = {Zip, City}`
3	Verdict	**Lossless ✔**

Decomposition B (a deliberately wrong split): `R1 ( Stu_ID, Stu_Name )` and `R2 ( Stu_Name, Zip, City )`.

Step	Check	Result
1	Common attributes R1 ∩ R2	`{ Stu_Name }`
2	Is Stu_Name a super key of R1?	`No — Stu_ID → Stu_Name, not the reverse`
3	Is Stu_Name a super key of R2?	`No — Zip → City, and Stu_Name → nothing here`
4	Verdict	**Lossy ✘**

Two students with the same name would produce a false match, and the join would pair each student with the other's zip and city. Decomposition B is therefore rejected.

12.4 Dependency Preservation

DEFINITION

A decomposition is **dependency-preserving** if every functional dependency of the original relation can be checked using only the resulting relations — that is, the union of the dependencies projected onto each new relation is equivalent to the original set.

Lossless join and dependency preservation are different properties, and a decomposition can have one without the other. The practical consequences are worth remembering:

Table 12.1 — The two properties and what each guarantees.

Property	What it guarantees	If it is missing
Lossless join	No information is lost or invented	The database now holds wrong data
Dependency preservation	Every rule can be enforced by a table constraint	Rules must be enforced by application code, or go unchecked

EXPLANATION — THE PRACTICAL TRADE-OFF

3NF can always be achieved while preserving all dependencies. BCNF can always be achieved losslessly, but sometimes at the cost of a dependency that no single table can enforce. This is the real reason some designers stop at 3NF even though BCNF is stricter: an unenforceable rule is a worse problem than the small redundancy BCNF would have removed.

Chapter summary

SUMMARY — CHAPTER 12

- A decomposition is **lossless** if the natural join of the pieces reproduces the original relation exactly — no rows lost, no rows invented.
- **Test:** the common attributes R1 ∩ R2 must be a super key of at least one piece.
- A decomposition is **dependency-preserving** if every original dependency can be checked on the new relations.
- 3NF can always give both properties; BCNF always gives losslessness but not always dependency preservation.

13. A Complete Worked Normalization Case Study

Part 3 • one relation, driven from UNF all the way to BCNF and 4NF

Parts 3.1 to 3.6 taught the forms one at a time, each on its own small example. This chapter does what an exam question usually asks: take **one** messy relation and drive it down the ladder, showing at each step which rule is broken, what dependency causes the break, and what the decomposition does about it. Every table here is built from the source's own examples; the synthesis is new and is labelled as such.

A NOTE ON PROVENANCE

The relation below is an **illustrative case study** assembled from the source's schemas (Student, Student_Project, Employee, Department). It is not a table printed in the source. It is included because the source teaches each normal form separately and never works one relation all the way down, and because that is exactly the form an examination question takes.

13.1 The Unnormalized Relation

Consider the following relation, which a college might naively use to keep track of student projects.

```
PROJECT_RECORD ( Stu_ID, Stu_Name, Dept, Proj_ID, Proj_Name, Guide_Name, Guide_Phone )
```

And suppose the following business rules hold. These are the functional dependencies; without them no normalization is possible.

```
F = {
   Stu_ID     → Stu_Name, Dept
   Proj_ID    → Proj_Name, Guide_Name
   Guide_Name → Guide_Phone
}
```

The source's rule for finding the key applies directly: attributes appearing only on the right side cannot be in any key; attributes appearing on the left must be. `Stu_ID` and `Proj_ID` both appear on the left, so both are needed. Compute the closure:

WORKED STEP — FINDING THE KEY

Pass	Result set	Dependency used
start	{ Stu_ID, Proj_ID }	—
1	{ Stu_ID, Proj_ID, Stu_Name, Dept }	Stu_ID → Stu_Name, Dept
2	{ … , Proj_Name, Guide_Name }	Proj_ID → Proj_Name, Guide_Name
3	{ … , Guide_Phone }	Guide_Name → Guide_Phone

Result: (Stu_ID, Proj_ID)⁺ contains every attribute, and neither Stu_ID⁺ nor Proj_ID⁺ does. So `( Stu_ID, Proj_ID )` is the only candidate key, and therefore the primary key.

Table 13.1 — The unnormalized relation. Key = (Stu_ID, Proj_ID). Several columns are repeated.

Stu_ID	Stu_Name	Dept	Proj_ID	Proj_Name	Guide_Name	Guide_Phone
S1	Ananya	CSE	P1	Library System	Dr. Rao	98100
S1	Ananya	CSE	P2	Hostel Portal	Dr. Nair	98200
S2	Rohit	CSE	P1	Library System	Dr. Rao	98100
S3	Meera	ECE	P3	Antenna Array	Dr. Rao	98100

Everything is wrong with this table at once: “Ananya / CSE” is stored twice, “Library System / Dr. Rao / 98100” twice, and “Dr. Rao / 98100” three times. Each repetition is an opportunity for inconsistency.

13.2 Step 1: To 1NF

Rule to check: are all values atomic? **Result:** yes — every cell holds one value, so the relation is already in 1NF. This is worth stating explicitly in an answer: 1NF is about atomicity, and nothing here is a list.

13.3 Step 2: To 2NF

Rule to check: does every non-prime attribute depend on the **whole** key?

WORKED STEP — THE PARTIAL DEPENDENCIES

The key is `( Stu_ID, Proj_ID )`. Test each non-prime attribute:

Attribute	Determined by	Whole key needed?	Verdict
Stu_Name	Stu_ID	No — half the key suffices	**Partial ✘**
Dept	Stu_ID	No	**Partial ✘**
Proj_Name	Proj_ID	No	**Partial ✘**
Guide_Name	Proj_ID	No	**Partial ✘**
Guide_Phone	Proj_ID	No	**Partial ✘**

Every non-prime attribute depends on only half the key. The relation is emphatically not in 2NF.

Fix: split along the dependency lines. The student facts move to a Student relation, the project facts to a Project relation, and a junction relation keeps the many-to-many fact that a student works on a project.

Table 13.2 — Student (Stu_ID, Stu_Name, Dept).

Stu_ID	Stu_Name	Dept
S1	Ananya	CSE
S2	Rohit	CSE
S3	Meera	ECE

Table 13.3 — Project (Proj_ID, Proj_Name, Guide_Name).

Proj_ID	Proj_Name	Guide_Name
P1	Library System	Dr. Rao
P2	Hostel Portal	Dr. Nair
P3	Antenna Array	Dr. Rao

Table 13.4 — Works_On (Stu_ID, Proj_ID), the junction that preserves the original fact.

Stu_ID	Proj_ID
S1	P1
S1	P2
S2	P1
S3	P3

Every non-prime attribute now depends on the whole key of its own relation, so all three are in 2NF.

13.4 Step 3: To 3NF

Rule to check: is any non-prime attribute transitively dependent on a key?

WORKED STEP — THE TRANSITIVE DEPENDENCY

Look at Table 13.3, `Project ( Proj_ID, Proj_Name, Guide_Name )`. Apply the 3NF test to the dependency chain:

```
Proj_ID    → Guide_Name        (part of the key determines the guide — fine, Proj_ID is the key)
Guide_Name → Guide_Phone       (the guide determines the phone)
```

Is `Guide_Name` a super key of Project? No. Is `Guide_Phone` prime? No. Both conditions fail — so once `Guide_Phone` is present, `Proj_ID → Guide_Name → Guide_Phone` is a transitive dependency and Project would violate 3NF.

At this point in our decomposition the phone number has already been moved out with the project facts, so the Project relation as written (without `Guide_Phone`) is in 3NF. But the correct schema must store the phone somewhere, and the right place is a relation keyed by the guide:

Table 13.5 — Guide (Guide_Name, Guide_Phone) — the second-hop relation that 3NF demands.

Guide_Name	Guide_Phone
Dr. Rao	98100
Dr. Nair	98200

Notice the payoff: “Dr. Rao's phone is 98100” is now stored **once** rather than once per project.

13.5 Step 4: To BCNF

Rule to check: is every determinant of a non-trivial dependency a super key?

WORKED STEP — CHECKING EACH RELATION

Relation	Key	Non-trivial dependencies	Determinant a super key?	BCNF
Student	Stu_ID	Stu_ID → Stu_Name, Dept	Yes	✔
Project	Proj_ID	Proj_ID → Proj_Name, Guide_Name	Yes	✔
Works_On	(Stu_ID, Proj_ID)	none (no non-key attributes)	—	✔
Guide	Guide_Name	Guide_Name → Guide_Phone	Yes	✔

Every determinant is now the key of its own relation, so the schema is in BCNF.

13.6 Step 5: To 4NF

Rule to check: does any relation hold two independent multi-valued facts?

Now consider what the Project relation does not capture. A project may have several guides, and a guide may have several contact numbers. If a designer tried to store both in one relation:

```
Project_Guide ( Proj_ID, Guide_Name, Guide_Phone )   ← would violate 4NF
```

because `Proj_ID →→ Guide_Name` and `Guide_Name →→ Guide_Phone` are independent, producing a cross product. The fix, exactly as in Chapter 10, is two relations — which is what Tables 13.3 and 13.5 already are. This is why the schema above reaches 4NF as well as BCNF.

13.7 The Final Schema

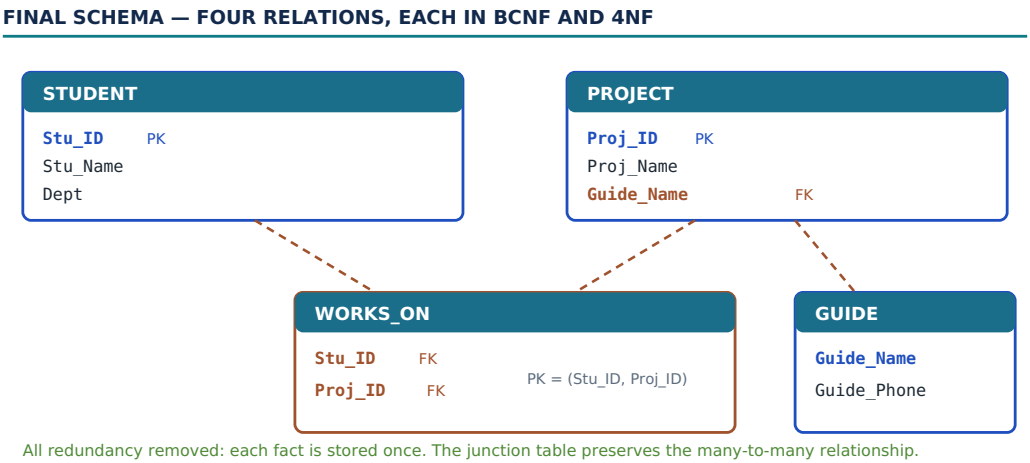


Figure 13.1 — The final schema. Four relations, each keyed by exactly the determinant that governs its attributes, and a junction table for the many-to-many relationship.

Table 13.6 — What each step achieved.

Step	Problem found	Action	Result
1NF	None — all values atomic	No change	Already atomic
2NF	Every non-prime attribute depended on half the key	Split into Student, Project, Works_On	Full dependencies only
3NF	Guide_Name → Guide_Phone is transitive from Proj_ID	Move the phone to a Guide relation	No transitive dependencies
BCNF	None remaining	Verified every determinant is a key	All relations in BCNF
4NF	None remaining — guides and phones already separated	Verified no independent multi-valued facts share a table	All relations in 4NF

THE PATTERN TO REMEMBER

Every step of this case study did the same thing: **find a determinant that is not the key, and make it the key of its own relation.** 2NF does it for determinants that are part of a composite key; 3NF does it for determinants that are non-prime; BCNF does it for any determinant; 4NF does it for multi-valued determinants. One idea, four applications.

Chapter summary

SUMMARY — CHAPTER 13

- Normalization is driven by dependencies, not by column lists: state the FDs first, then test.
- Find the key by closure; then test each non-prime attribute for partial dependency (2NF), each non-prime-to-non-prime chain for transitivity (3NF), each determinant for super-key status (BCNF), and each independent multi-valued fact for 4NF.
- Keep a junction relation whenever the original relation held a many-to-many fact.
- Verify losslessness at the end with the R1 ∩ R2 super-key test of Chapter 12.

14. Denormalization

Part 3 • deliberately adding redundancy back

Normalization is not an end in itself. It is a means to correctness, and it is paid for with joins. When joins become the bottleneck, a designer may deliberately reverse part of the normalization. The source covers this under the heading “Denormalization in Databases”, and its first sentence is the one that matters most.

14.1 Definition and Motivation

DEFINITION (SOURCE)

Denormalization is a database optimization technique in which we add redundant data to one or more tables. This can help us avoid costly joins in a relational database.

The source adds an essential clarification: **denormalization does not mean not doing normalization. It is an optimization technique that is applied after doing normalization.**

The source's reasoning runs as follows. In a traditional normalized database we store data in separate logical tables and try to minimise redundant data, striving to keep only one copy of each piece of data. The benefit is obvious: if a fact changes, it changes in one place. The drawback is equally obvious: when a query needs data from several tables, the database must join them, and if the tables are large that join can take an unnecessarily long time.

14.2 A Concrete Example

The source's example is a Courses table and a Teachers table. In a normalized design each Courses row stores the `teacherID` for a course but **not** the `teacherName`. To list all courses with the teacher's name, the database joins the two tables.

Table 14.1 — The normalized design (source's example).

Course	TeacherID
DBMS	T1
Operating Systems	T2
Networks	T1

Table 14.2 — The referenced Teachers table.

TeacherID	TeacherName
T1	Dr. Dhakar
T2	Dr. Verma

The query “list every course with its teacher's name” requires a join on `TeacherID`. The source states the trade-off in one line: in some ways this is great; if a teacher changes his or her name, we only have to update the name in one place. The drawback is that if tables are large, we may spend an unnecessarily long time doing joins on tables.

Denormalizing the Courses table means storing `TeacherName` directly in it:

Table 14.3 — The denormalized design: TeacherName is now stored in Courses as well.

Course	TeacherID	TeacherName
DBMS	T1	Dr. Dhakar
Operating Systems	T2	Dr. Verma
Networks	T1	Dr. Dhakar

Now the course list can be produced with no join at all — at the cost that "Dr. Dhakar" appears twice, and a name change must update every occurrence.

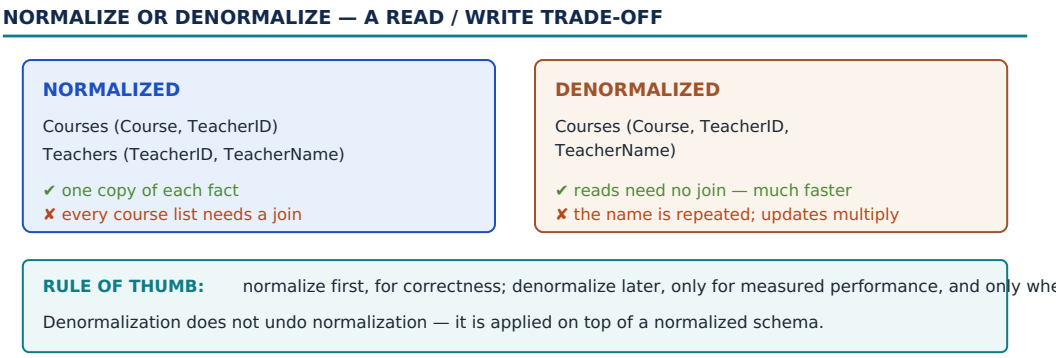


Figure 14.1 — Denormalization trades storage and update cost for read speed. It is an optimization applied after normalization, never instead of it.

14.3 Benefits and Trade-Offs

Table 14.4 — Denormalization: what it buys and what it costs.

Aspect	Normalized	Denormalized
Redundant data	Minimal — one copy per fact	Deliberately increased
Read queries needing joins	Many	Fewer or none
Query speed on large tables	Slower when joins are involved	Faster
Update cost	Low — change one row	High — change every copy
Risk of inconsistency	Low	Higher — copies can diverge
Storage	Smaller	Larger
When to use	Default for design	After measurement shows joins dominate

COMMON MISTAKE

Using denormalization as an excuse to skip normalization. The source is explicit: denormalization is applied **after** doing normalization. Designing a redundant schema from the start means you never had a correct design to optimize, and you inherit every anomaly of Chapter 6 with none of the justification.

Chapter summary

SUMMARY — CHAPTER 14

- **Denormalization** adds redundant data to avoid costly joins; it is an optimization, not a substitute for normalization.
- It is applied after normalization, when large-table joins dominate the workload.
- It buys read speed and costs update cost, storage and consistency risk.
- The source's example: storing `TeacherName` in the Courses table so a course list needs no join.

PART 4

Database Security

Confidentiality, integrity and availability · threats and their consequences · security levels from physical to network · encryption and key management · authentication and the three access-control models · the six control measures · the DBA's role

Source: UNIT 2 database-security slides — “Database Security: Introduction, Threats, Counter Measures”, including the access-control models, the authentication/authorization comparison and the DBA slides.

15. Database Security: Meaning, Importance and Scope

Part 4 • what is being protected, and why it is hard

The source defines database security in a single line and then spends several slides unpacking what that line implies. The unpacking is the interesting part: security is not one control but a stack of controls spanning six levels, and the threats it faces are correspondingly varied.

15.1 Definition

> **DEFINITION (SOURCE)**
>
> **Database security** is protection from malicious attempts to steal (view) or modify data.
>
> More fully: security of databases refers to the array of controls, tools, and procedures designed to ensure and safeguard confidentiality, integrity, and accessibility.

The source also gives the operational framing that exam answers usually want: database security ensures that sensitive and critical data stored in a database is secure from unauthorized access, misuse, or corruption, and the security measures include **access controls, encryption, authentication, and auditing** mechanisms.

15.2 The Five Things Security Must Protect

The source lists five aspects that database security must cover. They are worth learning as a list, because a “discuss the scope of database security” question is answered by naming them and giving an example of each.

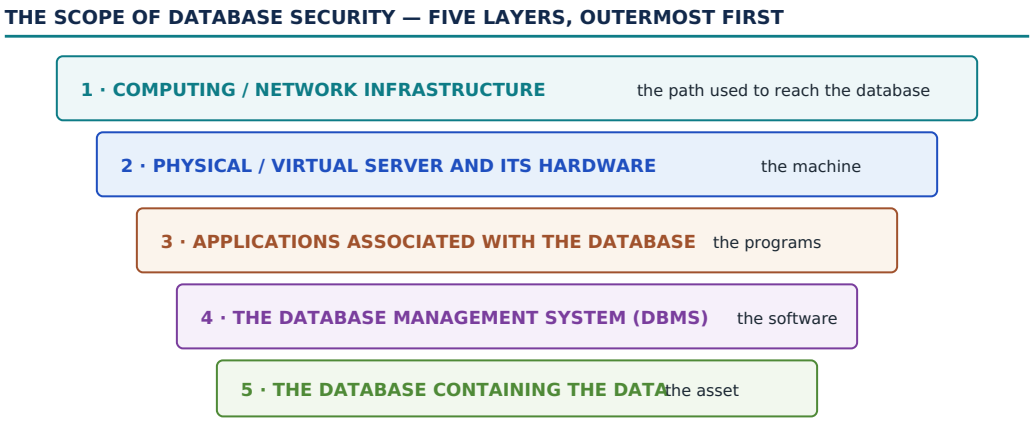


Figure 15.1 — Security must be applied at every layer, not only at the database itself. A weakness in any outer layer exposes the innermost one.

Table 15.1 — The five aspects the source says database security must safeguard.

#	Aspect (source wording)	A failure looks like
1	The database containing data	Data files copied or altered
2	Database management systems (DBMS)	Privilege escalation through the DBMS
3	Any applications associated with it	SQL injection through an application form
4	Physical database servers or the database server virtual, and the hardware that runs it	Disk stolen; server physically accessed
5	The infrastructure for computing or network used to connect to the database	Traffic intercepted in transit

15.3 The Usability Paradox

The source makes an observation that is worth quoting in any answer about the difficulty of the subject, because it explains why security is a design problem rather than a checklist:

> **THE SOURCE'S OBSERVATION**
>
> “Security of databases is a complicated and challenging task that requires all aspects of security practices and technologies. This is inherently at odds with the accessibility of databases. The more usable and accessible the database is, the more susceptible we are to threats from security. The more vulnerable it is to attacks and threats, the more difficult it is to access and utilize.”

This is a genuine trade-off, not a rhetorical flourish. Every control that blocks an attacker also blocks some legitimate work: stricter authentication slows logins, encryption costs processor time, and least-privilege grants force more administration. Security design is the art of choosing where on that spectrum a given system should sit.

15.4 Why Database Security Matters

The source gives four consequences of a breach. They are worth learning as a list because they are the “importance of database security” answer.

Table 15.2 — The four consequences of a breach (source).

Consequence	The source's explanation
Loss of intellectual property	Trade secrets, inventions or proprietary methods could be vital to maintaining an industry advantage. If they are stolen or disclosed and the competitive advantage is lost, it may be difficult to recover.
Damage to brand reputation	Customers or partners may not want to buy goods or services from a company they do not trust to protect their data.
Loss of business continuity	Some businesses cannot continue to function until a breach has been resolved.
Cost of repair and notification	Beyond notifying customers, the breached company must cover investigation and forensic services such as crisis management and triage repairs to affected systems.

Chapter summary

SUMMARY — CHAPTER 15

- Database security is protection from malicious attempts to steal or modify data; formally it is the array of controls, tools and procedures safeguarding **confidentiality, integrity and accessibility**.
- It must cover five layers: the data, the DBMS, the applications, the physical/virtual server and its hardware, and the computing/network infrastructure.
- Security is inherently at odds with accessibility — the trade-off is the central design tension.
- A breach costs intellectual property, reputation, business continuity and repair/notification expense.

16. Threats and the CIA Triad

Part 4 • what can go wrong, and how it is classified

Having established what security protects, the source turns to what threatens it. Its treatment is organised around the three properties security is meant to preserve, and around a concrete list of threat examples.

16.1 What a Threat Is

> **DEFINITION (SOURCE)**
> A **threat** is any intentional or accidental event that may adversely affect the system.

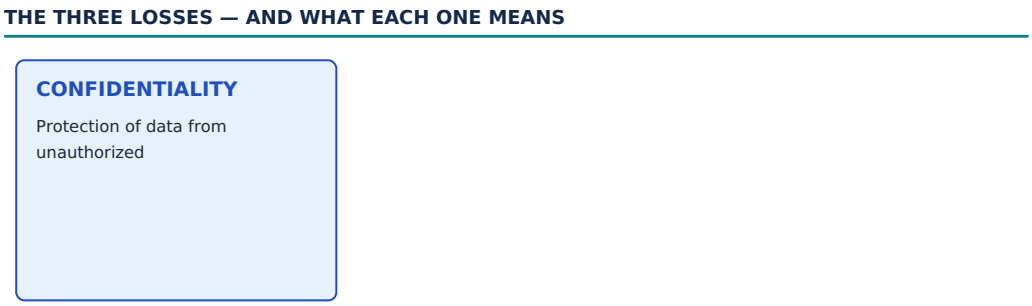
Two words in that definition carry weight. **Intentional** covers an attacker using another person's login or a hacker entering illegally. **Accidental** covers a careless administrator deleting a critical file or a user mistyping a destructive statement. Security that addresses only attackers leaves half the risk untouched — and the source's physical-level discussion (Section 17.3) explicitly names administrator error as a threat.

Examples of threats (source list)

- Using another person's log-in name to access data
- Unauthorized copying of data
- Program or data alteration
- Illegal entry by a hacker
- Viruses

16.2 Loss of Integrity, Availability and Confidentiality

The source's “Threats to databases” material is built around three losses, which together form what is usually called the **CIA triad** (Confidentiality, Integrity, Availability). Learn each definition exactly as the source gives it, because the wording carries the distinctions.



disclosure. Broken by: reading data you should not see Restored by: access control, encryption, inference control INTEGRITY Protection from improper **modification** — creation, insertion, update, deletion. Broken by: any unauthorized change, intentional or not Restored by: constraints, flow control, auditing AVAILABILITY Making objects available to a user or program that has a **legitimate right** to them. Broken by: denial of service, hardware failure, disaster Restored by: backup, replication, recovery Note the tension: confidentiality pushes toward restricting access, availability pushes toward permitting it. A secure system balances both. The source defines all three losses separately, and exam answers should too.

Figure 16.1 — The three losses the source names, with the type of attack that causes each and the class of control that addresses it.

THE THREE LOSSES, IN THE SOURCE'S WORDS

Loss of integrity. Database integrity refers to the requirement that information be protected from improper modification. Modification of data includes creation, insertion, updating, changing the status of data, and deletion. Integrity is lost if unauthorized changes are made to the data by either intentional or accidental acts. If the loss of system or data integrity is not corrected, continued use of the contaminated system or corrupted data could result in inaccuracy, fraud, or erroneous decisions.

Loss of availability. Database availability refers to making objects available to a human user or a program to which they have a legitimate right.

Loss of confidentiality. Database confidentiality refers to the protection of data from unauthorized disclosure. The impact of unauthorized disclosure of confidential information can range from violation of the Data Privacy Act to the jeopardization of national security. Unauthorized, unanticipated, or unintentional disclosure could result in loss of public confidence, embarrassment, or legal action against the organization.

COMMON MISTAKE

Treating availability as the least important of the three. The source defines it as a full member of the triad, and the reason is practical: a database that is correct and secret but unreachable has failed its users just as completely as one that has been corrupted. Note also that the source's definition is deliberately qualified — availability is owed to those with a legitimate right, not to everyone.

16.3 Threat Categories

Threats can be grouped by the layer at which they act. The table below collects the source's examples and places each against the triad property it most directly threatens.

Table 16.1 — Threats grouped by the property they endanger.

Threat	Layer	Primarily threatens
Using another person's log-in name	Human / application	Confidentiality, integrity
Unauthorized copying of data	Human / network	Confidentiality
Program or data alteration	Application / database	Integrity
Illegal entry by a hacker	Network / operating system	All three
Viruses	Operating system	Availability, integrity
Administrator error (e.g. deleting critical files)	Physical / operating system	Availability
Eavesdropping	Network	Confidentiality
Masquerading	Network	Confidentiality, integrity
Person-in-the-middle attack	Network	Confidentiality, integrity

Chapter summary

SUMMARY — CHAPTER 16

- A **threat** is any intentional or accidental event that may adversely affect the system.
- The three losses are **integrity** (improper modification), **availability** (objects withheld from legitimate users) and **confidentiality** (unauthorized disclosure).
- Each loss has a characteristic class of control: integrity → constraints and auditing; availability → backup and recovery; confidentiality → access control and encryption.
- Threats act at different layers, so controls must too.

17. Levels of Data Security, Physical and OS Security

Part 4 • where a security function should be enforced

One of the source's most useful ideas is that security is not a single mechanism applied at a single place. It is a set of decisions about which level should handle which function. This chapter sets out the levels and then examines the two outermost ones in detail.

17.1 Types of Security Issues

The source frames database security as “a broad area that addresses many issues”, and lists four kinds. They are worth knowing because they show that security is partly legal and organisational, not purely technical.

Table 17.1 — The four kinds of security issue (source).

Kind of issue	The source's description	Example
Legal and ethical	Various legal and ethical issues regarding the right to access certain information	Some information is deemed private and cannot be accessed legally by unauthorized organizations or persons
Policy	Policy issues at governmental, institutional or corporate level about what information should not be publicly available	Credit ratings and personal medical records
System-related	The system levels at which security functions should be enforced	Should a function be handled at the physical hardware level, the operating system level, or the DBMS level?
Multiple security levels	The need in some organizations to identify multiple security levels and categorize data and users by them	Top secret, secret, confidential, and unclassified

17.2 The Six Levels of Data Security

The source lists six levels, from the human being at the top down to the physical hardware at the bottom. The ordering is deliberate: the highest level is the least controllable.

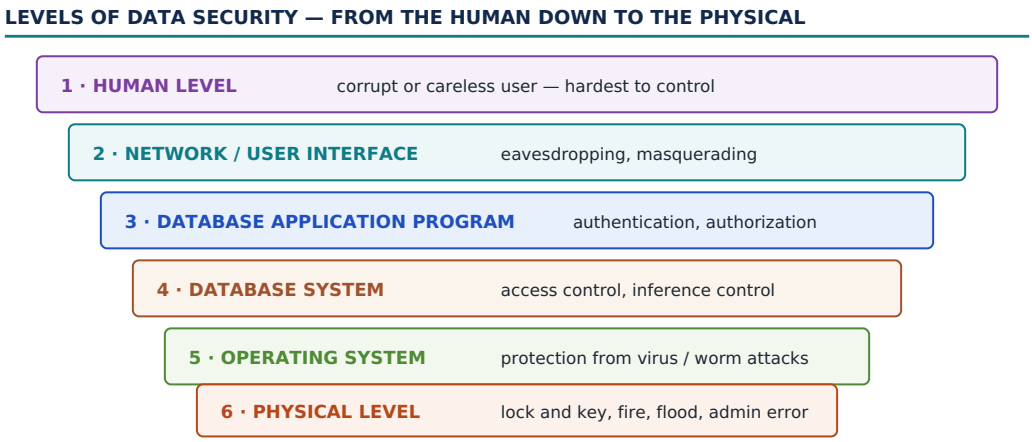


Figure 17.1 — The six levels of data security named by the source. A defensible system applies controls at all six.

17.3 Physical-Level Security

The source treats the physical level under the heading “Physical/OS Security” and gives both the threats and a concrete remedy.

Table 17.2 — Physical-level security: threats and remedies (source).

Concern	Source's point
Traditional protection	Lock-and-key security
Protection from disaster	Protection from floods, fire, etc. — the source cites the World Trade Center attack of 9/11, fires at IITM, and a WWW conference website as examples of physical disasters that destroyed data.
Protection from administrator error	For example, deleting critical files
Solution	**Remote backup for disaster recovery**, plus **archival backup** (for example DVDs or tapes)

EXPLANATION — WHY THE REMEDY IS BACKUP

Physical threats cannot be prevented by clever software: a fire does not care how strong your password is. For threats that cannot be prevented, the only workable strategy is to make the loss recoverable. That is why the source's answer at this level is not a control but a copy — remote backup for fast recovery, and archival backup for a durable record.

17.4 Operating-System-Level Security

The source's statement here is short and blunt: at the operating-system level, protection from virus and worm attacks is critical. The reasoning is straightforward once the six-level model is in view. The DBMS runs on top of the operating system, so an attacker who controls the OS can read the database files directly, bypassing every DBMS control. Operating-system security is therefore not optional plumbing; it is the foundation the DBMS's own controls stand on.

A USEFUL WAY TO REMEMBER THE LEVELS

Ask of any proposed control: which level does it live at, and what does it assume about the levels below? Application-level authentication assumes the OS is not compromised. DBMS access control assumes the files are not readable directly. Physical locks assume nothing at all, which is why they are the last line.

Chapter summary

SUMMARY — CHAPTER 17

- Security issues are legal/ethical, policy, system-related, and multi-level classification problems.
- The six levels of data security are: human, network/user interface, database application program, database system, operating system, and physical.
- **Physical security** covers lock-and-key, disaster protection and administrator error; the remedy is remote backup plus archival backup.
- **OS-level security** is critical for protection from virus and worm attacks, because the DBMS depends on the OS.

18. Encryption and Key Management

Part 4 • protecting data when the storage medium itself is lost

The source's encryption material is short but asks a good question that motivates the whole technique. It is worth following the question rather than merely listing the answer.

18.1 Why Encrypt

The source's motivating scenario: what if a laptop, disk or USB key with critical data is lost?

Notice why this scenario defeats the controls discussed so far. Access control and authentication protect data while the DBMS is running and reachable. But a lost disk is not a running database — it is a physical object in a stranger's hands. No password prompt stands between the finder and the bytes. The only control that helps is one that makes the bytes meaningless without a secret.

18.2 Storage-Level Encryption and Key Management

THE SOURCE'S APPROACH

Partial solution: encrypt the database at storage level, transparent to the application.

Main issue: key management.

Common practice: the user provides a decryption key (password) when the database is started up. This is supported by many database systems.

The phrase **“transparent to the application”** is the design goal. The application continues to issue ordinary SQL; the DBMS encrypts on write and decrypts on read. This means no application change is needed — but it also means the encryption cannot protect against an attacker who is already inside the running database with valid credentials, because at that point the DBMS is decrypting for them.

EXPLANATION — WHY KEY MANAGEMENT IS THE HARD PART

Encryption itself is a solved problem; keeping the key safe is not. The source names key management as the main issue for three connected reasons:

- The key must be available when the database starts, so it cannot simply be stored beside the data — a lost disk would then carry its own key.
- If the key is supplied by a person as a password at startup, the system is only as secure as that person and that password (see Section 20.2).
- If the key is lost, the data is permanently unreadable. Encryption converts a confidentiality risk into an availability risk, which is exactly the tension described in Figure 16.1.

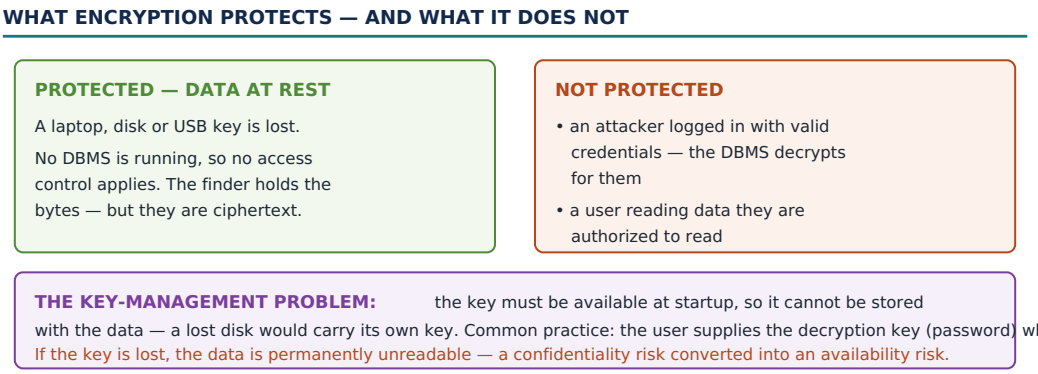


Figure 18.1 — Encryption is the only control that still works when the database is not running. Its hard part is not the cipher but the key.

18.3 Practical Guidance from the Source

The source's closing guidance on encryption is a practice, not a theory: it is standard practice now to encrypt credit card information, and other sensitive information. The reason is the same as the motivating scenario — credit card numbers are precisely the data whose disclosure is most damaging and whose storage medium is most likely to travel (laptops, backups, portable drives).

COMMON MISTAKE

Believing that encryption makes a database secure on its own. Encryption protects **data at rest** — the bytes on the medium. It does nothing about an attacker who logs in with valid credentials, nothing about a careless user who reads data they are authorized to read, and nothing about an application that is tricked into revealing data. Encryption is one control among the six in Chapter 21, not a substitute for the others.

Chapter summary

SUMMARY — CHAPTER 18

- Encryption is motivated by loss of the storage medium itself, which no access-control mechanism can prevent.
- The approach is storage-level encryption that is transparent to the application.
- The main problem is **key management**: the key must be available at startup yet not stored with the data, and losing it means losing the data.
- It is now standard practice to encrypt credit card and other sensitive information.

19. Authentication, Authorization and Access-Control Models

Part 4 • who are you, and what may you do

The source devotes a whole slide to the difference between authentication and authorization, and two more to the three access-control models. Together they answer the two questions every database must ask before it serves a row of data.

19.1 Authentication versus Authorization

DEFINITIONS (SOURCE)

Authentication: who are you? Prove it!

Authorization: what you are allowed to do.

The source's framing is that authentication and authorization mechanisms allow specific users access only to the required data. Authentication establishes identity; authorization decides what that identity may do.

Table 19.1 — Authentication versus authorization, following the source's comparison slide.

Point of comparison	Authentication	Authorization
Purpose	Decides whether users are what they claim they are	Decides which user is allowed to access what
How it works	User identity is verified via a username and password, or by answering a security question	Verifies whether the given access is allowed by following rules and policies
Order	Takes place **before** authorization	Takes place **once authentication is completed**
Methods	CAPTCHA test, biometric authentication, passwords	Permissions — read and write access to files and to the database; deciding the roles of users for accessing data
Who controls it	Controlled through the server, for finding out who is accessing what data or site	The server decides whether the client has authority to access a resource or not
Example	Employees authenticating through the network before opening the company mails	Once an employee is authenticated, the different systems decide which information will be accessed by which employee

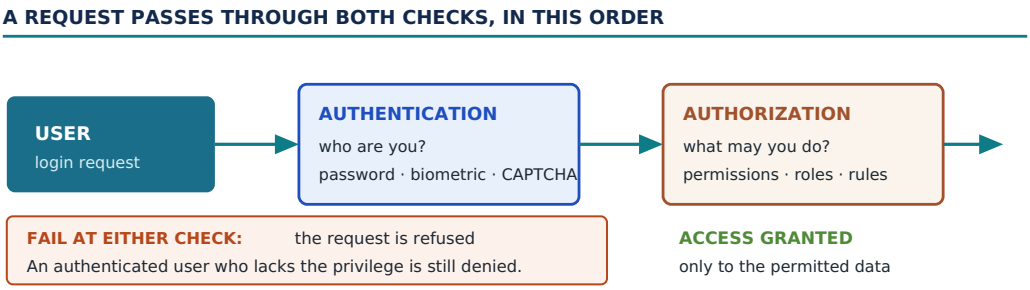


Figure 19.1 — Authentication always precedes authorization. Passing the first check is necessary but not sufficient; the second check is what actually protects the data.

COMMON MISTAKE

Using the two words interchangeably. An answer that says “authentication controls what a user can access” is wrong in a way that costs marks. Authentication answers who; authorization answers what. Keep the order in mind: authentication is always completed first.

19.2 Discretionary Access Control (DAC)

DEFINITION (SOURCE)

Discretionary Access Control: users have control over their own data, and permissions are granted using **Access Control Lists (ACLs)**.

The source adds the mechanism: the typical method of enforcing DAC in a database system is based on **granting and revoking privileges**. These are used to grant privileges to users, including the capability to access specific data files, records or fields in a specified mode — such as read, insert, delete, or update.

The word “discretionary” is the key: the owner of the data decides who else may use it. That flexibility is also the model's weakness, because a user who can read a table can usually copy it somewhere the owner no longer controls.

19.3 Mandatory Access Control (MAC)

DEFINITION (SOURCE)

Mandatory Access Control is based on a **central authority** assigning **security labels** to data and users. These labels define access levels, ensuring that users can only access data classified within their clearance level. MAC is commonly used in government and military applications, where strict control over data access is required.

The decisive difference from DAC is that the individual user has no discretion at all. A clearance level and a classification label are compared by the system, and the comparison — not the owner's preference — decides access. This is the model that implements the “top secret, secret, confidential, unclassified” hierarchy the source mentioned under multiple security levels.

19.4 Role-Based Access Control (RBAC)

DEFINITION (SOURCE)

Role-Based Access Control is based on **predefined roles** assigned to users. Each role has specific privileges, including the ability to read, insert, delete, or update data. Instead of granting permissions to individual users, RBAC assigns access based on **job responsibilities**, making it efficient for large organizations.

RBAC's advantage is administrative. In a large organization, granting privileges user by user means repeating the same grants hundreds of times and updating them whenever someone changes jobs. Granting a role once and assigning users to it means the permissions are described in one place.

19.5 Comparing the Three Models

Table 19.2 — The three access-control models compared.

Aspect	DAC	MAC	RBAC
Who decides access	The data owner	A central authority	The role definition
Basis of the decision	Granted/revoked privileges, ACLs	Security labels vs clearance levels	Job responsibilities
Flexibility	High	Low (deliberately)	Medium
Administrative effort in large organizations	High — per user	High but centralized	Low — per role
Typical setting	Commercial database systems	Government and military	Large enterprises
Weakness	Owner can leak data they can read	Rigid; label management is costly	Role explosion if roles multiply

Chapter summary

SUMMARY — CHAPTER 19

- **Authentication** proves identity (“who are you?”); **authorization** decides permitted actions (“what may you do?”). Authentication always happens first.
- **DAC** — owner-controlled, implemented with grants, revokes and ACLs.
- **MAC** — central authority assigns security labels; users are limited to their clearance level; used in government and military systems.
- **RBAC** — permissions attach to predefined roles, which are assigned to users; efficient for large organizations.

20. Network and Human-Level Security

Part 4 • protecting data in transit, and the weakest link

The two outer levels of Figure 17.1 need their own treatment, because the attacks there are unlike the ones a DBMS can handle internally. On the network, data is in transit and can be intercepted; at the human level, the credential itself can be guessed or shared.

20.1 Network-Level Threats

THE SOURCE'S NETWORK-LEVEL MATERIAL

At the network level, encryption **must** be used to prevent:

- **Eavesdropping** — unauthorized reading of messages.
- **Masquerading** — pretending to be an authorized user or legitimate site, or sending messages supposedly from authorized users.

These are handled by secure HTTP — **https://**. It must also prevent **person-in-the-middle** attacks: for example, someone impersonates the seller or the bank/credit-card company and fools the buyer into revealing information.

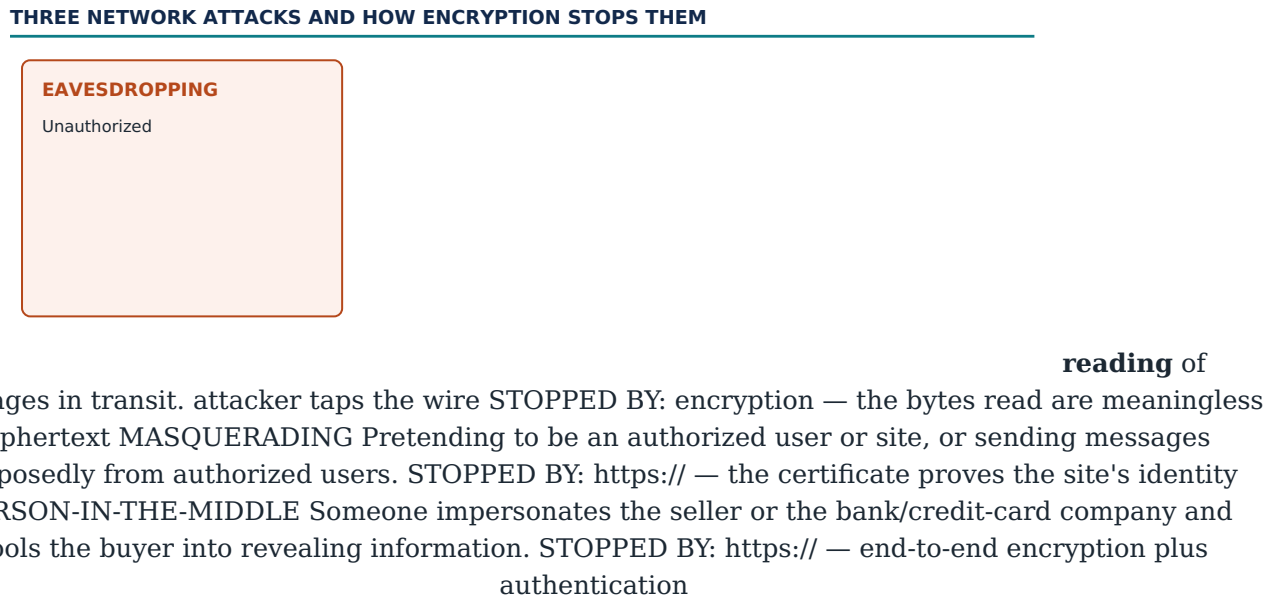


Figure 20.1 — The three network attacks the source names. Encryption addresses all three, and https:// is the practical implementation.

EXPLANATION — WHY ONE CONTROL COVERS THREE ATTACKS

All three attacks fail against the same thing: a channel whose contents cannot be read and whose endpoints can be authenticated. Eavesdropping fails because the attacker cannot read the ciphertext. Masquerading and person-in-the-middle fail because the client can verify that it is talking to the real server, not an impostor. That is precisely what https:// provides — encryption plus server authentication.

20.2 Passwords

THE SOURCE'S POINT

Password. Most users abuse passwords. For example: they choose easy-to-guess passwords, and they share passwords with others.

The source's phrasing is deliberately blunt, and the two failures it names are worth separating because they have different fixes:

Table 20.1 — The two password failures the source names, and the control for each.

Failure	Why it is dangerous	Characteristic control
Easy-to-guess password	An attacker can guess it without any technical attack at all	Password policy: length, complexity, expiry
Sharing passwords with others	Destroys the link between identity and person — authentication still succeeds, but it no longer means what it should	Policy plus audit; technical controls cannot detect consent

COMMON MISTAKE

Thinking that password sharing is a minor issue. It breaks the entire basis of authentication: every later control assumes the identity is genuine. If three people use one login, no access-control decision — and no audit trail — can be trusted.

20.3 Smartcards

THE SOURCE'S POINT

Smartcards. Need the smartcard itself, **plus** a PIN or password.

A smartcard adds a second factor: something the user has, combined with something the user knows. This directly addresses both password failures. A shared password is no longer enough, because the attacker also needs the physical card; and a guessed password is no longer enough for the same reason. The source's “Bill Gates” note in the slide is the memorable illustration of the residual risk — a smartcard can be lost or stolen, and if the PIN is written down beside it, both factors travel together.

THE SECURITY PRINCIPLE HERE

Two independent factors are stronger than one, because the two failure modes are independent. A password can be guessed or shared but not stolen from your pocket; a card can be stolen but not guessed. Requiring both means an attacker must succeed at two different kinds of attack.

Chapter summary

SUMMARY — CHAPTER 20

- Network threats are **eavesdropping** (unauthorized reading), **masquerading** (impersonating a user or site) and **person-in-the-middle** attacks (impersonating a seller or bank to extract information).
- All three are handled by encryption and authentication, implemented in practice by **https://**.
- **Passwords** fail in two ways: they are easy to guess, and they are shared. Both break the link between identity and person.
- **Smartcards** require the card plus a PIN or password, adding a second independent factor.

21. Control Methods of Database Security

Part 4 • the six measures that actually implement security

Chapters 15 to 20 described what is threatened and where. This chapter is the answer: the six control measures the source names, each defined and then tied back to the threat it addresses.

21.1 The Six Control Measures

THE SOURCE'S LIST

The following are the main control measures used to provide security of data in databases:

1. Authentication
2. Access control
3. Inference control
4. Flow control
5. Database security applying statistical method
6. Encryption

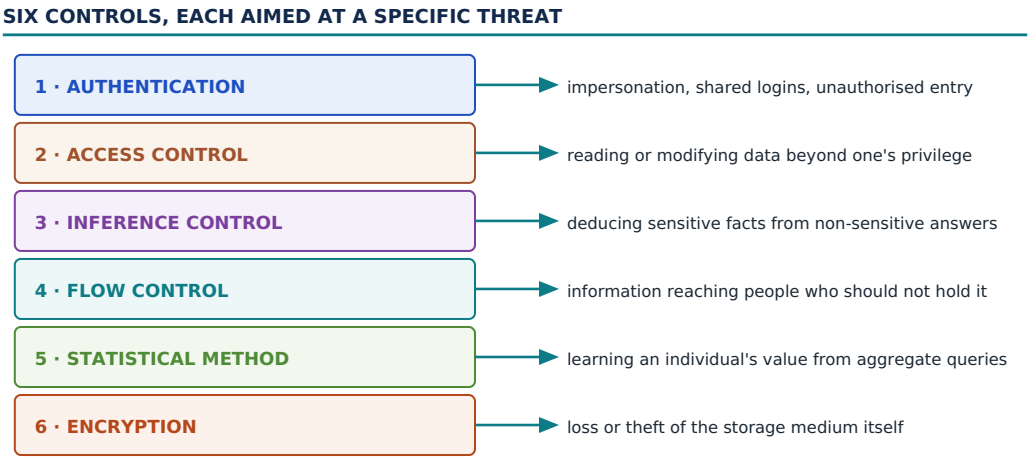


Figure 21.1 — The six control measures, each paired with the threat it is designed to stop. Notice that no two controls address the same threat, which is why all six are needed.

21.2 Authentication

DEFINITION (SOURCE)

Authentication is the process of confirming whether the user logs in according to the rights provided to them to perform the activities of the database. A particular user can log in only up to their privilege, and cannot access other sensitive data. The privilege of accessing sensitive data is restricted by using authentication.

By using authentication tools such as **biometrics** — retina and fingerprints — the database can be protected from unauthorized or malicious users.

Read the source's wording carefully, because it makes a claim that is easy to misstate. Authentication restricts the privilege of accessing sensitive data, but it does not by itself decide what a user may do — that is authorization (Section 19.1). Authentication is the gate: it establishes who is asking.

21.3 Access Control

DEFINITION (SOURCE)

The security mechanism of a DBMS must include provisions for **restricting access to the database by unauthorized users**. Access control is done by creating user accounts and controlling the login process through the DBMS, so that access to sensitive data is possible only for those database users who are allowed to access such data, and access by unauthorized persons is restricted.

The database system must also **keep track of all operations performed by a certain user throughout the entire login time.**

Two distinct duties are named here, and both matter:

Table 21.1 — The two duties of access control.

Duty	What it means	Why it matters
Restrict access	Create accounts; decide, per user, what may be read, inserted, deleted or updated	Prevents unauthorized reading and modification
Track operations	Keep a record of everything a user did during the whole login session	Makes after-the-fact investigation possible, and deters misuse by insiders

The three access-control models that implement the first duty — DAC, MAC and RBAC — were covered in Sections 19.2 to 19.4. The second duty is what is usually called auditing, and the source mentions it as one of the four security measures alongside access controls, encryption and authentication (Section 15.1).

21.4 Inference Control

DEFINITION (SOURCE)

Inference control is the countermeasure to the statistical-database security problem. It is used to prevent the user from completing any **inference channel**, and it protects sensitive information from **indirect disclosure**. Inferences are of two types: **identity disclosure** or **attribute disclosure**.

The threat inference control addresses is subtle and worth spelling out. No individual query breaks a rule. Each answer is a legitimate statistic about a group. But a sequence of legitimate answers can be combined to reveal a fact about one person — a fact that no single query was allowed to return.

WORKED STEP — HOW INFERENCE LEAKS A SECRET

Suppose the rule is that no query may return data about a single employee's salary.

```
Q1: SELECT AVG(salary) FROM emp;                          -- average over 100 employees: fine
Q2: SELECT AVG(salary) FROM emp WHERE dept = 'CSE';       -- average over 40: fine
Q3: SELECT AVG(salary) FROM emp WHERE dept = 'CSE'
      AND name <> 'Ananya';                              -- average over 39: fine
```

Each query is a statistic over a group, so each is permitted. But Q2 and Q3 together let the questioner solve for Ananya's salary exactly:

$$\text{salary(Ananya)} = 40 \times \text{AVG(Q2)} - 39 \times \text{AVG(Q3)}$$

The sensitive value has been disclosed without any single query violating the rule. That is **indirect disclosure**, and inference control exists to prevent it — for example by refusing queries whose result set is too small.

21.5 Flow Control and Covert Channels

DEFINITION (SOURCE)

Flow control prevents information from flowing in a way that reaches unauthorized users. The pathways by which information flows implicitly in ways that violate the privacy policy of a company are called **covert channels**.

Note the difference from access control. Access control asks “may this user read this item?” Flow control asks “may this fact move from where it is to where this user is?” A user might be perfectly entitled to read two separate values yet not entitled to the combination — for example, entitled to see project codes and entitled to see salaries, but not the mapping between them. Flow control restricts the propagation of information rather than the access to objects.

FROM THE SOURCE — A TERMINOLOGY NOTE

The source writes “convert channels”. The standard term in the security literature is **covert channels**, and the source's own definition (“pathways for information to flow implicitly in ways that violate the privacy policy”) is the standard definition of a covert channel. These notes use “covert channel” and note the source's spelling here so that you recognise both.

21.6 Statistical Database Security

DEFINITION (SOURCE)

Statistical database security focuses on the protection of confidential individual values stored in and used for statistical purposes, and used to retrieve summaries of values based on categories. It does not permit retrieval of individual information.

It allows access to the database to get statistical information — such as the number of employees in the company — but not to access the detailed confidential or personal information about a specific individual employee.

The source's example is the clearest way to hold the distinction: a count of employees is permitted; one employee's salary is not. The permitted class is **aggregate**; the forbidden class is **individual**. Inference control (Section 21.4) is the mechanism that keeps the boundary intact when a clever sequence of aggregates threatens to cross it.

21.7 Encryption

DEFINITION (SOURCE)

Encryption is mainly used to protect sensitive data such as credit card numbers, OTP numbers, and other sensitive numbers. The data is encoded using encoding algorithms.

Chapter 18 covered encryption in detail, including the key-management problem that makes it harder than it looks. In the list of six controls, encryption's distinct role is that it is the only one that still

works when the database is not running — that is, when the attacker holds the storage medium rather than a session.

21.8 Threat-to-Control Mapping

The single most useful revision table in this part: for each threat, which control addresses it and which control does not.

Table 21.2 — Threats against the control that addresses them.

Threat	Control that addresses it	Why the other controls do not help
An attacker logs in as another user	Authentication	Access control would faithfully grant that user's privileges to the impostor
A legitimate user reads data beyond their need	Access control	Authentication is satisfied — the user is genuine
A user deduces an individual value from aggregates	Inference control	Every individual query is authorized; only the sequence is dangerous
Information leaks to an unauthorized destination	Flow control	Each access may be individually permitted
Individual data exposed through statistics	Statistical database security	Access control works on objects, not on aggregate/individual boundaries
A disk or laptop with data is lost	Encryption	There is no running DBMS to enforce anything
A fire or flood destroys the server	Backup (Chapter 17)	No control can prevent a physical disaster

Chapter summary

SUMMARY — CHAPTER 21

- The six control measures are **authentication, access control, inference control, flow control, statistical database security** and **encryption**.
- **Authentication** confirms identity, using passwords or biometrics such as retina and fingerprints.
- **Access control** creates accounts and restricts access, and also **tracks all operations** a user performs during the login.
- **Inference control** prevents indirect disclosure through inference channels; inferences are identity disclosure or attribute disclosure.
- **Flow control** stops information reaching unauthorized users; the illicit pathways are **covert channels**.
- **Statistical database security** permits aggregate information but not individual values.
- **Encryption** encodes sensitive data such as credit card and OTP numbers.

22. The DBA, Privileges and Statistical Database Security

Part 4 • who owns the security policy

Every control in Chapter 21 needs an administrator. The source closes its security material by describing that role, because the controls are only as good as the authority that assigns them.

22.1 Who the DBA Is

DEFINITION (SOURCE)

The **database administrator (DBA)** is the **central authority** for managing a database system, and is responsible for the **overall security** of the database system.

The phrase “central authority” connects directly to the access-control models of Chapter 19. In DAC the DBA is the source of grants; in MAC the DBA is the central authority that assigns security labels; in RBAC the DBA defines the roles. Whichever model is in use, someone must hold the authority to make the initial decisions, and that is the DBA.

22.2 DBA Responsibilities

RESPONSIBILITIES (SOURCE)

The DBA's responsibilities include:

- **Granting privileges** to users who need to use the system.
- **Classifying users and data** in accordance with the policy of the organization.

The DBA is responsible for the overall security of the database system.

The two responsibilities map onto the two ideas already established. Granting privileges is the discretionary act — DAC's granting and revoking (Section 19.2). Classifying users and data is the mandatory act — assigning the security labels and clearance levels of MAC (Section 19.3). A DBA therefore operates in both worlds at once.

22.3 The DBA / Superuser Account

THE DBA ACCOUNT (SOURCE)

The DBA has a **DBA account** in the DBMS. Sometimes these are called a **system** or **superuser** account. These accounts provide powerful capabilities such as:

1. Account creation
2. Privilege granting
3. Privilege revocation
4. Security level assignment

Action 1 is **access control**, whereas actions 2 and 3 are **discretionary access**, and action 4 is used to control **mandatory authorization**.

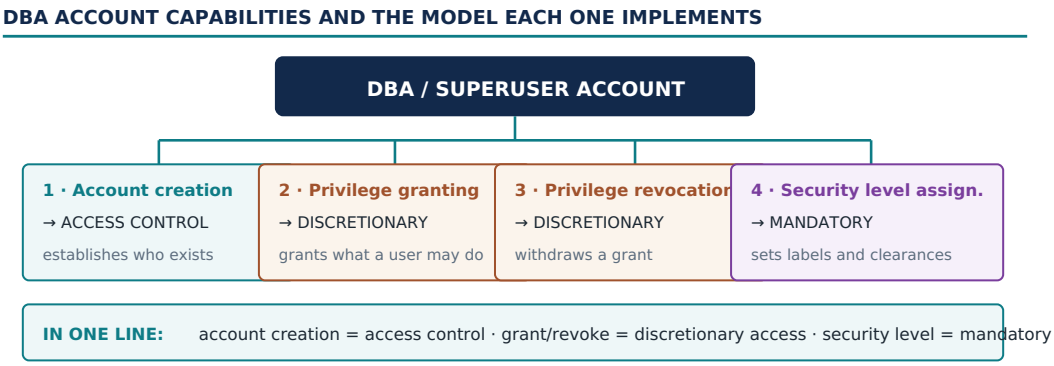


Figure 22.1 — The four DBA account capabilities and the security model each one implements. This mapping is the source's own.

COMMON MISTAKE

Treating the DBA account as just another user account. The source calls its capabilities “powerful”, and the reason is that it can create accounts, grant and revoke privileges, and assign security levels. An attacker who obtains the DBA account does not need to defeat any control — the attacker is the control. This is why DBA credentials require the strongest protection in the system.

Chapter summary

SUMMARY — CHAPTER 22

- The **DBA** is the central authority for managing the database system and is responsible for its overall security.
- Responsibilities: granting privileges to users who need them, and classifying users and data according to organizational policy.
- The DBA/superuser account provides account creation, privilege granting, privilege revocation and security level assignment.
- Account creation is access control; granting and revocation are discretionary access; security level assignment controls mandatory authorization.

PART 5

PL/SQL Control Structures

Conditional control: IF-THEN, IF-THEN-ELSE, ELSIF and CASE · iterative control: basic LOOP, EXIT, EXIT-WHEN, WHILE, FOR and REVERSE · sequential control: GOTO, labels and NULL

Source: UNIT 2, Chapters 2.3.1 (Conditional Control), 2.3.2 (Iterative Control) and 2.3.3 (Sequential Control) — every program, output and restriction below is the source's own.

23. PL/SQL Control Structures: Overview

Part 5 • what control structures are for, and the block that holds them

Up to this point the notes have dealt with the structure of data — keys, dependencies, normal forms — and with protecting it. This part turns to programming it. PL/SQL adds procedural statements to SQL so that a database can make decisions, repeat work and branch. The statements that do this are called **control structures**, and they are the subject of this part.

23.1 What Control Structures Are For

> **DEFINITION (SOURCE)**
>
> **PL/SQL control structures** are used to control the flow of execution. PL/SQL provides different kinds of statements to provide such procedural capabilities. These statements are almost the same as those provided by other languages.

The last sentence is worth taking seriously. If you have written a conditional or a loop in any language, you already understand the concepts here. What is new is only the PL/SQL syntax and two Oracle-specific behaviours: the `EXIT WHEN` shortcut (Section 25.2) and the fact that a `FOR` loop's counter cannot be assigned (Section 25.6).

23.2 The Three Categories

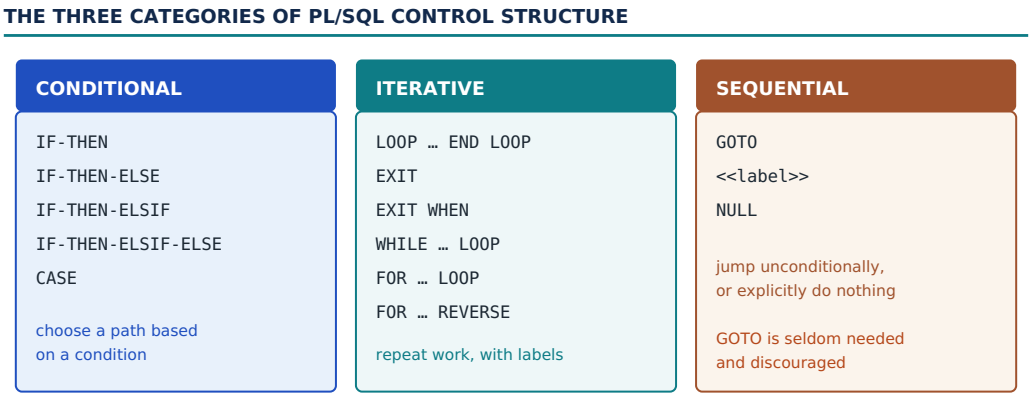


Figure 23.1 — The three categories, exactly as the source classifies them, with the statements belonging to each.

Table 23.1 — The three categories at a glance.

Category	Question it answers	Statements
Conditional control	Which path should run?	`IF` in its four forms, `CASE`
Iterative control	How many times should this run?	`LOOP`, `EXIT`, `EXIT WHEN`, `WHILE`, `FOR`
Sequential control	Where should control go next?	`GOTO`, labels, `NULL`

23.3 The Anonymous Block

Every PL/SQL program in this part has the same shape. It is worth naming the parts once, because every example below uses them and the source's programs assume them.

```
DECLARE
   -- declarations: variables, constants, cursors
BEGIN
   -- executable statements: the control structures live here
EXCEPTION
   -- error handling (optional)
END;
/
```

Table 23.2 — The parts of an anonymous PL/SQL block.

Section	Purpose	Required?
`DECLARE`	Declare variables, constants, cursors	Optional — omit if nothing to declare
`BEGIN … END;`	The executable body	Required
`EXCEPTION`	Handle run-time errors	Optional
`/`	Tells SQL*Plus to execute the block	Required when submitted from SQL*Plus

23.4 SET SERVEROUTPUT and the Substitution Variable

Two SQL*Plus conveniences appear in almost every example in the source, and understanding them prevents a lot of confusion when you run the programs yourself.

SET SERVEROUTPUT ON

`DBMS_OUTPUT.PUT_LINE` writes to a buffer on the server. SQL*Plus will not display that buffer unless you first run:

```
SET SERVEROUTPUT ON
```

Without it, a program runs successfully and prints nothing — a classic first-day mistake.

THE SUBSTITUTION VARIABLE &NAME

A name prefixed with `&` is a **substitution variable**. SQL*Plus prompts for a value and substitutes it into the text of the program before the program is compiled. That is why the transcripts in this part show lines such as:

```
Enter value for num1: 19
old 2: n1 NUMBER := &num1;
new 2: n1 NUMBER := 19;
```

The `old` and `new` lines are SQL*Plus showing the substitution it made. They are not produced by your program, and they are not part of its output — but they appear in the transcript, so these notes reproduce them exactly as the source shows them.

COMMON MISTAKE

Forgetting the slash (`/`) at the end of a block, or the semicolon at the end of a statement. In SQL*Plus, the semicolon ends a statement and the slash executes the block. A block without a slash just sits there, unexecuted, and nothing appears to happen.

Chapter summary

SUMMARY — CHAPTER 23

- Control structures control the flow of execution; PL/SQL's are similar to those of other languages.
- Three categories: **conditional** (IF forms and CASE), **iterative** (LOOP, EXIT, EXIT WHEN, WHILE, FOR), and **sequential** (GOTO, labels, NULL).
- Programs are anonymous blocks: `DECLARE`, `BEGIN … END;`, optional `EXCEPTION`, and a terminating `/`.
- `SET SERVEROUTPUT ON` is needed to see `DBMS_OUTPUT` text; `&name` is a substitution variable typed at the prompt.

24. Conditional Control: IF and CASE

Part 5 • choosing which statements run

The source states the purpose in one sentence: PL/SQL allows the use of an IF statement to control the execution of a block of code. The IF-THEN-ELSIF-ELSE-END IF construct allows specifying certain conditions under which a specific block of code should be executed. This chapter covers the four IF forms and the CASE statement.

24.1 IF-THEN

SYNTAX

```
IF condition
THEN
   statement;
END IF;
```

Use it when: statements should execute only when the condition is true, and there is nothing to do otherwise.

24.2 IF-THEN-ELSE

SYNTAX

```
IF condition
THEN
   [statements to execute when condition is TRUE]
ELSE
   [statements to execute when condition is FALSE]
END IF;
```

Use it when: one set of statements should execute when the condition is true, and a different set when it is false.

24.3 IF-THEN-ELSIF

SYNTAX

```
IF condition1
THEN
   statements to execute when condition1 is TRUE
ELSIF condition2
THEN
   statements to execute when condition2 is TRUE
END IF;
```

Use it when: one set of statements should execute when condition1 is true, or a different set when condition2 is true.

24.4 IF-THEN-ELSIF-ELSE

SYNTAX

```
IF condition1
THEN
   statements to execute when condition1 is TRUE
ELSIF condition2
THEN
   statements to execute when condition2 is TRUE
ELSE
   statements to execute when both condition1 and condition2 are FALSE
END IF;
```

Use it when: there are three or more outcomes — one for condition1, one for condition2, and one for neither.

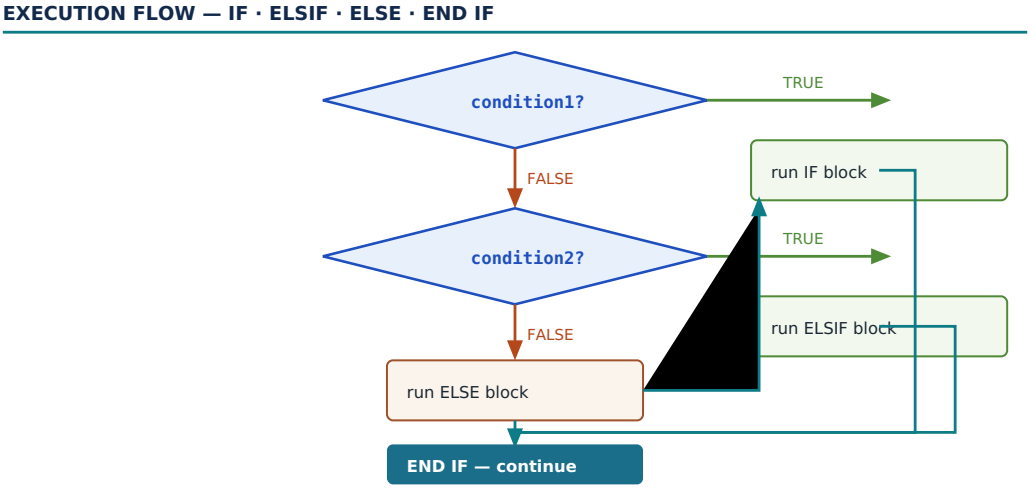


Figure 24.1 — The IF ladder. Conditions are tested top to bottom; the first true one wins, and exactly one branch runs. If none is true, the ELSE branch runs.

COMMON MISTAKE

Writing `ELSE IF` (two words) instead of `ELSIF` (one word). `ELSE IF` opens a nested IF statement that then needs its own `END IF;`, and the block will not compile without it. In PL/SQL the multi-way branch keyword is `ELSIF`.

24.5 The CASE Statement

DEFINITION AND SYNTAX (SOURCE)

The PL/SQL CASE statement provides the facility to execute a sequence of statements based on a **selector**. A selector may be a variable, a function, or an expression.

```
CASE [expression]
   WHEN condition1 THEN result1
   WHEN condition2 THEN result2
   …
   WHEN condition_n THEN result_n
   ELSE result
END;
```

CASE and the IF ladder solve the same problem — choosing one branch from several — but they read differently. Use CASE when every branch tests the same value against a list of possibilities; use the IF ladder when the conditions are unrelated expressions.

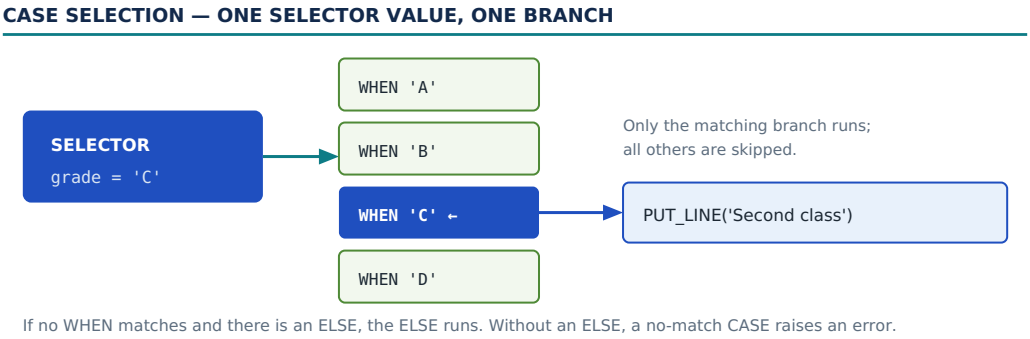


Figure 24.2 — A CASE statement tests the selector against each WHEN in turn and executes the first match.

24.6 Worked Conditional Programs

PROGRAM 1 — IF-THEN-ELSE (SOURCE EXAMPLE 1)

Problem: test whether a is less than 10 and print the appropriate message.

```
DECLARE
   a number(3) := 200;
BEGIN
   -- check the boolean condition using if statement
   IF( a < 10 ) THEN
      -- if condition is true then print the following
      dbms_output.put_line('a is less than 10 ' );
   ELSE
      dbms_output.put_line('a is not less than 10 ' );
   END IF;
   dbms_output.put_line('value of a is : ' || a);
END;
```

Dry run: a is initialised to 200. The test `a < 10` is false (200 is not less than 10), so the ELSE branch runs. Control then continues after `END IF` and the final `PUT_LINE` runs unconditionally.

Output (source):

```
a is not less than 10
value of a is : 200
PL/SQL procedure successfully completed.
```

PROGRAM 2 — EVEN OR ODD (SOURCE EXAMPLE 2)

Problem: check whether a number supplied by the user is even or odd.

```
DECLARE
   n1 NUMBER := &num1;
BEGIN
   -- test if the number provided by the user is even
   IF MOD(n1,2) = 0 THEN
      DBMS_OUTPUT.PUT_LINE ('The number. '||n1||
                            ' is even number');
   ELSE
      DBMS_OUTPUT.PUT_LINE ('The number '||n1||' is odd number.');
   END IF;
   DBMS_OUTPUT.PUT_LINE ('Done Successfully');
END;
/
```

Logic: `MOD(n1,2)` is the remainder when `n1` is divided by 2. It is 0 for an even number and 1 for an odd one, so a single test separates the two cases.

Dry run (num1 = 19): 19 mod 2 = 1, which is not 0, so the ELSE branch runs and prints “is odd number”. Then `Done Successfully` prints unconditionally.

Sample output (source):

```
Enter value for num1: 19
old 2: n1 NUMBER := &num1;
new 2: n1 NUMBER := 19;
The number 19 is odd number.
Done Successfully
```

PROGRAM 3 — POSITIVE, NEGATIVE OR ZERO (SOURCE EXAMPLE 3)

Problem: classify a number as positive, negative or zero.

```
DECLARE
   num1 NUMBER := &get_num;
BEGIN
   IF num1 < 0 THEN
      DBMS_OUTPUT.PUT_LINE ('The number '||num1||' is a negative number');
   ELSIF num1 = 0 THEN
      DBMS_OUTPUT.PUT_LINE ('The number '||num1||' is equal to zero');
   ELSE
      DBMS_OUTPUT.PUT_LINE ('The number '||num1||' is a positive number');
   END IF;
END;
/
```

Logic: three outcomes need the full IF-THEN-ELSIF-ELSE form. The order matters: testing `num1 < 0` first means the ELSE branch automatically covers all remaining cases, which are the positive numbers.

Sample outputs (source, three runs):

```
Enter value for get_num: 45
new 2: num1 NUMBER := 45;
The number 45 is a positive number
PL/SQL procedure successfully completed
```

```
Enter value for get_num: 0
new 2: num1 NUMBER := 0;
The number 0 is equal to zero
PL/SQL procedure successfully completed.
```

```
Enter value for get_num: -15
new 2: num1 NUMBER := -15;
The number -15 is a negative number
PL/SQL procedure successfully completed.
```

PROGRAM 4 — CASE WITH A GRADE (SOURCE EXAMPLE 1)

Problem: print the class obtained for a grade.

```
DECLARE
   grade char(1) := 'C';
BEGIN
   CASE grade
      when 'A' then dbms_output.put_line('Distinction');
      when 'B' then dbms_output.put_line('First class');
      when 'C' then dbms_output.put_line('Second class');
      when 'D' then dbms_output.put_line('Pass class');
      else dbms_output.put_line('Failed');
   END CASE;
END;
/
```

Dry run: the selector grade is 'C'. The CASE tests each WHEN in order: 'A' does not match, 'B' does not match, 'C' matches. The statement after `when 'C' then` runs, and the remaining branches — including the ELSE — are skipped.

Output (source):

```
Second class
```

PROGRAM 5 — CASE WITH A GRADE READ FROM THE PROMPT (SOURCE EXAMPLE 2)

```
DECLARE
   grd CHAR(1);
BEGIN
   -- Accept value for grade
   grd := '&new_grd';
   CASE grd
      WHEN 'A' THEN dbms_output.Put_line('Your Grade is: Outstanding');
      WHEN 'B' THEN dbms_output.Put_line('Your Grade is: Excellent');
      WHEN 'C' THEN dbms_output.Put_line('Your Grade is: Very Good');
      WHEN 'D' THEN dbms_output. Put_line('Your Grade is: Average');
      WHEN 'F' THEN dbms_output.Put_line('Your Grade is: Poor');
      ELSE dbms_output.Put_line('No such grade in the list.');
   END CASE;
END;
/
```

Sample output (source):

```
Enter value for new_grd: D
old 5: grd := '&new_grd';
new 5: grd := 'D';

Your Grade is: Average
PL/SQL procedure successfully completed.
```

PROGRAM 6 — WHICH DAY IS A DATE (SOURCE EXAMPLE 3)

Problem: given a date, print the name of the weekday.

```
SET serveroutput ON
DECLARE
   t_dt  DATE := To_date('&input_a_date', 'DD-MON-YYYY');
   t_day VARCHAR2(1);
BEGIN
   t_day := To_char(t_dt, 'D');
   CASE t_day
      WHEN '1' THEN dbms_output.Put_line ('The date you entered is Sunday.');
      WHEN '2' THEN dbms_output.Put_line ('The date you entered is Monday.');
      WHEN '3' THEN dbms_output.Put_line ('The date you entered is Tuesday.');
      WHEN '4' THEN dbms_output.Put_line ('The date you entered is Wednesday.');
      WHEN '5' THEN dbms_output.Put_line ('The date you entered is Thursday.');
      WHEN '6' THEN dbms_output.Put_line ('The date you entered is Friday.');
      WHEN '7' THEN dbms_output.Put_line ('The date you entered is Saturday.');
   END CASE;
END;
/
```

Logic: `TO_CHAR(t_dt,'D')` returns the day-of-week as a single character, '1' for Sunday through '7' for Saturday. The CASE then maps that character to a name. Note that this CASE has **no ELSE branch** — if the character were ever outside '1'...'7' the statement would raise an error, which is safe here because `TO_CHAR(…,'D')` cannot return anything else.

Sample outputs (source, two runs):

```
Enter value for input_a_date: 15-may-2018
new 2: t_dt DATE := To_date('15-may-2018', 'DD-MON-YYYY');
The date you entered is Tuesday.
PL/SQL procedure successfully completed.
```

```
Enter value for input_a_date: 14-sep-2017
new 2: t_dt DATE := To_date('14-sep-2017', 'DD-MON-YYYY');
The date you entered is Thursday.
PL/SQL procedure successfully completed.
```

EXPLANATION — WHY 15-MAY-2018 WAS A TUESDAY

The output is worth verifying rather than trusting, because a date question in an exam may ask you to do exactly this. 15 May 2018 fell on a Tuesday, and 14 September 2017 fell on a Thursday. Both outputs in the source are therefore consistent with the calendar, which is a good sign that the program logic is right.

Chapter summary

SUMMARY — CHAPTER 24

- Four IF forms: `IF-THEN` (true only), `IF-THEN-ELSE` (two outcomes), `IF-THEN-ELSIF` (multiple conditions, no else), `IF-THEN-ELSIF-ELSE` (multiple conditions plus a default).
- Conditions are tested top to bottom; the first true branch runs and the rest are skipped.
- **CASE** chooses one branch based on a selector, which may be a variable, function or expression. Without an ELSE, a CASE that matches nothing raises an error.
- Use the keyword `ELSIF`, never `ELSE IF`, for a multi-way branch.

25. Iterative Control: LOOP, WHILE and FOR

Part 5 • repeating statements — and knowing when to stop

The source introduces the topic precisely: LOOP statements let you execute a sequence of statements multiple times. There are three forms of LOOP statements: LOOP, WHILE-LOOP, and FOR-LOOP. The three forms differ in one crucial respect — **where the condition is tested**, and therefore whether the body can run zero times.

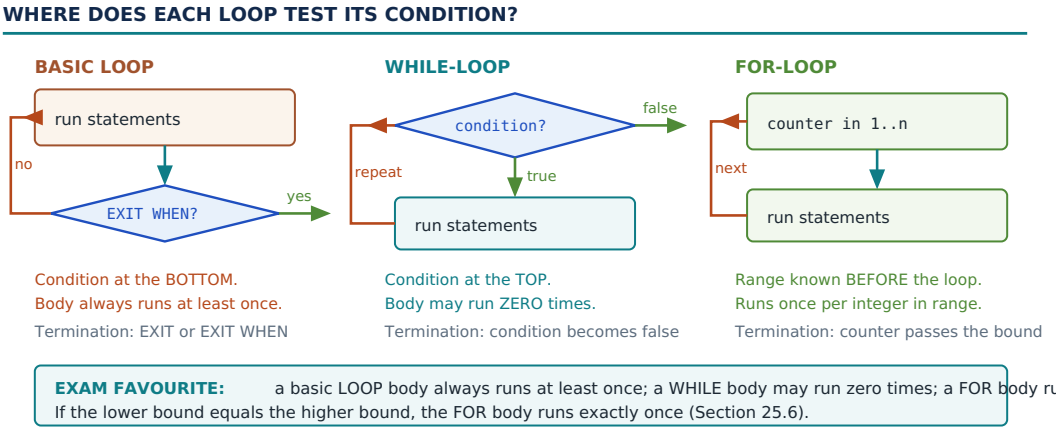


Figure 25.1 — The decisive difference between the three loop forms is where the condition is tested, and therefore whether the body can be skipped entirely.

25.1 The Basic LOOP

SYNTAX (SOURCE)

```
LOOP
   sequence_of_statements
END LOOP;
```

The simplest form of LOOP statement is the basic (or infinite) loop, which encloses a sequence of statements between the keywords `LOOP` and `END LOOP`. With each iteration the sequence is executed, then control resumes at the top of the loop.

Note the source's parenthetical: it calls this the “infinite” loop, and the label is fair — nothing in the syntax itself stops it. A basic LOOP terminates only when an `EXIT` statement executes inside it.

25.2 EXIT and EXIT-WHEN

EXIT (SOURCE)

If further processing is undesirable or impossible, you can use an `EXIT` statement to complete the loop. You can place one or more EXIT statements anywhere inside a loop, but nowhere outside a loop.

EXIT forces a loop to complete **unconditionally**. When an EXIT statement is encountered, the loop completes immediately and control passes to the next statement.

```
LOOP
   ...
   IF credit_rating < 3 THEN
      ...
      EXIT;   -- exit loop immediately
   END IF;
END LOOP;
-- control resumes here
```

EXIT-WHEN (SOURCE)

EXIT-WHEN lets a loop complete **conditionally**. When the EXIT statement is encountered, the condition in the WHEN clause is evaluated. If the condition is true, the loop completes and control passes to the next statement after the loop.

```
LOOP
   FETCH c1 INTO ...;
   EXIT WHEN c1%NOTFOUND;   -- exit loop if condition is true
   ...
END LOOP;
CLOSE c1;
```

The source adds an essential observation about termination: until the condition is true, the loop cannot complete. So, a statement inside the loop must change the value of the condition. In the FETCH example, if the FETCH returns a row the condition is false; when the FETCH fails to return a row the condition becomes true, the loop completes, and control passes to the CLOSE statement.

EXIT-WHEN REPLACES A SIMPLE IF — THE SOURCE'S OWN COMPARISON

These two fragments are logically equivalent, but the second is easier to read:

Using IF	Using EXIT-WHEN

`IF count > 100 THEN`
`  EXIT;`
`END IF;`
	`EXIT WHEN count > 100;`

This is a good example of a PL/SQL idiom worth adopting: when a loop's only purpose for an IF is to exit, use `EXIT WHEN` instead.

COMMON MISTAKE — EXIT OUTSIDE A LOOP

The source makes the point with a deliberately wrong example:

```
BEGIN
   ...
   IF credit_rating < 3 THEN
      ...
      EXIT;   -- NOT ALLOWED
   END IF;
END;
```

`EXIT` must be placed inside a loop. To complete a PL/SQL block before its normal end is reached, use the `RETURN` statement instead. The distinction is: `EXIT` leaves a **loop**; `RETURN` leaves a **subprogram or block**.

25.3 Loop Labels and Nested Loops

LABELS (SOURCE)

Like PL/SQL blocks, loops can be labelled. The label is an undeclared identifier enclosed by double angle brackets, and it must appear at the beginning of the LOOP statement:

```
<<label_name>>
LOOP
   sequence_of_statements
END LOOP;
```

Optionally, the label name can also appear at the end of the LOOP statement:

```
<<my_loop>>
LOOP
   ...
END LOOP my_loop;
```

When you nest labelled loops, use ending label names to improve readability. More importantly, labels give you a way to exit more than one loop at once:

```
<<outer>>
LOOP
   ...
   LOOP
      ...
      EXIT outer WHEN ...;   -- exit both loops
   END LOOP;
   ...
END LOOP outer;
```

EXPLANATION — WHAT EXIT OUTER DOES

The source states the effect precisely: every enclosing loop up to and including the labelled loop is exited. So `EXIT outer` from the inner loop does not merely leave the inner loop — it leaves the inner loop and the outer loop in one step, landing on the statement after `END LOOP outer;`. This is the clean way to break out of deeply nested processing without setting and testing flag variables.

25.4 WHILE-LOOP

SYNTAX (SOURCE)

```
WHILE condition LOOP
   sequence_of_statements
END LOOP;
```

Before each iteration of the loop the condition is evaluated. If the condition is true the sequence is executed, then control resumes at the top of the loop. If the condition is false or null, the loop is bypassed and control passes to the next statement.

The source gives a worked illustration and draws out the crucial consequence:

```
WHILE total <= 25000 LOOP
   ...
   SELECT sal INTO salary FROM emp WHERE ...;
   total := total + salary;
END LOOP;
```

EXPLANATION — THE NUMBER OF ITERATIONS IS UNKNOWN IN ADVANCE

The source's own words: the number of iterations depends on the condition and is unknown until the loop completes. The condition is tested at the top of the loop, so the sequence might execute zero times. In the last example, if the initial value of total is larger than 25000, the condition is false and the loop is bypassed.

This is the single most important fact about WHILE: **zero iterations is a legitimate outcome.**

25.5 Building a Loop-That-Runs-At-Least-Once

The source notes that some languages have a `LOOP UNTIL` or `REPEAT UNTIL` structure which tests the condition at the bottom, so the body always executes at least once. **PL/SQL has no such structure** — but you can build one, and the source shows both ways.

METHOD 1 — A BASIC LOOP WITH EXIT WHEN AT THE BOTTOM

```
LOOP
   sequence_of_statements
   EXIT WHEN boolean_expression;
END LOOP;
```

Because the EXIT WHEN is the last thing in the body, the body always runs at least once before the test is reached. This is the closest PL/SQL equivalent of `REPEAT UNTIL`.

METHOD 2 — A WHILE LOOP WITH AN INITIALISED BOOLEAN FLAG

```
done := FALSE;
WHILE NOT done LOOP
   sequence_of_statements
   done := boolean_expression;
END LOOP;
```

The flag starts FALSE, so `NOT done` is true on the first test and the body runs. At the end of the body the flag is recomputed. The source's warning is important: a statement inside the loop must assign a new value to the Boolean variable. Otherwise you have an infinite loop.

A FACT WORTH REMEMBERING

The source points out that these two loops are logically equivalent — a WHILE whose condition is always true is just a basic loop:

Always-true WHILE	Basic LOOP
`WHILE TRUE LOOP ... END LOOP;`	`LOOP ... END LOOP;`

25.6 FOR-LOOP and REVERSE

SYNTAX (SOURCE)

```
FOR counter IN [REVERSE] lower_bound..higher_bound LOOP
   sequence_of_statements
END LOOP;
```

Whereas the number of iterations through a WHILE loop is unknown until the loop completes, the number of iterations through a FOR loop is **known before the loop is entered**. FOR loops iterate over a specified range of integers. The range is part of an iteration scheme enclosed by the keywords FOR and LOOP, and a double dot (. .) serves as the range operator.

THE RULE THAT CATCHES PEOPLE OUT

The range is evaluated when the FOR loop is first entered and is never re-evaluated. This means the bounds are computed once. If a variable used as a bound changes inside the loop, the number of iterations does not change.

The three behaviours of a FOR loop

Table 25.1 — FOR loop behaviour in the three cases the source illustrates.

Case	Code	Values of the counter	Iterations
Normal, ascending	`FOR i IN 1..3 LOOP`	`1, 2, 3`	3
Lower = upper	`FOR i IN 3..3 LOOP`	`3`	1
REVERSE	`FOR i IN REVERSE 1..3 LOOP`	`3, 2, 1`	3

COMMON MISTAKE — WRITING REVERSE BOUNDS BACKWARDS

With `REVERSE`, iteration proceeds downward from the higher bound to the lower bound, and after each iteration the loop counter is decremented. But you still write the range bounds in **ascending** order, not descending. So `FOR i IN REVERSE 1..3` is correct and gives 3, 2, 1 — while `FOR i IN REVERSE 3..1` is **not** the way to write it. The source states this explicitly: "you write the range bounds in ascending (not descending) order."

COMMON MISTAKE — ASSIGNING THE FOR COUNTER

Inside a FOR loop the loop counter can be **referenced like a constant** but **cannot be assigned values**. The source's example shows both what is legal and what is not:

```
FOR ctr IN 1..10 LOOP
   IF NOT finished THEN
      INSERT INTO ... VALUES (ctr, ...);   -- legal: reading ctr
      factor := ctr * 2;                   -- legal: reading ctr
   ELSE
      ctr := 10;                           -- NOT ALLOWED: assigning ctr
   END IF;
END LOOP;
```

The reason is that the loop itself controls the counter. If your body could change it, the loop's iteration count would no longer match the range, and the guarantee that a FOR loop runs exactly once per integer in the range would be broken.

25.7 Worked Iterative Programs

PROGRAM 1 — REVERSE A NUMBER WITH WHILE (SOURCE EXAMPLE 1)

Problem: find the reverse of a given number using a WHILE loop.

```
DECLARE
   num Number(3) := 123;
   ans Number(3) := 0;
   i   Number(3) := 0;
BEGIN
   WHILE num != 0
   LOOP
      i   := mod(num,10);
      ans := (ans * 10) + i;
      num := floor(num/10);
   END LOOP;
   dbms_output.put_line('reverse of given number is: ' || ans);
END;
/
```

Logic: on each pass, `MOD(num,10)` peels off the last digit, the accumulating answer is shifted one place left (`ans*10`) and the new digit is added, and `FLOOR(num/10)` removes the last digit from num. The loop stops when num reaches 0.

Dry run (num = 123):

Iteration	num at test	i = MOD(num,10)	ans = ans*10 + i	num = FLOOR(num/10)
1	123	3	0*10 + 3 = 3	12
2	12	2	3*10 + 2 = 32	1
3	1	1	32*10 + 1 = 321	0
4	0	condition `num != 0` is false → loop ends		

Output (source):

```
SQL>set serveroutput on
SQL>start d://rev.sql
reverse of given number is: 321
PL/SQL successfully completed.
```

PROGRAM 2 — INSIDE AND OUTSIDE A LOOP WITH EXIT (SOURCE EXAMPLE 2)

Problem: print the value of a variable inside and outside a loop using a LOOP with EXIT.

```
DECLARE
   n NUMBER := 0;
BEGIN
   LOOP
      DBMS_OUTPUT.PUT_LINE ('The value of n inside the loop is: ' || TO_CHAR(n));
      n := n + 1;
      IF n > 5 THEN
         EXIT;
      END IF;
   END LOOP;
   DBMS_OUTPUT.PUT_LINE('The value of n outside the loop is: ' || TO_CHAR(n));
END;
/
```

Logic: this is a basic LOOP, so the body always runs at least once. n starts at 0, is printed, then incremented, and the loop exits as soon as n exceeds 5. The final print happens after the loop, so it shows the value that caused the exit.

Dry run:

Pass	Printed inside	n after increment	n > 5?
1	n = 0	1	no
2	n = 1	2	no
3	n = 2	3	no
4	n = 3	4	no
5	n = 4	5	no
6	n = 5	6	**yes → EXIT**

Sample output (source):

```
SQL> /
The value of n inside the loop is: 0
The value of n inside the loop is: 1
The value of n inside the loop is: 2
The value of n inside the loop is: 3
The value of n inside the loop is: 4
The value of n inside the loop is: 5
The value of n outside the loop is: 6
PL/SQL procedure successfully completed.
```

Note the pattern the source is illustrating: **n is printed as 0…5 inside but as 6 outside**, because the increment happens before the exit test. Reading this output carefully is a good check that you understand the order of statements in a loop body.

PROGRAM 3 — COMPUTING PI WITH A FOR LOOP (SOURCE EXAMPLE 1)

Problem: calculate pi with 500 terms of the Leibniz series using a simple FOR loop.

```
DECLARE
   p NUMBER := 0;
BEGIN
   FOR k IN 1..500 LOOP           -- calculate pi with 500 terms
      p := p + ( ( (-1) ** (k + 1) ) / ((2 * k) - 1) );
   END LOOP;
   p := 4 * p;
   DBMS_OUTPUT.PUT_LINE( 'pi is approximately : ' || p );   -- print result
END;
/
```

Logic: the series is 4 × (1 − 1/3 + 1/5 − 1/7 + …). The term `(-1)**(k+1)` supplies the alternating sign, and `(2*k - 1)` supplies the odd denominator. The loop runs exactly 500 times because the range is known before entry, and the final multiplication by 4 is done once, after the loop.

Output (source): the program prints `pi is approximately :` followed by the computed value. As with any finite truncation of this series, the value approaches π but is not exactly π — the source shows the approximation the program produced.

PROGRAM 4 — FIRST N NUMBERS WITH DIFFERENCE 3 (SOURCE EXAMPLE 2)

Problem: print the first n numbers starting from 1 with a common difference of 3.

```
DECLARE
   n number := &first_n_number;
   i number := 1;
   m number := 1;
BEGIN
   DBMS_OUTPUT.PUT_LINE ('The first '||n||' numbers are: ');
   DBMS_OUTPUT.PUT (i||' ');
   for i in 1..n-1 loop
      m := m + 3;
      dbms_output.put(m||' ');
   END LOOP;
   dbms_output.new_line;
END;
/
```

Logic: the first term (1) is printed before the loop. The loop then runs `n-1` times, each pass adding 3 to m and printing it, which produces the remaining `n-1` terms. Note that the loop variable i is not

used inside the body at all — the series is carried by m. This is a deliberate illustration that a FOR loop can be used purely as a counter.

Dry run (n = 10): 1 is printed first. The loop runs 9 times, adding 3 each time, printing 4, 7, 10, 13, 16, 19, 22, 25, 28. Total printed: ten numbers.

Sample output (source):

```
Enter value for first_n_number: 10
old 2: n number:= &first_n_number;
new 2: n number:= 10;
The first 10 numbers are:
1 4 7 10 13 16 19 22 25 28
PL/SQL procedure successfully completed.
```

FROM THE SOURCE — A SCOPE NOTE WORTH NOTICING

In Program 4 the source declares `i number := 1;` in the DECLARE section, and then also writes `for i in 1..n-1 loop`. The FOR loop's counter is its **own** implicitly declared variable, scoped to the loop; it shadows the outer i. The program works correctly because the body never relies on the outer i. It is nonetheless a name reuse worth avoiding in your own code — and it is a fair viva question about PL/SQL scoping rules.

Chapter summary

SUMMARY — CHAPTER 25

- Three loop forms: **basic LOOP**, **WHILE-LOOP** and **FOR-LOOP**.
- A basic LOOP has no built-in termination; it ends via `EXIT` (unconditional) or `EXIT WHEN` (conditional). Its body always runs at least once.
- `EXIT` must be inside a loop; use `RETURN` to leave a block or subprogram early.
- Loops can be labelled with `<<name>>`; `EXIT outer` exits all enclosing loops up to and including the labelled one.
- WHILE tests at the top, so its body may run **zero** times. The iteration count is unknown until the loop completes.
- PL/SQL has no LOOP UNTIL; build one with a basic LOOP plus `EXIT WHEN` at the bottom, or a WHILE on an initialised Boolean flag.
- FOR tests a range that is evaluated once, before entry. Lower = upper runs once; `REVERSE` counts down but is written with ascending bounds; the counter cannot be assigned.

26. Sequential Control: GOTO and NULL

Part 5 • the two statements you should rarely need

The source opens this topic with a candid assessment that is worth reading before the syntax: the GOTO statement is seldom needed. Occasionally, it can simplify logic enough to warrant its use. The NULL statement can improve readability by making the meaning and action of conditional statements clear. Overuse of GOTO statements can result in code that is hard to understand and maintain. Use GOTO statements sparingly.

That framing matters, because a student who only learns the syntax may conclude that GOTO is a normal tool. It is not: it is an escape hatch for unusual situations, and the source's own advice is to prefer an exception handler when branching out of deeply nested structure.

26.1 The GOTO Statement

DEFINITION (SOURCE)

The **GOTO** statement branches to a label **unconditionally**. The label must be unique within its scope and must precede an executable statement or a PL/SQL block. When executed, the GOTO statement transfers control to the labelled statement or block. The labelled statement or block can be **down or up** in the sequence of statements.

```
GOTO label;
..
..
<< label >>
statement;
```

The source's description in the second paragraph is the same rule in plainer words: a GOTO statement in PL/SQL provides an unconditional jump from the GOTO to a labelled statement in the same subprogram. The phrase “in the same subprogram” is important — it is the first of the restrictions in Section 26.3.

26.2 Worked GOTO Programs

PROGRAM 1 — PRIMALITY TEST WITH GOTO (SOURCE)

Problem: test whether n is prime, using GOTO to skip the “is prime” message when a divisor is found.

```
DECLARE
   p VARCHAR2(30);
   n PLS_INTEGER := 37;   -- test any integer > 2 for prime
BEGIN
   FOR j in 2..ROUND(SQRT(n)) LOOP
      IF n MOD j = 0 THEN          -- test for prime
         p := ' is not a prime number';   -- not a prime number
         GOTO print_now;
      END IF;
   END LOOP;
   p := ' is a prime number';

   <<print_now>>
   DBMS_OUTPUT.PUT_LINE(TO_CHAR(n) || p);
END;
/
```

Logic: the loop tests every integer from 2 up to √n. If any of them divides n exactly, the message is set to "not prime" and control jumps straight to the label, skipping the loop's remaining iterations and the "is prime" assignment. If the loop finishes without finding a divisor, execution falls through to `p := ' is a prime number'` and then reaches the label naturally.

Dry run (n = 37): √37 ≈ 6.08, so `ROUND(SQRT(37))` = 6 and the loop tests j = 2, 3, 4, 5, 6. None divides 37 exactly (37 mod 2 = 1, mod 3 = 1, mod 4 = 1, mod 5 = 2, mod 6 = 1), so the IF never fires, the GOTO never executes, and p becomes " is a prime number".

Output: `37 is a prime number`. (The source shows the program; the output follows directly from the trace above, since 37 is prime.)

WHY THIS USE OF GOTO IS DEFENSIBLE

This is the kind of case the source means by "occasionally it can simplify logic". The alternative without GOTO would be a Boolean flag set in the loop and tested afterwards. The GOTO version is shorter, and the label sits immediately after the loop, so the jump distance is tiny and the control flow is easy to follow. Note also the detail that makes the program correct: the label must precede an **executable** statement, which is why it sits directly above `DBMS_OUTPUT.PUT_LINE` and not on a blank line.

PROGRAM 2 — SKIPPING A VALUE WITH GOTO (SOURCE)

Problem: print the value of a from 10 upwards, skipping the value 15.

```
DECLARE
   a number(2) := 10;
BEGIN
   <<loopstart>>
   -- while loop execution
   WHILE a < 20 LOOP
      dbms_output.put_line ('value of a: ' || a);
      a := a + 1;
      IF a = 15 THEN
         a := a + 1;
         GOTO loopstart;
      END IF;
   END LOOP;
END;
/
```

Logic: the label sits immediately before the WHILE. Each pass prints a, increments it, and if the new value is 15 it is incremented again (to 16) and control jumps back to the label. Because the label is above the WHILE, the jump re-enters the loop — and the loop's condition is tested again.

Dry run:

Printed	a after increment	a = 15?	Action
10	11	no	continue
11	12	no	continue
12	13	no	continue
13	14	no	continue
14	15	**yes**	a becomes 16, GOTO loopstart
16	17	no	continue
17	18	no	continue
18	19	no	continue
19	20	no	loop condition 20 < 20 false → end

Notice that **15 is never printed** — that is the point of the program.

Output (source):

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 16
value of a: 17
value of a: 18
value of a: 19

PL/SQL procedure successfully completed.
```

FROM THE SOURCE — A NOTE ON THIS EXAMPLE

This example is a clear illustration of how GOTO works, but it is also a clear illustration of why the source advises against it. The same effect is achieved more readably without any jump at all — for example by incrementing a and testing `IF a <> 15 THEN PUT_LINE(...)`. The source presents it as a demonstration of the mechanism, and the surrounding text explicitly warns that overuse makes code hard to understand. Read it as a syntax lesson, not as a recommended style.

26.3 Restrictions on GOTO

The source lists the restrictions explicitly. They are a favourite examination question, and they all follow from one principle: **GOTO may not jump into the middle of a structured construct, nor out of a subprogram.**

RESTRICTIONS (SOURCE)

- A GOTO statement cannot branch **into** an IF statement, CASE statement, LOOP statement or sub-block.
- A GOTO statement cannot branch from one IF statement clause to another, or from one CASE statement WHEN clause to another.
- A GOTO statement cannot branch from an outer block into a sub-block (that is, an inner BEGIN-END block).
- A GOTO statement cannot branch out of a subprogram. To end a subprogram early, either use the RETURN statement or have GOTO branch to a place right before the end of the subprogram.
- A GOTO statement cannot branch from an exception handler back into the current BEGIN-END block. However, a GOTO statement **can** branch from an exception handler into an enclosing block.

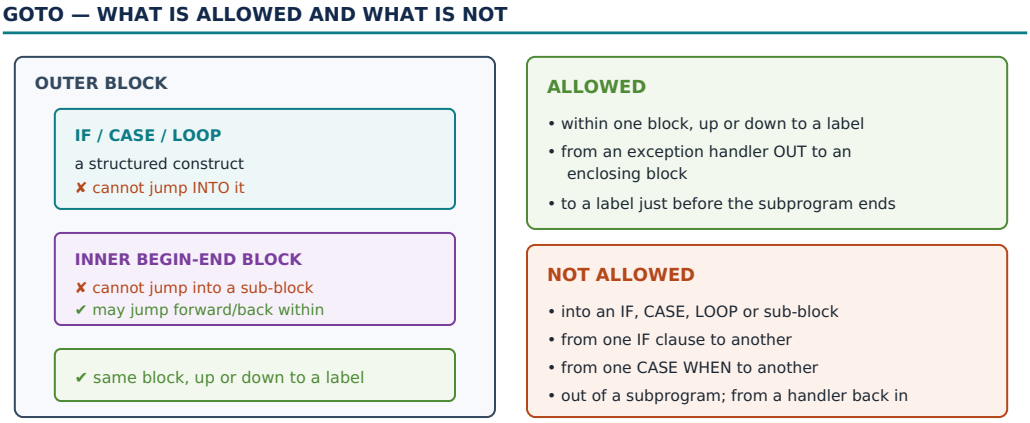


Figure 26.1 — GOTO may jump within a block, but never into a structured construct and never out of a subprogram. The exception-handler rule is asymmetric: outward is allowed, inward is not.

COMMON MISTAKE — THE ASYMMETRIC EXCEPTION-HANDLER RULE

The last restriction is the one most often misremembered, because it is not symmetric. A GOTO **cannot** branch from an exception handler back into the current BEGIN-END block — but it **can** branch from an exception handler into an enclosing block. The logic is that jumping back in would re-enter a block whose execution had already failed, which is precisely the situation the exception handler exists to deal with.

26.4 The NULL Statement

DEFINITION (SOURCE)

The **NULL** statement does nothing and passes control to the next statement. Some languages refer to such an instruction as a **no-op** (no operation).

The source's framing of its purpose is what makes it worth knowing: the NULL statement can improve readability by making the meaning and action of conditional statements clear. Its value is not that it does anything, but that it **says** that nothing should be done — which is different from forgetting to write anything.

PROGRAM 3 — NULL TO SHOW NO ACTION (SOURCE)

Problem: give a commission increase only to sales representatives, and explicitly do nothing for everyone else.

```
DECLARE
   v_job_id VARCHAR2(10);
   v_emp_id NUMBER(6) := 110;
BEGIN
   SELECT job_id INTO v_job_id
   FROM   employees
   WHERE  employee_id = v_emp_id;

   IF v_job_id = 'SA_REP' THEN
      UPDATE employees
      SET    commission_pct = commission_pct * 1.2;
   ELSE
      NULL;   -- do nothing if not a sales representative
   END IF;
END;
/
```

Logic: the SELECT reads the employee's job into a variable. If the job is `SA_REP`, the commission is increased by 20 per cent; otherwise the ELSE branch runs `NULL`, which does nothing at all and passes control to the next statement.

WHY WRITE NULL AT ALL?

Because the ELSE branch is **documented intent**. A reader of this block can see immediately that the case was considered and deliberately left alone, rather than overlooked. An empty ELSE (or no ELSE at all) leaves the reader wondering. This is exactly the readability benefit the source names.

PROGRAM 4 — GOTO AND NULL TOGETHER (SOURCE)

```
DECLARE
   v_count NUMBER;
BEGIN
   SELECT count(JOB_ID) INTO v_count
   FROM   i_jobs
   WHERE  job_name = 'DEVELOPER';

   IF v_count = 0 THEN
      GOTO insert_row;
   ELSE
      GOTO null_row;
   END IF;

   <<insert_row>>
   INSERT INTO i_jobs VALUES (1,'DEVELOPER',10);

   <<null_row>>
   DBMS_OUTPUT.PUT_LINE('NULL statement example');
   NULL;
END;
```

Logic: the SELECT counts how many rows in i_jobs have the job name 'DEVELOPER'. If the count is zero, control jumps to the `insert_row` label and inserts the row; then execution continues into the

next label's statements. If the count is not zero, control jumps to `null_row`, skipping the INSERT entirely, and prints the message.

> **NOTE THE FALL-THROUGH BEHAVIOUR**
>
> This example shows an important property of labels: they do not stop execution. After the INSERT at `insert_row`, control **falls through** into the `null_row` statements, so the message is printed in both paths. If the intention were to print only when nothing was inserted, the `insert_row` branch would need its own GOTO to skip the message. This is precisely the kind of subtlety the source's warning about GOTO refers to — and it is a good exam question about labels and control flow.

Chapter summary

> **SUMMARY — CHAPTER 26**
>
> - **GOTO** branches unconditionally to a label; the label must be unique in scope and precede an executable statement or block.
> - The jump may be up or down within the same block. Use GOTO sparingly — the source recommends an exception handler instead of branching out of deeply nested structure.
> - **Restrictions:** no jumping into an IF, CASE, LOOP or sub-block; no jumping between IF clauses or CASE WHEN clauses; no jumping from an outer block into a sub-block; no jumping out of a subprogram; and from an exception handler you may branch out to an enclosing block but not back into the current block.
> - **NULL** is a no-op. Its value is readability: it documents that a branch is deliberately empty.
> - Labels do not stop execution — control falls through from one label into the next.

PART 6

PL/SQL Cursors

What a cursor is · the context area and the active set · implicit cursors and their four attributes · explicit cursors and the declare–open–fetch–close life cycle · cursor loops and termination

Source: UNIT 2, Chapter 2.3.4 (Introduction to PL/SQL Cursor) — the CUSTOMERS table, the SQL%ROWCOUNT program and the complete explicit-cursor program are the source's own.

27. Cursors: Concept and Implicit Cursors

Part 6 • working with a result set one row at a time

A cursor exists to solve a mismatch. SQL is set-oriented: a query returns a whole set of rows at once. A procedural program is row-oriented: it does one thing at a time. Something must bridge the two, and in PL/SQL that something is the cursor.

27.1 What a Cursor Is

DEFINITION (SOURCE)

A **PL/SQL cursor is a pointer that points to the result set of an SQL query against database tables.**

The source explains why it is needed: when you work with an Oracle database, you work with a complete set of rows returned from a SELECT statement. However, the application in some cases cannot work effectively with the entire result set. Therefore the database server needs to provide a mechanism for the application to work with one row, or a subset of the result set, at a time. Oracle created the PL/SQL cursor to provide these extensions.

The single word to hold on to is **pointer**. A cursor is not the data; it is a position within the data. Each fetch moves the pointer forward one row, and the program acts on whatever row the pointer currently indicates.

27.2 The Context Area and the Active Set

THE SOURCE'S EXPLANATION

Oracle has dedicated memory locations for executing SQL statements, and holds that processed information — for example, the total number of rows updated. A cursor in PL/SQL gives a name and acts as a pointer to the area of work called a **context area**, and then uses its information. It keeps the number of rows processed by the SQL statement. These rows are called the **active set**. The size of the active set is equal to the count of the rows that meet the condition.

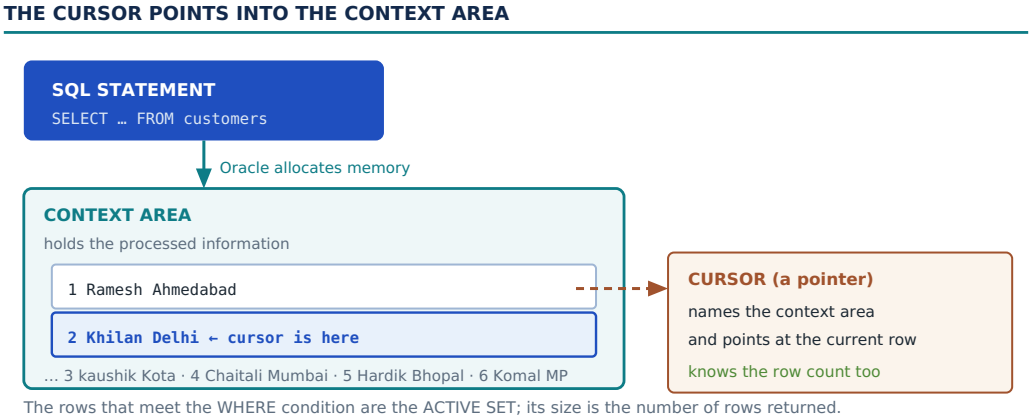


Figure 27.1 — The DBMS allocates a context area for a SQL statement; the cursor is a named pointer into it, and the rows it can reach are the active set.

Table 27.1 — The four terms, defined.

Term	Meaning
Cursor	A pointer to the result set of a SQL query against database tables
Context area	The work area of memory Oracle allocates for executing a SQL statement
Active set	The rows held in the context area — those that meet the condition
Size of the active set	Equal to the count of rows that meet the condition

27.3 Implicit Cursors

DEFINITION (SOURCE)

Implicit cursors are automatically created by Oracle whenever an SQL statement is executed, when there is no explicit cursor for the statement. **Programmers cannot control the implicit cursors** and the information in them.

Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with that statement. For INSERT operations, the cursor holds the data that needs to be inserted. For UPDATE and DELETE operations, the cursor identifies the rows that would be affected.

Three consequences follow, and they are worth stating separately because exam questions often probe them:

1. **You do not declare an implicit cursor.** Oracle creates it for you whenever you run a DML statement or a `SELECT … INTO`.
2. **You do not open or close it.** Oracle manages its whole life cycle, and closes it automatically after executing the associated statement.
3. **You can read its attributes.** This is the part that makes implicit cursors useful. The attributes tell you what the statement just did.

27.4 Implicit Cursor Attributes

THE SQL CURSOR (SOURCE)

In PL/SQL you can refer to the most recent implicit cursor as the **SQL cursor**, which always has attributes such as `%FOUND`, `%ISOPEN`, `%NOTFOUND` and `%ROWCOUNT`. The SQL cursor has additional attributes, `%BULK_ROWCOUNT` and `%BULK_EXCEPTIONS`, designed for use with the FORALL statement.

Any SQL cursor attribute is accessed as `sql%attribute_name`.

Table 27.2 — The four most-used implicit cursor attributes, in the source's words.

Attribute	Returns TRUE / value when…
`SQL%FOUND`	Returns TRUE if an INSERT, UPDATE or DELETE statement affected one or more rows, or a SELECT INTO statement returned one or more rows. Otherwise it returns FALSE.
`SQL%NOTFOUND`	The logical opposite of `%FOUND`. Returns TRUE if an INSERT, UPDATE or DELETE statement affected no rows, or a SELECT INTO returned no rows. Otherwise it returns FALSE.
`SQL%ISOPEN`	**Always returns FALSE** for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
`SQL%ROWCOUNT`	Returns the number of rows affected by an INSERT, UPDATE or DELETE statement, or returned by a SELECT INTO statement.

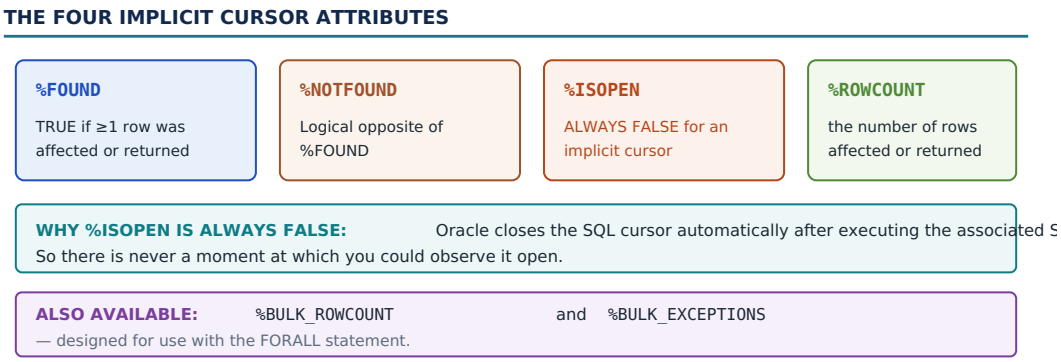


Figure 27.2 — The four implicit cursor attributes. The odd one out is `%ISOPEN`, which is defined to be always FALSE because the implicit cursor is never open when you can look at it.

27.5 Worked Implicit Cursor Program

PROGRAM — USING SQL%ROWCOUNT (SOURCE)

Problem: increase every customer's salary by 500, and report how many customers were affected.

The source uses this CUSTOMERS table throughout the chapter:

Table 27.3 — The CUSTOMERS table used in the source's cursor examples.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	kaushik	23	Kota	2000
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500
6	Komal	22	MP	4500

```
DECLARE
   total_rows number(2);
BEGIN
   UPDATE customers
   SET    salary = salary + 500;

   IF sql%notfound THEN
      dbms_output.put_line('no customers selected');
   ELSIF sql%found THEN
      total_rows := sql%rowcount;
      dbms_output.put_line( total_rows || ' customers selected ');
   END IF;
END;
/
```

Logic: the UPDATE has no WHERE clause, so it affects every row. Oracle associates an implicit cursor with it. The block then tests `sql%notfound`: if no rows were affected, it prints a “none” message. Otherwise `sql%found` is true, so it copies `sql%rowcount` — the number of rows the UPDATE affected — into `total_rows` and prints it.

DRY RUN

Step	Action	State
1	`total_rows` declared as `number(2)`	`total_rows = NULL`
2	`UPDATE customers SET salary = salary + 500`	6 rows affected; `sql%rowcount` = 6
3	Test `sql%notfound`	FALSE (rows were affected) → skip
4	Test `sql%found`	TRUE → take this branch
5	`total_rows := sql%rowcount`	`total_rows = 6`
6	Print	`6 customers selected`

Output (source):

```
6 customers selected
PL/SQL procedure successfully completed.
```

The source adds: if you check the records in the customers table, you will find that the rows have been updated. Every salary is now 500 higher than in Table 27.3.

EXPLANATION — WHAT THE ATTRIBUTES PROVE HERE

Notice what the program demonstrates. It never names the implicit cursor, never opens it and never closes it — yet it still learns a fact about what just happened. That is the whole point of implicit cursor attributes: they are the DBMS telling you the outcome of the statement it just executed for you.

Chapter summary

SUMMARY — CHAPTER 27

- A **cursor** is a pointer to the result set of a SQL query against database tables, created so that a program can work with one row (or a subset) at a time instead of the whole result set.
- The **context area** is the memory Oracle allocates for a SQL statement; the rows in it are the **active set**, whose size equals the number of rows meeting the condition.
- **Implicit cursors** are created automatically for DML statements and `SELECT … INTO`. Programmers cannot control them.
- The implicit cursor is referred to as the **SQL cursor**, with attributes `%FOUND`, `%NOTFOUND`, `%ISOPEN` (always FALSE) and `%ROWCOUNT`, accessed as `sql%attribute_name`.
- `%BULK_ROWCOUNT` and `%BULK_EXCEPTIONS` exist for use with FORALL.

28. Explicit Cursors and Cursor Loops

Part 6 • declare, open, fetch, close — and terminate correctly

An implicit cursor is convenient but opaque: you can read its attributes, but you cannot control it. When a program needs real control over a multi-row query — reading rows one at a time, deciding when to stop — it needs an explicit cursor.

28.1 Declaring, Opening, Fetching and Closing

DEFINITION (SOURCE)

Explicit cursors are programmer-defined cursors for gaining more control over the context area. An explicit cursor should be defined in the **declaration section** of the PL/SQL block. It is created on a SELECT statement which returns more than one row.

```
CURSOR cursor_name IS select_statement;
```

The source lists the four steps of working with an explicit cursor, and the order matters:

1. **Declaring** the cursor, for initialising the memory.
2. **Opening** the cursor, for allocating the memory.
3. **Fetching** the cursor, for retrieving the data.
4. **Closing** the cursor, to release the allocated memory.

Table 28.1 — The four steps, with the source's syntax and explanation for each.

Step	Syntax (source)	What it does (source)
Declare	`CURSOR c_customers IS SELECT id, name, address FROM customers;`	Defines the cursor with a name and the associated SELECT statement.
Open	`OPEN c_customers;`	Allocates the memory for the cursor and makes it ready for fetching the rows returned by the SQL statement into it.
Fetch	`FETCH c_customers INTO c_id, c_name, c_addr;`	Accesses one row at a time.
Close	`CLOSE c_customers;`	Releases the allocated memory.

28.2 The Cursor Life Cycle

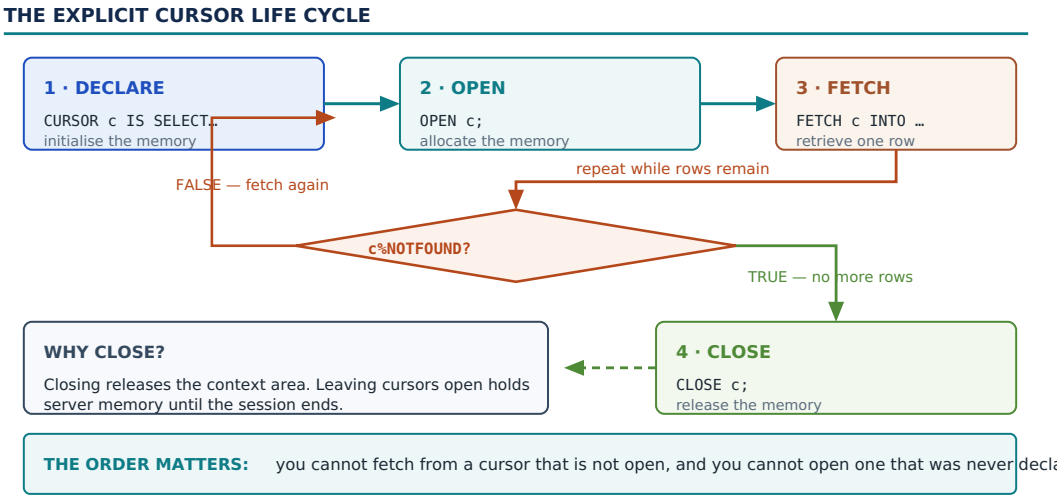


Figure 28.1 — The explicit cursor life cycle. The fetch step repeats inside a loop; the loop is terminated by testing an attribute such as `%NOTFOUND`, and the cursor is then closed to release its memory.

28.3 Worked Explicit Cursor Program

PROGRAM — READING THE CUSTOMERS TABLE (SOURCE)

Problem: print the id, name and address of every customer.

```
DECLARE
   c_id    customers.id%type;
   c_name  customer.name%type;
   c_addr  customers.address%type;

   CURSOR c_customers is
      SELECT id, name, address FROM customers;
BEGIN
   OPEN c_customers;
   LOOP
      FETCH c_customers into c_id, c_name, c_addr;
      EXIT WHEN c_customers%notfound;
      dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
   END LOOP;
   CLOSE c_customers;
END;
/
```

Structure: the declaration section holds three variables, each typed with `%type` so that its data type matches the corresponding column automatically, plus the cursor itself. The body opens the cursor, then loops: fetch a row into the three variables, test the cursor's `%notfound` attribute to decide whether to stop, and if there is a row, print it. After the loop the cursor is closed.

EXPLANATION — WHY EXIT WHEN COMES STRAIGHT AFTER FETCH

The order inside the loop is not arbitrary. `FETCH` must run before `EXIT WHEN c_customers%notfound` because the attribute describes the most recent fetch. Placing the EXIT immediately after the FETCH means the loop stops the moment a fetch finds no row, and never prints an uninitialised row. If the print came first, the final iteration would print stale values from the previous row.

DRY RUN — TRACING THE SIX ROWS

Iteration	FETCH retrieves	%NOTFOUND	Printed
1	1, Ramesh, Ahmedabad	FALSE	1 Ramesh Ahmedabad
2	2, Khilan, Delhi	FALSE	2 Khilan Delhi
3	3, kaushik, Kota	FALSE	3 kaushik Kota
4	4, Chaitali, Mumbai	FALSE	4 Chaitali Mumbai
5	5, Hardik, Bhopal	FALSE	5 Hardik Bhopal
6	6, Komal, MP	FALSE	6 Komal MP
7	no row	**TRUE**	— loop exits

Six rows printed, and the seventh fetch — the one that finds nothing — terminates the loop without printing anything. This is the standard shape of every explicit cursor loop.

Output (source):

```
1 Ramesh Ahmedabad
2 Khilan Delhi
3 kaushik Kota
4 Chaitali Mumbai
5 Hardik Bhopal
6 Komal MP
PL/SQL procedure successfully completed.
```

FROM THE SOURCE — TWO SMALL DEFECTS IN THE PRINTED PROGRAM

The source's program contains two slips that do not affect the logic but would stop the block compiling if typed literally. These notes reproduce the source's code as printed, and flag them here so you are not caught out:

- `c_name customer.name%type;` — the table name is missing its plural: elsewhere the table is `customers`. It should be `customers.name%type`.
- The first two variables are declared as `c_id` and `c_name` but the FETCH statement elsewhere in the source's chapter writes `c_id, c_name, c_addr;` the declaration in the earlier section of the source uses `c_id customers.id%type;`. Both spellings refer to the same variables; the mismatch is a transcription artefact.

The **output** the source shows is correct for the logic, and matches the dry run above. If you type the program yourself, use `customers.name%type`.

28.4 Implicit versus Explicit Cursors

Table 28.2 — Implicit and explicit cursors compared.

Aspect	Implicit cursor	Explicit cursor
Who creates it	Oracle, automatically, for DML and `SELECT … INTO`	The programmer, in the declaration section
Declared?	No	Yes — `CURSOR name IS select_statement;`
Opened / closed by you?	No — Oracle manages it and closes it automatically	Yes — `OPEN` and `CLOSE` are your responsibility
Can you control it?	No	Yes — that is its purpose
Returns	One row for `SELECT … INTO`, or the affected rows for DML	A set of rows, fetched one at a time
Attributes	`SQL%FOUND`, `SQL%NOTFOUND`, `SQL%ISOPEN` (always FALSE), `SQL%ROWCOUNT`	`c%FOUND`, `c%NOTFOUND`, `c%ISOPEN`, `c%ROWCOUNT` — using the cursor's own name
Typical use	A single-row query or a DML statement whose row count you want to know	A multi-row query processed row by row

EXPLANATION — THE ONE ATTRIBUTE THAT DIFFERS IN MEANING

For an implicit cursor `%ISOPEN` is **always FALSE** because Oracle has already closed it. For an explicit cursor `%ISOPEN` is genuinely useful: it is TRUE between `OPEN` and `CLOSE`. This is a favourite viva question precisely because the same attribute name behaves differently in the two cases.

28.5 Cursor Errors and Exam Traps

COMMON MISTAKE 1 — FORGETTING TO OPEN THE CURSOR

Fetching from a cursor that has not been opened raises `ORA-01001: invalid cursor`. The `OPEN` statement is what allocates the context area; without it there is nothing to fetch from.

COMMON MISTAKE 2 — TESTING %NOTFOUND BEFORE THE FIRST FETCH

Immediately after `OPEN`, the cursor has not fetched anything, so `%FOUND` and `%NOTFOUND` do not yet reflect the data. The reliable pattern — the one the source uses — is `FETCH` first, then `EXIT WHEN c%NOTFOUND`. A loop that tests `%NOTFOUND` before its first fetch may process one row too many or too few.

COMMON MISTAKE 3 — FORGETTING TO CLOSE

An unclosed cursor keeps its context area allocated until the session ends. In a program that opens many cursors, that is a memory leak. `CLOSE c_customers;` releases the memory, and the source places it after the loop for exactly this reason.

COMMON MISTAKE 4 — A CURSOR FOR A QUERY THAT RETURNS ONE ROW

The source specifies that an explicit cursor “is created on a SELECT Statement which returns more than one row”. If a query is guaranteed to return a single row, `SELECT … INTO` with an implicit cursor is the simpler and more appropriate tool. Using a cursor loop for a single row works but adds ceremony for no benefit.

THE PATTERN TO MEMORISE

```
CURSOR c IS SELECT … ;      -- declare (in DECLARE section)
…
OPEN  c;                    -- allocate
LOOP
   FETCH c INTO …;          -- get the next row
   EXIT WHEN c%NOTFOUND;    -- stop when there is none
   … process the row …      -- only reached for a real row
END LOOP;
CLOSE c;                    -- release
```

Every explicit cursor program in this chapter, and in the source, is a variation on these six lines.

Chapter summary

SUMMARY — CHAPTER 28

- **Explicit cursors** are programmer-defined and declared in the declaration section; they are created on a SELECT that returns more than one row.
- The four steps are **declare** (initialise memory), **open** (allocate memory), **fetch** (retrieve one row at a time) and **close** (release memory).
- A cursor loop fetches, tests `%NOTFOUND` to decide whether to stop, and processes the row. The `EXIT WHEN` must come immediately after the `FETCH`.
- Explicit cursors are closed by you; implicit cursors are closed by Oracle, which is why `SQL%ISOPEN` is always FALSE.
- Common errors: fetching without opening, testing `%NOTFOUND` before the first fetch, forgetting to close, and using a cursor where a single-row `SELECT … INTO` would do.

PART 7

SQL Views

Views and base tables · creating views from one and from many tables · querying, dropping and replacing · updating, inserting and deleting through a view · WITH CHECK OPTION · the six uses of a view

Source: UNIT 2, Chapter 2.3.5 (SQL Views) — the StudentDetails and StudentMarks tables and every example query are the source's own.

29. SQL Views: Concept and Creation

Part 7 • a stored query that behaves like a table

A view is the database's answer to a common problem: many different users need to see different slices of the same data, but you do not want to maintain a separate copy for each. A view stores the question, not the answer — and every time it is used, the question is asked again against the current data.

29.1 Definition and Base Tables

DEFINITION (SOURCE)

An **SQL View** is a specific representation of data from one or more tables. The tables referred to in the view are known as **base tables**. Creating a view does not take any storage space, because only the **query** is stored in the data dictionary and the actual data is not stored anywhere.

The maximum number of columns that can be defined in a SQL View is **1000**, as in tables.

The source also gives the informal framing that is worth holding alongside the formal one: views in SQL are kind of virtual tables. A view also has rows and columns as they are in a real table in the database. We can create a view by selecting fields from one or more tables present in the database. A view can either have all the rows of a table or specific rows based on a certain condition.

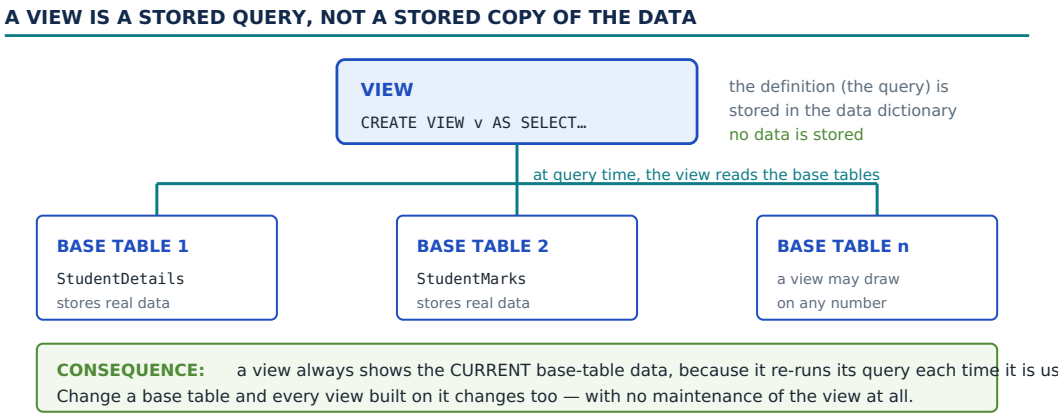


Figure 29.1 — A view stores only its defining query. At query time the DBMS runs that query against the base tables, so the view is always up to date.

EXPLANATION — WHY “NO STORAGE SPACE” IS A REAL BENEFIT

Because only the query is stored, a view cannot become inconsistent with its base tables. There is no second copy to fall out of date — the source's phrase is that the actual data “is not stored anywhere”. This is the opposite of the redundancy problem of Part 3: a view is a form of reuse that adds no redundancy at all.

29.2 Benefits of Views

BENEFITS (SOURCE)

The reasons for using views in applications can be many, like:

- **Reducing complexity.**
- **Improving security.**
- **Renaming the table columns.**

Chapter 30 expands these three into the source's fuller list of six uses (Section 30.7). The three here are the headline benefits; the six there are the detailed justifications.

29.3 The Sample Tables Used Throughout

The source uses two tables for every view example in the chapter. Learn them once and every query below is readable. The source names them **StudentDetails** and **StudentMarks**; the rows are reconstructed from the outputs the source reports, since the source shows the tables only as captioned images.

Table 29.1 — StudentDetails (reconstructed from the source's reported outputs).

S_ID	NAME	ADDRESS	AGE
1	Ramesh	Ahmedabad	32
2	Khilan	Delhi	25
3	kaushik	Kota	23
4	Chaitali	Mumbai	25
5	Hardik	Bhopal	27
6	Komal	MP	22

Table 29.2 — StudentMarks (reconstructed from the source's reported outputs).

NAME	MARKS	AGE
Ramesh	90	32
Khilan	85	25
kaushik	70	23
Chaitali	80	25
Hardik	75	27
Komal	95	22

FROM THE SOURCE — RECONSTRUCTED TABLES

The source shows these two tables as images with the caption “Sample Tables:” and no textual rows. The rows above are reconstructed from the outputs the source reports for its own queries — in particular, the `DetailsView` output (rows where `S_ID < 5`) and the `MarksView` output (which joins on NAME). Where the source's output confirms a value, the value above matches it. Treat these rows as **reconstructed for illustration**: if your class slides show different sample data, use those values — the SQL and the logic are unaffected.

29.4 Creating a View from a Single Table

SYNTAX (SOURCE)

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM   table_name
WHERE  condition;
```

Part	Meaning (source)
`view_name`	Name for the view
`table_name`	Name of the table
`condition`	Condition to select rows

EXAMPLE 1 — DETAILSVIEW (SOURCE)

Problem: create a view named `DetailsView` from the table `StudentDetails`, showing only the name and address of students whose id is less than 5.

```
CREATE VIEW DetailsView AS
SELECT NAME, ADDRESS
FROM   StudentDetails
WHERE  S_ID < 5;
```

Line by line: `CREATE VIEW DetailsView AS` names the view and begins its definition; `SELECT NAME, ADDRESS` chooses two columns, so the view has exactly two columns; `FROM StudentDetails` names the single base table; `WHERE S_ID < 5` restricts the view to rows 1 to 4.

Resulting view:

Table 29.3 — The view DetailsView after creation.

NAME	ADDRESS
Ramesh	Ahmedabad
Khilan	Delhi
kaushik	Kota
Chaitali	Mumbai

Note what the view does not contain: the S_ID column, the AGE column, and the rows for Hardik and Komal. A view can hide both columns and rows, which is the basis of its security benefit.

EXAMPLE 2 — STUDENTNAMES (SOURCE)

Problem: create a view named `StudentNames` showing student id and name, ordered by name.

```
CREATE VIEW StudentNames AS
SELECT S_ID, NAME
FROM   StudentDetails
ORDER BY NAME;
```

Resulting view (ordered by NAME):

Table 29.4 — The view StudentNames.

S_ID	NAME
4	Chaitali
5	Hardik
3	kaushik
2	Khilan
6	Komal
1	Ramesh

IMPORTANT — ORDER BY MAKES THIS VIEW NON-UPDATABLE

This view contains an `ORDER BY` clause. Section 30.3 lists that as one of the conditions that prevents a view from being updated. So `StudentNames` can be queried but not modified through. This example is therefore a good illustration of the trade-off: ordering in a view costs updatability.

29.5 Creating a View from Multiple Tables

THE SOURCE'S INSTRUCTION

To create a View from multiple tables we can simply include multiple tables in the SELECT statement.

EXAMPLE 3 — MARKSVIEW (SOURCE)

Problem: create a view named `MarksView` from the two tables `StudentDetails` and `StudentMarks`, showing each student's name, address and marks.

```
CREATE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS
FROM   StudentDetails, StudentMarks
WHERE  StudentDetails.NAME = StudentMarks.NAME;
```

Line by line: the SELECT list qualifies each column with its table, which is required here because `NAME` exists in both tables. The FROM clause lists both tables, producing a Cartesian product; the WHERE clause then keeps only those pairs whose NAME matches, which is precisely an equi-join. The result is one row per student who appears in both tables.

Resulting view:

Table 29.5 — The view MarksView, joining StudentDetails and StudentMarks on NAME.

NAME	ADDRESS	MARKS
Ramesh	Ahmedabad	90
Khilan	Delhi	85
kaushik	Kota	70
Chaitali	Mumbai	80
Hardik	Bhopal	75
Komal	MP	95

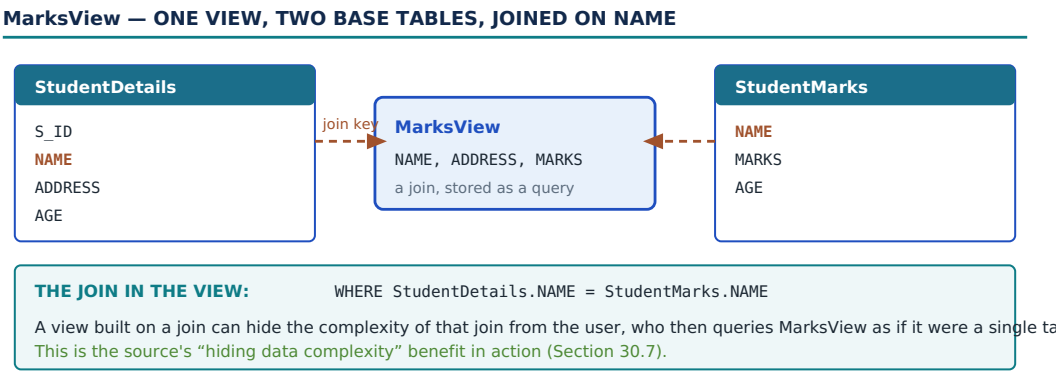


Figure 29.2 — MarksView draws on two base tables. The join condition lives inside the view definition, so the user never has to write it.

29.6 Querying a View

THE SOURCE'S INSTRUCTION

To see the data in the View, we can query the view in the same manner as we query a table:

```
SELECT * FROM DetailsView;
SELECT * FROM StudentNames;
SELECT * FROM MarksView;
```

This is the point of a view, and it is easy to overlook: **once created, a view is queried exactly like a table**. The user writes an ordinary `SELECT` and never needs to know whether the name refers to a base table or a view. In the case of `MarksView`, that single `SELECT *` hides a two-table join.

Chapter summary

SUMMARY — CHAPTER 29

- A **view** is a specific representation of data from one or more tables; the tables it refers to are its **base tables**.
- Creating a view takes no storage space — only the query is stored, in the data dictionary. The maximum number of columns in a view is 1000.
- Benefits: reducing complexity, improving security, and renaming columns.
- `CREATE VIEW name AS SELECT … FROM … WHERE …;` creates a view from one table; including several tables in the FROM clause creates a view from multiple tables.
- A view is queried exactly like a table, with an ordinary `SELECT`.

30. Modifying, Dropping and Updating Views

Part 7 • what a view allows you to change, and what it forbids

So far a view has been read-only. This chapter covers the rest of its life: removing it, redefining it, and — most importantly for examinations — the exact conditions under which a view can be written through.

30.1 Dropping a View

SYNTAX (SOURCE)

We have learned about creating a View, but what if a created View is not needed any more? SQL allows us to delete an existing View. We can delete or drop a View using the DROP statement.

```
DROP VIEW view_name;
```

Part	Meaning (source)
`view_name`	Name of the View which we want to delete

EXAMPLE (SOURCE)

```
DROP VIEW MarksView;
```

What this does: it removes the view definition from the data dictionary. It does **not** delete any data, because the view never held any data of its own — the rows it displayed came from `StudentDetails` and `StudentMarks`, and both tables are untouched. This is an important distinction to state in an answer: dropping a view destroys a query, not data.

30.2 Creating or Replacing a View

SYNTAX (SOURCE)

We can use the `CREATE OR REPLACE VIEW` statement to add or remove fields from a view.

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2,..
FROM   table_name
WHERE  condition;
```

EXAMPLE — ADDING A FIELD TO MARKSVIEW (SOURCE)

Problem: update the view `MarksView` and add the field `AGE` from the `StudentMarks` table.

```
CREATE OR REPLACE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS,
       StudentMarks.AGE
FROM   StudentDetails, StudentMarks
WHERE  StudentDetails.NAME = StudentMarks.NAME;
```

Result: the view now has four columns instead of three. Fetching all the data from `MarksView` gives the earlier three columns plus the age:

Table 30.1 — MarksView after the CREATE OR REPLACE, with AGE added.

NAME	ADDRESS	MARKS	AGE
Ramesh	Ahmedabad	90	32
Khilan	Delhi	85	25
kaushik	Kota	70	23
Chaitali	Mumbai	80	25
Hardik	Bhopal	75	27
Komal	MP	95	22

WHY CREATE OR REPLACE RATHER THAN DROP THEN CREATE

Both approaches achieve the same end, but `CREATE OR REPLACE` is preferable in practice. Dropping a view and recreating it creates a window in which the view does not exist, and it also discards any privileges granted on the view, which must then be granted again. `CREATE OR REPLACE` changes the definition in one step.

30.3 Conditions Required to Update a View

This is the most examinable part of the chapter, and the source states it as a numbered list of five conditions. The framing is important: there are certain conditions needed to be satisfied to update a view. If any one of these conditions is not met, then we will not be allowed to update the view.

THE FIVE CONDITIONS (SOURCE, VERBATIM INTENT)

1. The SELECT statement which is used to create the view should **not** include a `GROUP BY` clause or an `ORDER BY` clause.
2. The SELECT statement should **not** have the `DISTINCT` keyword.
3. The View should have all **NOT NULL** values.
4. The view should **not** be created using nested queries or complex queries.
5. The view should be created from a **single table**. If the view is created using multiple tables, we will not be allowed to update the view.

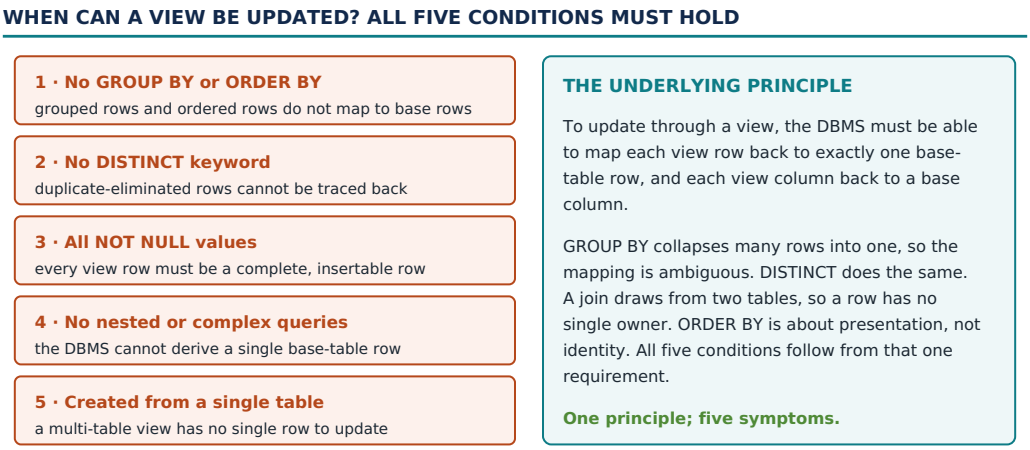


Figure 30.1 — The five conditions are not arbitrary rules: each one prevents the DBMS from mapping a view row back to a single base-table row.

EXPLANATION — A GOOD WAY TO REMEMBER THE FIVE

Notice that the five conditions all answer the same question: can the DBMS work out which base-table row this view row came from? `GROUP BY` and `DISTINCT` collapse several base rows into one, so the answer is no. A join mixes two tables, so there is no single row. A nested or complex query hides the mapping behind intermediate steps. And an ORDER BY says nothing about identity at all. If you remember the one question, you can reconstruct the five conditions in an examination.

30.4 Inserting a Row Through a View

SYNTAX (SOURCE)

We can insert a row in a View in the same way as we do in a table. We can use the `INSERT INTO` statement of SQL to insert a row in a View.

```
INSERT INTO view_name(column1, column2, column3,..)
VALUES(value1, value2, value3..);
```

Part	Meaning (source)
`view_name`	Name of the View

EXAMPLE — INSERTING INTO DETAILSVIEW (SOURCE)

Problem: insert a new row into `DetailsView`, the view created earlier from a single table.

```
INSERT INTO DetailsView(NAME, ADDRESS)
VALUES("Suresh","Gurgaon");
```

What happens: `DetailsView` is a single-table view with no GROUP BY, no ORDER BY and no DISTINCT, so it is updatable — and the INSERT is therefore redirected to the base table `StudentDetails`. A new row is added there with NAME = 'Suresh' and ADDRESS = 'Gurgaon'.

Result: querying the view again now shows five rows:

Table 30.2 — DetailsView after the INSERT.

NAME	ADDRESS
Ramesh	Ahmedabad
Khilan	Delhi
kaushik	Kota
Chaitali	Mumbai
Suresh	Gurgaon

NOTE THE COLUMN LIST — AND WHY IT MATTERS HERE

The INSERT names only `(NAME, ADDRESS)`. The base table `StudentDetails` has further columns (S_ID, AGE) which are not in the view, so they receive their default or NULL values in the new base-table row. That is why the view shows Suresh but cannot show an id or age for him: those columns are not part of the view at all. This is worth stating in an answer, because it shows that a view restricts what can be supplied as well as what can be seen.

30.5 Deleting a Row Through a View

SYNTAX AND THE KEY FACT (SOURCE)

Deleting rows from a view is also as simple as deleting rows from a table. We can use the `DELETE` statement of SQL to delete rows from a view. **Also, deleting a row from a view first deletes the row from the actual table, and the change is then reflected in the view.**

```
DELETE FROM view_name
WHERE condition;
```

Part	Meaning (source)
`view_name`	Name of view from where we want to delete rows
`condition`	Condition to select rows

EXAMPLE — DELETING THE ROW JUST INSERTED (SOURCE)

Problem: delete the last row from `DetailsView` — the one just added.

```
DELETE FROM DetailsView
WHERE NAME = "Suresh";
```

What happens: the DELETE is redirected to the base table `StudentDetails`, where the row with NAME = 'Suresh' is removed. The view then reflects the change, because it re-runs its query each time it is used.

Result: querying `DetailsView` again shows the original four rows, and Suresh is gone.

Table 30.3 — DetailsView after the

DELETE — back to four rows.

NAME	ADDRESS
Ramesh	Ahmedabad
Khilan	Delhi
kaushik	Kota
Chaitali	Mumbai

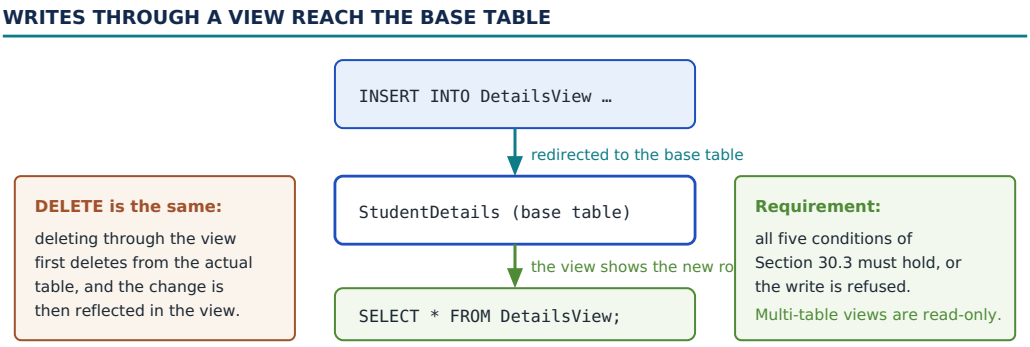


Figure 30.2 — A write through an updatable view is redirected to the base table, and the view reflects the change because it re-runs its query.

30.6 WITH CHECK OPTION

DEFINITION (SOURCE)

The `WITH CHECK OPTION` clause in SQL is a very useful clause for views. It is applicable to an updatable view. If the view is not updatable, then there is no meaning in including this clause in the CREATE VIEW statement.

- The `WITH CHECK OPTION` clause is used to **prevent the insertion of rows in the view where the condition in the WHERE clause in the CREATE VIEW statement is not satisfied**.
- If we have used the `WITH CHECK OPTION` clause in the CREATE VIEW statement, and if the UPDATE or INSERT clause does not satisfy the conditions, then they will **return an error**.

This is a genuinely useful mechanism, and its purpose is easy to state. A view's `WHERE` clause defines which rows belong to the view. Without `WITH CHECK OPTION`, you can insert a row through the view that does not satisfy that condition — and the row then vanishes from the view the moment it is inserted, because the view's query excludes it. `WITH CHECK OPTION` forbids that, turning a silent disappearance into an explicit error.

EXAMPLE — SAMPLEVIEW WITH WITH CHECK OPTION (SOURCE)

Problem: create a view of students whose NAME is not null, and prevent any insert that would violate that condition.

```
CREATE VIEW SampleView AS
SELECT S_ID, NAME
FROM   StudentDetails
WHERE  NAME IS NOT NULL
WITH CHECK OPTION;
```

What the view does: it selects only S_ID and NAME, and only for rows where NAME is not null. The `WITH CHECK OPTION` means that any INSERT or UPDATE through this view must also satisfy `NAME IS NOT NULL`, or the statement fails.

The statement the source shows as invalid:

```
INSERT INTO SampleView(S_ID)
VALUES(6);
```

Why it fails: this INSERT supplies only S_ID. It does not supply a NAME, and the source's note explains the consequence: the default value of the NAME column is null. So the new row would have NAME = NULL, which violates the view's `WHERE NAME IS NOT NULL` condition. Because the view was created `WITH CHECK OPTION`, the DBMS refuses the insert and returns an error.

THE KEY POINT TO STATE IN AN EXAM

The view is updatable — the source says so explicitly: “though the View is updatable but then also the below query for this View is not valid”. The reason the insert fails is not that the view is read-only, but that the new row would fall outside the view's own condition. `WITH CHECK OPTION` converts that situation from a silent anomaly into a refused statement.

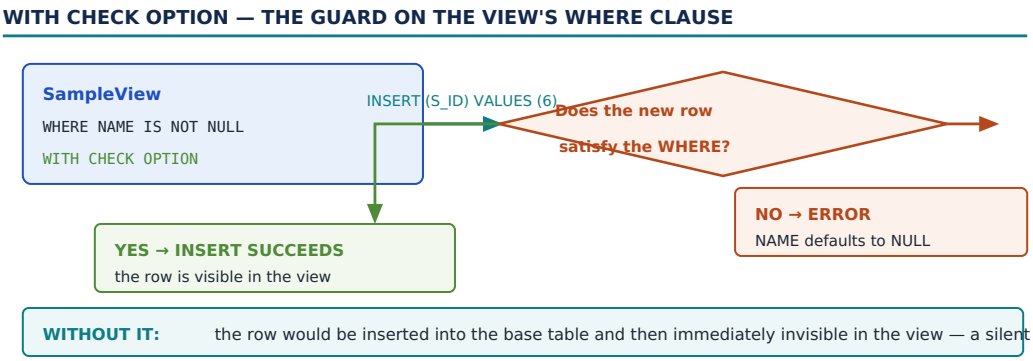


Figure 30.3 — `WITH CHECK OPTION` makes the view's WHERE clause a constraint on writes as well as reads.

30.7 Uses of a View

The source closes the chapter with six reasons a good database should contain views. This is a standard examination question, and the source's own wording is worth reproducing closely.

Table 30.4 — The six uses of a view (source).

#	Use	The source's explanation
1	**Restricting data access**	Views provide an additional level of table security by restricting access to a predetermined set of rows and columns of a table.
2	**Hiding data complexity**	A view can hide the complexity that exists in a multiple-table join.
3	**Simplifying commands for the user**	Views allow the user to select information from multiple tables without requiring the user to actually know how to perform a join.
4	**Storing complex queries**	Views can be used to store complex queries.
5	**Renaming columns**	Views can also be used to rename the columns without affecting the base tables, provided the number of columns in the view matches the number of columns specified in the SELECT statement. Thus renaming helps to hide the names of the columns of the base tables.
6	**Multiple view facility**	Different views can be created on the same table for different users.

EXPLANATION — THE SIX USES FALL INTO THREE GROUPS

It helps to see the pattern. Uses **1** and **5** are about **security and hiding**: restrict which rows and columns a user can reach, and obscure the underlying column names. Uses **2**, **3** and **4** are about **simplification**: put the join inside the view so the user never writes it. Use **6** is about **customisation**: one table, many audiences, each with their own view. If you can reproduce those three headings, the six individual uses follow naturally.

Chapter summary

SUMMARY — CHAPTER 30

- `DROP VIEW view_name;` removes a view definition. It deletes no data, because the view stored none.
- `CREATE OR REPLACE VIEW …` adds or removes fields from a view in one step, without a window in which the view does not exist.
- A view is **updatable** only if all five conditions hold: no `GROUP BY` or `ORDER BY`; no `DISTINCT`; all `NOT NULL` values; not created from nested or complex queries; and created from a **single table**.
- `INSERT INTO view …` and `DELETE FROM view …` are redirected to the base table, and the change is reflected in the view.
- `WITH CHECK OPTION` prevents an insert or update through the view that would not satisfy the view's `WHERE` condition; such a statement returns an error.
- Six uses: restricting data access, hiding data complexity, simplifying commands, storing complex queries, renaming columns, and the multiple-view facility.

PART 8

Revision and Examination

Consolidated comparison tables and a formula sheet · a full question bank with answers · MCQ with an answer key · viva questions · the last-minute revision sheet · source coverage and references

Everything in this part is drawn from the examples and definitions of Parts 1–7, which are themselves drawn from the supplied UNIT 2 material.

31. Consolidated Comparison Tables and Formula Sheet

Part 8 • everything that is easier to compare than to remember

This chapter gathers the comparisons that appear scattered through the notes into single tables. It is the chapter to read first when revising, because most examination questions are ultimately a comparison of two similar things.

31.1 Key Comparison Tables

Table 31.1 — The ten keys, one line each, with the single fact that distinguishes each.

Key	Definition in one line	The distinguishing fact
Super key	Any set of attributes that identifies each tuple uniquely	May contain attributes that are not needed
Candidate key	A minimal super key	No attribute can be removed without losing uniqueness
Primary key	The candidate key the designer implements	Exactly one per table; never NULL
Alternate key	A candidate key not chosen as primary	Still a valid minimal identifier
Foreign key	An attribute whose values must exist as a key elsewhere	May repeat and may be NULL; creates the relationship
Partial key	Identifies a group of related tuples, not one tuple	Cannot identify a single tuple
Composite key	A primary key of two or more attributes	Uniqueness comes from the combination
Unique key	Unique and non-updatable, but may be NULL	NULL is permitted; several per table
Surrogate key	A generated artificial identifier	Carries no meaning; updatable; never NULL
Secondary key	An attribute used for indexing	About search speed, not identity

Table 31.2 — Super key vs candidate key vs primary key.

Question	Super key	Candidate key	Primary key
Unique?	Yes	Yes	Yes
Minimal?	No	Yes	Yes
How many per table?	Many	One or more	Exactly one
NULL allowed?	Not meaningful	No	No
Chosen by designer?	No — it simply exists	No — it simply exists	**Yes**

Table 31.3 — Composite vs compound vs surrogate.

Question	Composite key	Compound key	Surrogate key
Number of attributes	Two or more	Two or more	One
Made of real data attributes?	Yes	Yes	**No — artificial**
Are the parts foreign keys?	May or may not be	**Yes, all of them**	No
Carries meaning?	Yes	Yes	**No**

31.2 Normal-Form Summary Table

Table 31.4 — The six normal forms: the rule, the dependency removed, and the test.

Form	Rule	Removes	Test to apply
1NF	Every attribute holds an atomic value	Multi-valued attributes, repeating groups	Can any cell need to hold two values?
2NF	1NF + every non-prime attribute is fully functionally dependent on the key	Partial dependencies	Does a non-prime attribute depend on only part of a composite key?
3NF	2NF + no non-prime attribute is transitively dependent on a key. For every non-trivial X → A: X is a super key **or** A is prime	Transitive dependencies	Is the determinant a super key, or is the dependent prime?
BCNF	3NF + for every non-trivial X → A, X is a super key	Non-super-key determinants	Is every determinant a super key?
4NF	BCNF + no multivalued dependency	Multivalued dependencies	Are two independent multi-valued facts sharing one table?
5NF	4NF + every join dependency is implied by the candidate keys	Join dependencies not implied by keys	Can the table be losslessly split further on a rule, not a key?

Table 31.5 — What each form guarantees, and what it costs.

Form	Guarantee	Cost or caveat
1NF	No lists inside cells; every value queryable	Introduces repetition of the other columns
2NF	No non-prime attribute depends on part of a key	More tables, so more joins
3NF	No indirect dependencies; always achievable dependency-preserving	Some redundancy can remain in rare cases
BCNF	Every determinant is a super key; always lossless	Not always dependency-preserving
4NF	No cross-product redundancy from independent facts	More tables
5NF	No join dependency beyond those implied by keys	Detecting join dependencies is very hard in practice

31.3 Dependency Rules and Axioms

Table 31.6 — Armstrong's axioms and the derived rules.

Rule	Statement	Kind
Reflexivity	If $Y \subseteq X$ then $X \to Y$	Axiom
Augmentation	If $X \to Y$ then $XZ \to YZ$	Axiom
Transitivity	If $X \to Y$ and $Y \to Z$ then $X \to Z$	Axiom
Union	If $X \to Y$ and $X \to Z$ then $X \to YZ$	Derived
Decomposition	If $X \to YZ$ then $X \to Y$ and $X \to Z$	Derived (right side only)
Pseudotransitivity	If $X \to Y$ and $WY \to Z$ then $WX \to Z$	Derived

Table 31.7 — The dependency types and the form that removes each.

Dependency	Notation	Meaning	Removed by
Trivial FD	$X \to Y, Y \subseteq X$	Always true; no information	—
Non-trivial FD	$X \to Y, Y \not\subset X$	Genuinely constrains the data	—
Partial dependency	part of key → non-prime	Non-prime depends on part of a composite key	2NF
Transitive dependency	$X \to Y \to Z, Y \not\to X$	Indirect dependency through a non-prime attribute	3NF
Multivalued dependency	$P \twoheadrightarrow Q, P \twoheadrightarrow R$	Two independent multi-valued facts	4NF
Join dependency	*(A, B, C)	Table reconstructible from projections	5NF

31.4 PL/SQL Control-Structure Summary

Table 31.8 — The IF forms and what each is for.

Form	When to use	Branches
`IF-THEN`	Act only when the condition is true	1
`IF-THEN-ELSE`	Two mutually exclusive outcomes	2
`IF-THEN-ELSIF`	Several conditions, no default needed	2 or more
`IF-THEN-ELSIF-ELSE`	Several conditions plus a default	3 or more
`CASE`	One selector tested against a list of values	One per WHEN, plus optional ELSE

Table 31.9 — The loop forms compared. This table answers most loop questions on its own.

Question	Basic LOOP	WHILE-LOOP	FOR-LOOP
Where is the condition tested?	Nowhere — you exit explicitly	At the **top**	Range is fixed before entry
Can the body run zero times?	**No — always at least once**	**Yes**	**Yes, if the range is empty**
Is the iteration count known in advance?	No	No	**Yes**
How does it terminate?	`EXIT` or `EXIT WHEN`	Condition becomes false or null	Counter passes the bound
Is there a loop counter?	No	No	Yes — cannot be assigned

Table 31.10 — GOTO restrictions and the NULL statement.

Item	Rule
GOTO target	Label must be unique in scope and precede an executable statement or block
Direction	May go up or down, but only within the same block
Into a construct	**Not allowed into IF, CASE, LOOP or a sub-block**
Between clauses	**Not allowed from one IF clause or CASE WHEN to another**
Out of a subprogram	**Not allowed; use RETURN, or jump to just before the end**
Exception handler	**Cannot jump back into the current block; can jump out to an enclosing block**
NULL	A no-op; used to document that a branch is deliberately empty

31.5 Cursor and View Summary

Table 31.11 — Implicit vs explicit cursors (condensed).

Question	Implicit	Explicit
Declared by	Oracle	The programmer
Open / close	Oracle does both automatically	You do both
Attributes	`SQL%FOUND, SQL%NOTFOUND, SQL%ISOPEN, SQL%ROWCOUNT`	`c%FOUND, c%NOTFOUND, c%ISOPEN, c%ROWCOUNT`
`%ISOPEN` value	**Always FALSE**	**TRUE between OPEN and CLOSE**
Used for	Single-row queries, DML row counts	Multi-row queries processed row by row

Table 31.12 — View operations and the five conditions for updatability.

Operation	Statement
Create	`CREATE VIEW v AS SELECT … FROM … WHERE …;`
Query	`SELECT * FROM v;`
Redefine	`CREATE OR REPLACE VIEW v AS …;`
Drop	`DROP VIEW v;`
Insert through	`INSERT INTO v(col, …) VALUES(…);`
Delete through	`DELETE FROM v WHERE …;`
Guard writes	`CREATE VIEW v AS … WHERE … WITH CHECK OPTION;`

Table 31.13 — The five conditions a view must satisfy to be updatable.

#	Condition	Why
1	No `GROUP BY` or `ORDER BY`	Grouped rows do not map to base rows; ordering is presentation only
2	No `DISTINCT`	Duplicate elimination loses the row identity
3	All `NOT NULL` values	Every view row must be a complete, insertable row
4	No nested or complex queries	The DBMS cannot derive a single base-table row
5	Created from a single table	A join has no single row to update

31.6 Security Summary

Table 31.14 — The security material in one table.

Concept	Definition in one line
Database security	Protection from malicious attempts to steal or modify data
Confidentiality	Protection of data from unauthorized disclosure
Integrity	Protection from improper modification
Availability	Making objects available to a user or program with a legitimate right
Threat	Any intentional or accidental event that may adversely affect the system
Authentication	Who are you? Prove it!
Authorization	What you are allowed to do
DAC	Owner-controlled access via grants, revokes and ACLs
MAC	Central authority assigns security labels and clearance levels
RBAC	Permissions attach to predefined roles based on job responsibilities
Inference control	Prevents indirect disclosure via inference channels
Flow control	Prevents information reaching unauthorized users; illicit paths are covert channels
Statistical DB security	Permits aggregate information, not individual values
Encryption	Encodes sensitive data; main issue is key management
DBA	Central authority responsible for the overall security of the database system

Table 31.15 — The six control measures and the threat each addresses.

Control	Addresses
Authentication	Impersonation; unauthorized entry
Access control	Reading or modifying beyond one's privilege; also tracks operations
Inference control	Deducing sensitive facts from legitimate answers
Flow control	Information reaching unauthorized users
Statistical database security	Individual values exposed through statistics
Encryption	Loss or theft of the storage medium

32. Question Bank with Answers

Part 8 • two-mark, five-mark, descriptive, output-prediction, debugging and normalization problems

Every question below is answerable from the material in Parts 1–7, and the answers are written in the form an examiner expects: the key term first, then the supporting detail. Answers to output-prediction and normalization questions show the reasoning, not merely the result.

32.1 Two-Mark Questions

Table 32.1 — Two-mark questions with direct answers.

#	Question	Answer
1	Define a super key.	A set of attributes that identifies each tuple uniquely in a relation; it may contain attributes that are not needed for uniqueness.
2	What is a candidate key?	A minimal super key — a super key from which no attribute can be removed without losing uniqueness.
3	What is a primary key?	The candidate key that the database designer selects and implements; one per table, never NULL, never updated while referenced.
4	What is an alternate key?	A candidate key that is not the primary key.
5	Define a foreign key.	An attribute whose values must exist as values of a key in another (or the same) relation; it creates the relationship and enforces referential integrity.
6	What is a partial key?	A key that cannot identify a tuple uniquely but can select a group of related tuples.
7	What is a composite key?	A primary key made up of two or more attributes, where the combination guarantees uniqueness.
8	What is a surrogate key?	An artificial key, usually an integer, generated to identify each record uniquely when no natural primary key exists; it carries no meaning.
9	What is a secondary key?	A key required for indexing, so that searching is faster.
10	Define functional dependency.	A constraint X → Y meaning that any two tuples agreeing on X must also agree on Y.
11	What is a trivial functional dependency?	A dependency A → B in which B is a subset of A; it is always true and carries no information.
12	What is a transitive dependency?	X → Z formed indirectly, where X → Y and Y → Z and Y does not determine X.
13	What is a multivalued dependency?	A situation where the existence of rows for one value of P implies further rows, with two independent multi-valued facts about P.
14	What is a join dependency?	A condition where a table can be recreated by joining multiple tables each holding a subset of its attributes; a generalisation of the multivalued dependency.
15	State the purpose of normalization.	To design a consistent database by minimising redundancy and ensuring data integrity through lossless decomposition.
16	Name the three anomalies.	Update anomaly, insertion anomaly and deletion anomaly.

#	Question	Answer
17	State the 1NF rule.	Every attribute must contain an atomic value; multi-valued attributes, composite attributes and repeating groups are disallowed.
18	State the 2NF rule.	The relation is in 1NF and every non-prime attribute is fully functionally dependent on the key — no proper subset of the key determines it.
19	State the 3NF rule.	The relation is in 2NF and no non-prime attribute is transitively dependent on a key; equivalently, for every non-trivial X → A, X is a super key or A is prime.
20	State the BCNF rule.	For every non-trivial functional dependency X → A, X must be a super key.
21	State the 4NF rule.	The relation is in BCNF and has no multivalued dependency.
22	State the 5NF rule.	Every join dependency in the relation is implied by the candidate keys.
23	Define denormalization.	An optimization technique that adds redundant data to one or more tables to avoid costly joins; applied after normalization.
24	What is a lossless decomposition?	A decomposition whose relations, when joined back, reproduce exactly the original relation with no rows lost and no spurious rows created.
25	What is dependency preservation?	A property of a decomposition whereby every functional dependency of the original relation can be checked on the resulting relations.
26	Define database security.	Protection from malicious attempts to steal (view) or modify data.
27	Name the three properties of the CIA triad.	Confidentiality, integrity and availability.
28	What is a threat?	Any intentional or accidental event that may adversely affect the system.
29	Distinguish authentication from authorization.	Authentication establishes who the user is; authorization decides what that user is allowed to do. Authentication comes first.
30	What is DAC?	Discretionary access control — users control their own data and permissions are granted using access control lists.
31	What is MAC?	Mandatory access control — a central authority assigns security labels to data and users so that users access only data within their clearance.
32	What is RBAC?	Role-based access control — permissions are attached to predefined roles, and access is assigned by job responsibility.
33	What is inference control?	The countermeasure to the statistical-database security problem; it prevents indirect disclosure through inference channels.
34	What is flow control?	A control that prevents information from flowing in a way that reaches unauthorized users; the illicit paths are covert channels.
35	What is statistical database security?	Protection of confidential individual values while permitting aggregate or summary information to be retrieved.

#	Question	Answer
36	What are the DBA's responsibilities?	Granting privileges to users who need them, and classifying users and data according to organizational policy; overall security of the system.
37	Name the three categories of PL/SQL control structure.	Conditional control, iterative control and sequential control.
38	What does EXIT WHEN do?	It completes a loop conditionally: if the WHEN condition is true, the loop ends and control passes to the next statement after the loop.
39	What does the NULL statement do?	Nothing — it is a no-op that passes control to the next statement; it documents that a branch is deliberately empty.
40	Define a cursor.	A pointer to the result set of a SQL query against database tables.
41	What is the active set?	The rows held in the context area — those meeting the condition; its size equals the number of such rows.
42	Name the four implicit cursor attributes.	`%FOUND`, `%NOTFOUND`, `%ISOPEN` and `%ROWCOUNT`.
43	Why is `SQL%ISOPEN` always FALSE?	Because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
44	Name the four steps of an explicit cursor.	Declare, open, fetch and close.
45	Define an SQL view.	A specific representation of data from one or more tables; the tables it refers to are its base tables.
46	Does creating a view take storage space?	No — only the query is stored in the data dictionary; the actual data is not stored anywhere.
47	What is the maximum number of columns in a view?	1000, as in tables.
48	What does WITH CHECK OPTION do?	It prevents an INSERT or UPDATE through the view that would not satisfy the WHERE condition of the CREATE VIEW statement, returning an error instead.
49	Does dropping a view delete data?	No — it removes the view definition; the base-table data is untouched because the view stored no data.
50	Name three benefits of views.	Reducing complexity, improving security, and renaming the table columns.

32.2 Five-Mark Questions

Q1. Explain the difference between a super key, a candidate key and a primary key, with an example.

Answer. A **super key** is any set of attributes that identifies each tuple uniquely; it may contain attributes that are not needed for uniqueness. A **candidate key** is a minimal super key: removing any attribute destroys uniqueness. The **primary key** is the candidate key the designer chooses to implement.

Consider `Student ( roll, name, sex, age, address, class, section )`, assuming names are unique. The whole tuple is a super key. `( class, section, roll, sex )` is also a super key, but it is not minimal because `sex` is removable. `( class, section, roll )` and `( name, address )` are candidate keys. If the designer implements `( class, section, roll )` as the primary key, then `( name, address )` becomes the alternate key. A table may have several candidate keys but exactly one primary key.

Q2. Explain the three anomalies that normalization removes, with an example of each.

Answer. Take an unnormalized relation `( Stu_ID, Stu_Name, Dept, Course, Instructor )` with key (Stu_ID, Course).

Update anomaly. A student's department is stored once per course the student takes. Updating the department in one row leaves the other row unchanged, so the database now contains two different answers to the same question — an inconsistent state.

Insertion anomaly. A new course with its instructor cannot be recorded until a student enrols in it, because the primary key requires a Stu_ID. Recording the fact would need an invented or NULL key value.

Deletion anomaly. If one student is the only person taking a course, deleting that student also deletes the fact that the course is taught by that instructor — information that belonged to the course, not the student.

All three arise from the same cause: facts about different things (students, courses, instructors) sharing one relation. Normalization separates them.

Q3. State and explain the five conditions required to update a view.

Answer. A view is updatable only if all five hold:

1. The defining SELECT must not contain `GROUP BY` or `ORDER BY`. Grouped rows do not correspond to base rows, and ordering is a presentation concern.
2. It must not contain `DISTINCT`. Eliminating duplicates loses the identity of the underlying rows.
3. All view values must be NOT NULL, so that each view row is a complete, insertable row.
4. The view must not be built from nested or complex queries, which hide the mapping to the base table.
5. The view must be built from a **single** table. A view over multiple tables (a join) has no single row to update.

The five conditions share one underlying requirement: the DBMS must be able to map each view row back to exactly one base-table row, and each view column back to a base column.

Q4. Explain the six control measures of database security and the threat each addresses.

Answer. The source lists six:

1. **Authentication** — confirms that a user is who they claim to be, using passwords or biometrics such as retina and fingerprints; addresses impersonation and unauthorized entry.
2. **Access control** — creates user accounts and restricts access so that sensitive data is reachable only by authorized users; the DBMS must also track all operations a user performs during the login. Addresses reading or modifying data beyond one's privilege.
3. **Inference control** — the countermeasure to the statistical-database problem; prevents indirect disclosure through inference channels. Inferences are identity disclosure or attribute disclosure.
4. **Flow control** — prevents information from flowing to unauthorized users; the illicit pathways are called covert channels.
5. **Statistical database security** — permits summary information such as the number of employees, but not the confidential individual values.
6. **Encryption** — encodes sensitive data such as credit card and OTP numbers, protecting it when the storage medium itself is lost.

Each control addresses a different threat, which is why all six are needed.

Q5. Explain the three forms of LOOP statement in PL/SQL, and state how many times each body can execute.

Answer.

1. **Basic LOOP.** Statements are enclosed between `LOOP` and `END LOOP` with no built-in condition; termination requires `EXIT` or `EXIT WHEN` inside the body. Because there is no test before the first execution, the body runs **at least once**.
2. **WHILE-LOOP.** `WHILE condition LOOP … END LOOP;`. The condition is evaluated before each iteration; if it is false or null the loop is bypassed. The body may therefore run **zero times**, and the number of iterations is unknown until the loop completes.
3. **FOR-LOOP.** `FOR counter IN [REVERSE] lower..higher LOOP … END LOOP;`. The range is evaluated once when the loop is first entered and is never re-evaluated. The body runs **once per integer** in the range — exactly once if the bounds are equal, and zero times only if the range is empty. The counter can be read but not assigned.

Q6. Explain the restrictions on the GOTO statement in PL/SQL.

Answer. A GOTO branches unconditionally to a label, which must be unique within its scope and must precede an executable statement or a PL/SQL block. The restrictions are:

- It cannot branch **into** an IF statement, CASE statement, LOOP statement or sub-block.
- It cannot branch from one IF statement clause to another, or from one CASE WHEN clause to another.
- It cannot branch from an outer block into a sub-block (an inner BEGIN-END block).
- It cannot branch out of a subprogram. To end a subprogram early, use `RETURN`, or have GOTO branch to a place just before the end of the subprogram.
- It cannot branch from an exception handler back into the current BEGIN-END block — but it **can** branch from an exception handler into an enclosing block.

The source adds that GOTO is seldom needed and should be used sparingly, because overuse makes control flow hard to follow; an exception handler is usually preferable to branching out of deeply nested structure.

32.3 Ten-Mark / Descriptive Questions

Q1. Explain the complete normalization ladder from 1NF to 5NF, stating the rule and the problem addressed at each level. Illustrate with the Student_Project and Employee examples.

Answer structure. Define normalization and its purpose. Then, for each form, state the rule, name the dependency removed, and give the source's example:

- **1NF** — atomic values only. Example: the EMPLOYEE table whose EMP_PHONE column holds several numbers in one cell; splitting them into separate rows reaches 1NF and makes the key composite.
- **2NF** — no partial dependency. Example: `Student_Project ( Stu_ID, Proj_ID, Stu_Name, Proj_Name )`, where Stu_Name depends only on Stu_ID and Proj_Name only on Proj_ID. Decomposing into a Student relation and a Project relation removes the partial dependencies; a junction relation preserves the many-to-many fact.
- **3NF** — no transitive dependency. Example: `Employee ( Emp_ID, Emp_Name, Emp_Zip, Emp_State, Emp_City, Emp_District )`, where Emp_Zip determines the location attributes. Splitting into Employee_ID and Employee_Zip stores each location fact once.
- **BCNF** — every determinant a super key. Example: the Employee relation with candidate key {Emp_ID, Emp_Dept}, where Emp_ID → Emp_Nationality and Emp_Dept → Dept_Type, No_of_Employees have determinants that are only part of the key. Three relations — Employee_Nationality, Employee_Department and Employee_ID_Dept — make every determinant a key.
- **4NF** — no multivalued dependency. Example: `Students ( Stud_ID, Stud_Course, Stud_Hobby )`, where course and hobby are independent facts, producing a cross product. Splitting into StudentCourse and StudentHobby removes it.
- **5NF** — every join dependency implied by the candidate keys. Example: the agent/company/product table, where a tuple-formation rule makes the join dependency real; the source notes that detecting such dependencies is very hard, so 5NF is largely of academic interest.

Conclude with the cumulative nature of the ladder and the note that 3NF is usually sufficient in practice, with denormalization applied afterwards only when measurement justifies it.

Q2. Discuss database security in full: definition, importance, the levels of security, threats, and the control measures.

Answer structure.

1. **Definition and scope.** Protection from malicious attempts to steal or modify data; formally the array of controls, tools and procedures safeguarding confidentiality, integrity and accessibility. Security must cover the data, the DBMS, associated applications, the physical/virtual server and its hardware, and the computing/network infrastructure.
2. **Importance.** Loss of intellectual property, damage to brand reputation, loss of business continuity, and the cost of investigation, repair and customer notification.
3. **Threats and the three losses.** Loss of integrity (improper modification, leading to inaccuracy, fraud or erroneous decisions); loss of availability (objects withheld from legitimate users); loss of confidentiality (unauthorized disclosure, from violation of the Data Privacy Act to jeopardised national security).
4. **Levels.** Human, network/user interface, database application program, database system, operating system and physical. Physical security uses lock-and-key, disaster protection and backup; OS security is critical against viruses and worms.
5. **Network and human level.** Eavesdropping, masquerading and person-in-the-middle attacks, handled by https://; passwords fail because they are guessable and shared; smartcards require the card plus a PIN.
6. **Control measures.** Authentication, access control, inference control, flow control, statistical database security and encryption, each tied to the threat it addresses.
7. **DBA.** The central authority: grants privileges, classifies users and data, and holds the superuser account with account creation, privilege granting, privilege revocation and security level assignment.

32.4 Output-Prediction Questions

Q1 — A BASIC LOOP WITH EXIT

Predict the output.

```
DECLARE
   n NUMBER := 0;
BEGIN
   LOOP
      DBMS_OUTPUT.PUT_LINE ('inside: ' || n);
      n := n + 1;
      IF n > 3 THEN
         EXIT;
      END IF;
   END LOOP;
   DBMS_OUTPUT.PUT_LINE ('outside: ' || n);
END;
/
```

Reasoning. This is a basic LOOP, so the body runs at least once. n is printed before the increment, and the exit test comes after the increment, so the printed sequence is 0, 1, 2, 3 while the value at exit is 4.

```
inside: 0
inside: 1
inside: 2
inside: 3
outside: 4
```

Q2 — A WHILE THAT NEVER RUNS

Predict the output.

```
DECLARE
   total NUMBER := 30000;
BEGIN
   WHILE total <= 25000 LOOP
      DBMS_OUTPUT.PUT_LINE ('in loop');
      total := total + 1000;
   END LOOP;
   DBMS_OUTPUT.PUT_LINE ('after loop, total = ' || total);
END;
/
```

Reasoning. The WHILE condition is tested at the top. total starts at 30000, which is greater than 25000, so the condition is false on the first test and the loop body never executes. This is exactly the case the source describes: the loop is bypassed.

```
after loop, total = 30000
```

Q3 — A FOR LOOP WITH REVERSE

Predict the output.

```
BEGIN
   FOR i IN REVERSE 1..4 LOOP
      DBMS_OUTPUT.PUT (i || ' ');
   END LOOP;
   DBMS_OUTPUT.NEW_LINE;
END;
/
```

Reasoning. REVERSE counts downward from the higher bound to the lower, so the counter takes the values 4, 3, 2, 1. The bounds are still written in ascending order (1..4), as the source requires.

```
4 3 2 1
```

Q4 — CASE WITH NO MATCHING WHEN AND NO ELSE

What happens?

```
DECLARE
   grade CHAR(1) := 'E';
BEGIN
   CASE grade
      WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Distinction');
      WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('First class');
      WHEN 'C' THEN DBMS_OUTPUT.PUT_LINE('Second class');
   END CASE;
END;
/
```

Reasoning. The selector is 'E'. No WHEN matches, and this CASE has no ELSE branch. A CASE statement with no matching branch and no ELSE raises a run-time error (`CASE_NOT_FOUND`). The correct prediction is therefore an error, not empty output — which is the point of the question.

Q5 — CURSOR LOOP OUTPUT

Given the CUSTOMERS table (Table 27.3), predict the output.

```
DECLARE
   CURSOR c IS SELECT name, salary FROM customers WHERE salary > 4000;
   n VARCHAR2(20);
   s NUMBER;
BEGIN
   OPEN c;
   LOOP
      FETCH c INTO n, s;
      EXIT WHEN c%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(n || ' ' || s);
   END LOOP;
   CLOSE c;
END;
/
```

Reasoning. The cursor selects only rows with salary greater than 4000. From Table 27.3 those are Chaitali (6500), Hardik (8500) and Komal (4500). Ramesh, Khilan and kaushik are excluded by the condition.

```
Chaitali 6500
Hardik 8500
Komal 4500
```

Q6 — SQL%ROWCOUNT AFTER AN UPDATE

Predict the output.

```
DECLARE
   r NUMBER;
BEGIN
   UPDATE customers SET salary = salary + 500 WHERE age < 26;
   r := sql%rowcount;
   DBMS_OUTPUT.PUT_LINE(r || ' rows updated');
END;
/
```

Reasoning. The WHERE clause restricts the update to customers aged under 26. From Table 27.3 those are Khilan (25), kaushik (23), Chaitali (25) and Komal (22) — four rows. So `sql%rowcount` is 4.

```
4 rows updated
```

32.5 Debugging Questions

Q1 — FIND THE ERROR

```
DECLARE
   a NUMBER := 10;
BEGIN
   IF a > 5
      DBMS_OUTPUT.PUT_LINE('big');
   END IF;
END;
/
```

Answer. The `THEN` keyword is missing. The correct form is:

```
   IF a > 5 THEN
      DBMS_OUTPUT.PUT_LINE('big');
   END IF;
```

Q2 — FIND THE ERROR

```
DECLARE
   n NUMBER := 0;
BEGIN
   WHILE n < 5 LOOP
      DBMS_OUTPUT.PUT_LINE(n);
   END LOOP;
END;
/
```

Answer. The loop is infinite. Nothing inside the body changes `n`, so the condition `n < 5` remains true forever. The source states the rule directly: a statement inside the loop must change the value of the condition. Adding `n := n + 1;` inside the body fixes it.

Q3 — FIND THE ERROR

```
BEGIN
   FOR i IN 1..3 LOOP
      DBMS_OUTPUT.PUT_LINE(i);
      i := i + 1;
   END LOOP;
END;
/
```

Answer. The loop counter cannot be assigned inside a FOR loop. The counter may be referenced like a constant but not modified, because the loop itself controls it. The statement `i := i + 1;` must be removed.

Q4 — FIND THE ERROR

```
DECLARE
   CURSOR c IS SELECT id, name FROM customers;
BEGIN
   LOOP
      FETCH c INTO v_id, v_name;
      EXIT WHEN c%NOTFOUND;
   END LOOP;
END;
/
```

Answer. Three faults. First, the cursor is never `OPEN`ed, so the FETCH raises `ORA-01001: invalid cursor`. Second, the variables `v_id` and `v_name` are never declared. Third, the cursor is never `CLOSE`d. The corrected skeleton is:

```
DECLARE
   v_id   customers.id%type;
   v_name customers.name%type;
   CURSOR c IS SELECT id, name FROM customers;
BEGIN
   OPEN c;
   LOOP
      FETCH c INTO v_id, v_name;
      EXIT WHEN c%NOTFOUND;
   END LOOP;
   CLOSE c;
END;
/
```

Q5 — FIND THE ERROR

```
CREATE VIEW HighMarks AS
SELECT StudentDetails.NAME, StudentMarks.MARKS
FROM   StudentDetails, StudentMarks
WHERE  StudentDetails.NAME = StudentMarks.NAME
ORDER BY StudentMarks.MARKS;

INSERT INTO HighMarks(NAME, MARKS) VALUES('Test', 99);
```

Answer. The INSERT will be refused. The view is built from **two** tables (it is a join) and it also contains an `ORDER BY` clause. Two of the five conditions for an updatable view are therefore violated: a view must be created from a single table, and its defining SELECT must not contain `ORDER BY`. The view is read-only.

Q6 — FIND THE ERROR

```
CREATE VIEW SampleView AS
SELECT S_ID, NAME
FROM   StudentDetails
WHERE  NAME IS NOT NULL
WITH CHECK OPTION;

INSERT INTO SampleView(S_ID) VALUES (6);
```

Answer. This is the source's own example, and it is a deliberate trap. The view is updatable, so the INSERT is attempted — but it supplies no NAME, and the default value of NAME is NULL. The new row would therefore violate the view's `WHERE NAME IS NOT NULL` condition. Because the view was created `WITH CHECK OPTION`, the DBMS refuses the insert and returns an error.

32.6 Normalization Problems with Step-by-Step Solutions

PROBLEM 1 — DRIVE A RELATION TO 3NF

Relation. `R ( Stu_ID, Stu_Name, Zip, City, Course )`, with `Stu_ID → Stu_Name, Zip`, `Zip → City`, and the key being `( Stu_ID, Course )`.

Step 1 — is it in 1NF? Assume all values are atomic, so yes.

Step 2 — is it in 2NF? The key is (Stu_ID, Course). Test each non-prime attribute:

Attribute	Determined by	Whole key needed?
Stu_Name	Stu_ID	**No — partial**
Zip	Stu_ID	**No — partial**
City	Zip	**No — not even on the key**

Not in 2NF. **Fix:** split along the partial dependencies into `Student ( Stu_ID, Stu_Name, Zip )` and `Enrolment ( Stu_ID, Course )`.

Step 3 — is the result in 3NF? Student has key Stu_ID, and the dependency `Stu_ID → Zip` is fine. But where does `City` live? If City is placed in Student, then `Stu_ID → Zip → City` is a transitive dependency, so Student would not be in 3NF. **Fix:** move City into a relation keyed by Zip: `ZipInfo ( Zip, City )`.

Final schema (all in 3NF):

```
Student   ( Stu_ID, Stu_Name, Zip )     key Stu_ID
ZipInfo   ( Zip, City )                 key Zip
Enrolment ( Stu_ID, Course )            key (Stu_ID, Course)
```

Check the decomposition is lossless. Student ∩ ZipInfo = { Zip }, and Zip is a super key of ZipInfo, so the join Student ⋈ ZipInfo is lossless. Enrolment shares Stu_ID with Student, and Stu_ID is a super key of Student, so that join is lossless too.

PROBLEM 2 — DRIVE A RELATION TO BCNF

Relation. `R ( Emp_ID, Emp_Nationality, Emp_Dept, Dept_Type, No_of_Employees )` — the source's own BCNF example.

Step 1 — find the candidate key. Neither Emp_ID alone nor Emp_Dept alone determines everything, so the candidate key is `{ Emp_ID, Emp_Dept }`.

Step 2 — list the dependencies.

```
Emp_ID   → Emp_Nationality
Emp_Dept → Dept_Type, No_of_Employees
```

Step 3 — test BCNF. For each dependency, is the determinant a super key?

Dependency	Determinant	Super key?
`Emp_ID → Emp_Nationality`	`Emp_ID`	**No — only part of the key**
`Emp_Dept → Dept_Type, No_of_Employees`	`Emp_Dept`	**No — only part of the key**

Not in BCNF. **Step 4 — fix:** give each non-super-key determinant its own relation, and keep a junction for the original relationship.

```
Employee_Nationality ( Emp_ID, Emp_Nationality )              key Emp_ID
Employee_Department  ( Emp_Dept, Dept_Type, No_of_Employees ) key Emp_Dept
Employee_ID_Dept     ( Emp_ID, Emp_Dept )                     key (Emp_ID, Emp_Dept)
```

Step 5 — verify. In each relation, every determinant is now the key, hence a super key. All three are in BCNF. The junction relation has no non-key attributes, so it is trivially in BCNF.

PROBLEM 3 — DRIVE A RELATION TO 4NF

Relation. `Students ( Stud_ID, Stud_Course, Stud_Hobby )` — the source's own 4NF example. Suppose student S1 takes DBMS and OS, and has hobbies Cricket and Music.

Step 1 — is there a multivalued dependency? For a single Stud_ID, the set of courses and the set of hobbies are independent. So `Stud_ID ↠ Stud_Course` and `Stud_ID ↠ Stud_Hobby`.

Step 2 — confirm the symptom. The table holds the cross product: two courses × two hobbies = four rows. Every course is paired with every hobby, which is redundancy, not information.

Step 3 — fix. Split into one relation per multi-valued fact:

```
StudentCourse ( Stud_ID, Stud_Course )   key (Stud_ID, Stud_Course)
StudentHobby  ( Stud_ID, Stud_Hobby )    key (Stud_ID, Stud_Hobby)
```

Step 4 — verify. Each relation is in BCNF (the whole key determines the non-key attribute), and neither contains a multivalued dependency, so both are in 4NF. The two relations together carry exactly the same information as the original, with the cross-product rows removed.

PROBLEM 4 — TEST A DECOMPOSITION FOR LOSSLESSNESS

Relation. `R ( A, B, C, D )` with dependencies `A → B` and `C → D`, key `( A, C )`.

Question. Is the decomposition into `R1 ( A, B )` and `R2 ( A, C, D )` lossless?

Test. R1 ∩ R2 = { A }. Is A a super key of R1? Yes — A → B and R1 = { A, B }. So the decomposition is **lossless**.

Question. Is the alternative decomposition into `R1 ( A, B )` and `R2 ( C, D )` lossless?

Test. R1 ∩ R2 = ∅. The empty set is a super key of neither relation (it determines nothing). So this decomposition is **lossy** — joining the pieces would produce the full cross product A × B × C × D, which contains spurious tuples. The key `( A, C )` would be lost entirely.

THE METHOD TO REUSE IN AN EXAMINATION

1. Write down the functional dependencies. Nothing else is possible without them.
2. Find the candidate key by computing closures.
3. Test 2NF: does any non-prime attribute depend on part of a composite key?
4. Test 3NF: does any non-prime attribute depend on another non-prime attribute?
5. Test BCNF: is any determinant not a super key?
6. Test 4NF: are two independent multi-valued facts in one table?
7. Verify losslessness at the end using the R1 ∩ R2 super-key test.

33. MCQ Bank with Answer Key

Part 8 • thirty multiple-choice questions with verified answers

Each question has one correct option, and the answer key explains why that option is correct and why the most tempting distractor is not.

Table 33.1 — Multiple-choice questions.

#	Question and options
1	A minimal super key is called a … (a) primary key (b) candidate key (c) foreign key (d) secondary key
2	The number of primary keys a table may have is … (a) zero (b) one (c) two (d) unlimited
3	Which key may contain a NULL value? (a) primary key (b) candidate key (c) unique key (d) none of these
4	An artificial key generated to identify records uniquely is a … (a) composite key (b) alternate key (c) surrogate key (d) partial key
5	Which key is used purely for indexing? (a) super key (b) secondary key (c) candidate key (d) foreign key
6	The constraint that values in one table must exist in another is called … (a) entity integrity (b) referential integrity (c) domain integrity (d) user integrity
7	In X → Y, X is called the … (a) dependent (b) determinant (c) domain (d) degree
8	A dependency A → B in which B is a subset of A is … (a) non-trivial (b) trivial (c) completely non-trivial (d) transitive
9	Which is **not** one of Armstrong's axioms? (a) reflexivity (b) augmentation (c) transitivity (d) decomposition
10	If X → Y and Y → Z, then X → Z. This is … (a) reflexivity (b) augmentation (c) transitivity (d) union
11	A non-prime attribute depending on part of a composite key is a … (a) transitive dependency (b) partial dependency (c) multivalued dependency (d) join dependency
12	Which normal form removes partial dependencies? (a) 1NF (b) 2NF (c) 3NF (d) BCNF
13	Which normal form removes transitive dependencies? (a) 2NF (b) 3NF (c) 4NF (d) 5NF
14	BCNF differs from 3NF in that it does not allow the exception where … (a) X is a super key (b) A is a prime attribute (c) the dependency is trivial (d) A is non-prime
15	Multivalued dependencies are removed by … (a) 2NF (b) 3NF (c) 4NF (d) 5NF
16	Join dependencies are addressed by … (a) 3NF (b) BCNF (c) 4NF (d) 5NF
17	Which normal form requires atomic values in every cell? (a) 1NF (b) 2NF (c) 3NF (d) BCNF

#	Question and options
18	Which technique deliberately adds redundancy to avoid joins? (a) normalization (b) denormalization (c) decomposition (d) indexing
19	Loss of **availability** means … (a) unauthorized modification of data (b) unauthorized disclosure of data (c) objects being unavailable to legitimate users (d) data being duplicated
20	“Who are you? Prove it!” describes … (a) authorization (b) authentication (c) inference control (d) flow control
21	Access control based on a central authority assigning security labels is … (a) DAC (b) MAC (c) RBAC (d) ACL
22	Which control prevents indirect disclosure through inference channels? (a) flow control (b) inference control (c) encryption (d) authentication
23	The pathways by which information flows implicitly in violation of policy are called … (a) tunnels (b) covert channels (c) sockets (d) buffers
24	The main problem with database encryption is … (a) speed (b) key management (c) storage size (d) the SQL dialect
25	Which PL/SQL construct executes its body at least once? (a) WHILE-LOOP (b) FOR-LOOP (c) basic LOOP (d) none of these
26	In a FOR loop, the range is evaluated … (a) on every iteration (b) once, when the loop is first entered (c) never (d) only if REVERSE is used
27	Which statement cannot be used outside a loop? (a) NULL (b) EXIT (c) GOTO (d) RETURN
28	For an implicit cursor, `SQL%ISOPEN` always returns … (a) TRUE (b) FALSE (c) NULL (d) the row count
29	Which statement is **not** allowed inside a FOR loop body? (a) reading the counter (b) assigning the counter (c) calling a procedure (d) an INSERT
30	Which view clause prevents an insert that would violate the view's WHERE condition? (a) WITH GRANT OPTION (b) WITH CHECK OPTION (c) WITH READ ONLY (d) WITH DISTINCT

Table 33.2 — Answer key with a one-line reason for each.

#	Answer	Reason
1	**(b) candidate key**	A candidate key is defined as a minimal super key.
2	**(b) one**	A relation is allowed to have only one primary key; it may have several candidate keys.
3	**(c) unique key**	A unique key is unique and non-updatable but may be NULL; a primary key may not.
4	**(c) surrogate key**	A surrogate key is artificial, usually an integer, and generated before insertion.
5	**(b) secondary key**	The source defines a secondary key as one required for indexing and faster searching.
6	**(b) referential integrity**	The source names the foreign-key concept as referential integrity.
7	**(b) determinant**	X is the determinant; Y is the dependent set.
8	**(b) trivial**	Trivial means B is a subset of A; it is always true and adds no information.
9	**(d) decomposition**	Reflexivity, augmentation and transitivity are the axioms; decomposition is derived from them.
10	**(c) transitivity**	This is the statement of the transitivity axiom.
11	**(b) partial dependency**	Depending on part of a composite key is the definition of a partial dependency.
12	**(b) 2NF**	2NF requires full functional dependency on the key.
13	**(b) 3NF**	3NF forbids a non-prime attribute being transitively dependent on a key.
14	**(b) A is a prime attribute**	3NF allows X → A when A is prime even if X is not a super key; BCNF does not.
15	**(c) 4NF**	4NF requires BCNF plus the absence of multivalued dependencies.
16	**(d) 5NF**	5NF requires every join dependency to be implied by the candidate keys.
17	**(a) 1NF**	1NF requires atomic values: no multi-valued or composite attributes.
18	**(b) denormalization**	Denormalization adds redundant data to avoid costly joins, after normalization.
19	**(c) objects unavailable to legitimate users**	Availability is defined as making objects available to those with a legitimate right.
20	**(b) authentication**	Authentication answers “who are you?”; authorization answers “what may you do?”.
21	**(b) MAC**	Mandatory access control uses security labels assigned by a central authority.
22	**(b) inference control**	Inference control is the countermeasure to the statistical-database security problem.
23	**(b) covert channels**	The source defines these pathways as covert channels (it spells it “convert channels”).

#	Answer	Reason
24	**(b) key management**	The source names key management as the main issue with storage-level encryption.
25	**(c) basic LOOP**	With no test before the first execution, the body always runs at least once.
26	**(b) once, when first entered**	The source states the range is evaluated when the loop is first entered and never re-evaluated.
27	**(b) EXIT**	EXIT must be inside a loop; to leave a block, use RETURN.
28	**(b) FALSE**	Oracle closes the SQL cursor automatically, so it is never observed open.
29	**(b) assigning the counter**	The loop counter may be referenced like a constant but not assigned.
30	**(b) WITH CHECK OPTION**	WITH CHECK OPTION prevents inserts or updates that do not satisfy the view's WHERE clause.

34. Viva Questions

Part 8 • short spoken answers to the questions examiners actually ask

Viva answers should be short, start with the key term, and stop. Each answer below is written to be spoken in one or two sentences.

Keys

1. What is a key?

An attribute or set of attributes that helps identify a row in a relation and establish relationships between tables.

2. Difference between a super key and a candidate key?

A super key identifies uniquely but may contain redundant attributes; a candidate key is a minimal super key, so no attribute can be removed.

3. Can a foreign key be NULL?

Yes. A foreign key may take the NULL value, meaning no related row is currently assigned.

4. Can a foreign key be non-unique?

Yes — in fact a foreign key is usually not unique, because many rows in the referencing table normally point to the same master row.

5. What is the difference between a composite key and a compound key?

A composite key is a primary key of two or more attributes; a compound key is a composite key in which every part is itself a foreign key.

6. What is a prime attribute?

An attribute that appears in some candidate key; attributes in no candidate key are non-prime.

Dependencies and normalization

7. What does X → Y mean?

Any two tuples that agree on X must also agree on Y — X determines Y.

8. State Armstrong's three axioms.

Reflexivity: if $Y \subseteq X$ then $X \rightarrow Y$. Augmentation: if $X \rightarrow Y$ then $XZ \rightarrow YZ$. Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$.

9. Can you split the left-hand side of a dependency?

No. From $AB \rightarrow C$ you cannot infer $A \rightarrow C$. Decomposition applies to the right-hand side only.

10. Why does normalization matter?

It removes redundancy and therefore the update, insertion and deletion anomalies that redundancy causes.

11. What is a partial dependency?

A non-prime attribute that depends on only part of a composite candidate key; 2NF removes it.

12. What is a transitive dependency?

A chain $X \rightarrow Y \rightarrow Z$ where Y does not determine X; 3NF removes it.

13. Difference between 3NF and BCNF?

3NF permits X → A when A is a prime attribute even if X is not a super key; BCNF requires every determinant of a non-trivial dependency to be a super key.

14. What is a multivalued dependency?

When two independent multi-valued facts about the same determinant share one table, producing a cross product; 4NF removes it.

15. Why is 5NF largely of academic interest?

Because join dependencies are very difficult to detect in practice, as the source itself notes.

16. What is a lossless decomposition?

One whose relations, joined back, reproduce the original relation exactly, with no rows lost and no spurious rows created.

17. State the lossless-join test.

The common attributes of the two relations must form a super key of at least one of them.

18. Does denormalization mean abandoning normalization?

No. It is an optimization applied after normalization, adding redundancy deliberately to avoid costly joins.

Security

19. What is database security?

Protection from malicious attempts to steal or modify data — the array of controls, tools and procedures safeguarding confidentiality, integrity and accessibility.

20. Name the three properties security must preserve.

Confidentiality, integrity and availability.

21. Difference between authentication and authorization?

Authentication proves who the user is; authorization decides what that user may do. Authentication happens first.

22. Name the three access-control models.

Discretionary (owner-controlled, using ACLs), mandatory (labels and clearance levels), and role-based (permissions attached to roles).

23. What is inference control?

A control that prevents a user from deducing sensitive information indirectly, by combining legitimate answers.

24. What is a covert channel?

A pathway by which information flows implicitly in a way that violates the privacy policy of the organization.

25. What is the main issue with encryption?

Key management — the key must be available at startup but must not be stored with the data, and losing it makes the data unreadable.

26. What is a threat?

Any intentional or accidental event that may adversely affect the system.

PL/SQL control structures

27. Name the three categories of control structure.

Conditional, iterative and sequential.

28. Difference between EXIT and EXIT WHEN?

EXIT leaves the loop unconditionally; EXIT WHEN evaluates a condition first and leaves only if it is true.

29. Can a WHILE loop body execute zero times?

Yes. The condition is tested at the top, so if it is false or null on entry the loop is bypassed.

30. Can a FOR loop body execute zero times?

Not for a normal ascending range — but if the range is empty (lower bound greater than higher bound without REVERSE) the body does not execute.

31. What happens if the FOR loop bounds are equal?

The body executes exactly once.

32. What does EXIT outer do in nested loops?

It exits every enclosing loop up to and including the labelled loop named `outer`.

33. What does the NULL statement do?

Nothing — it is a no-op, used to make it clear that a branch is deliberately empty.

34. Name two GOTO restrictions.

It cannot branch into an IF, CASE, LOOP or sub-block; and it cannot branch out of a subprogram. (Also: no jumping between IF clauses or CASE WHEN clauses; and from an exception handler it may jump out to an enclosing block but not back into the current block.)

Cursors and views

35. What is a cursor?

A pointer to the result set of a SQL query against database tables.

36. What is the active set?

The rows in the context area that meet the condition; its size equals the number of such rows.

37. Name the four explicit cursor steps.

Declare, open, fetch, close.

38. Why is `SQL%ISOPEN` always FALSE?

Because Oracle closes the implicit cursor automatically after executing the associated statement.

39. Why must EXIT WHEN come immediately after FETCH?

Because the cursor attribute reflects the most recent fetch; testing it immediately prevents the loop from processing a row that was not retrieved.

40. What is a view?

A specific representation of data from one or more tables, storing only the query and not the data.

41. What is a base table?

A table referred to in the definition of a view — the table that actually stores the data.

42. How many columns can a view have?

A maximum of 1000, as in tables.

43. Name the five conditions for updating a view.

No GROUP BY or ORDER BY; no DISTINCT; all NOT NULL values; not built from nested or complex queries; and built from a single table.

44. What does WITH CHECK OPTION do?

It makes the view's WHERE condition a constraint on writes: an INSERT or UPDATE that would not satisfy it returns an error.

45. Does dropping a view delete data?

No — it removes only the view definition, because the view stored no data of its own.

35. Last-Minute Revision Sheet

Part 8 • the night-before page

Definitions you must be able to write exactly

Table 35.1 — One-line definitions.

Term	One-line definition
Super key	A set of attributes that identifies each tuple uniquely; may contain redundant attributes
Candidate key	A minimal super key
Primary key	The candidate key the designer implements; one per table, never NULL
Alternate key	A candidate key that is not the primary key
Foreign key	An attribute whose values must exist as key values elsewhere; may repeat and may be NULL
Partial key	Identifies a group of related tuples, not a single tuple
Composite key	A primary key of two or more attributes
Unique key	Unique and non-updatable, but may be NULL
Surrogate key	An artificial generated identifier with no meaning
Secondary key	A key used for indexing and faster searching
Functional dependency	X → Y: equal X forces equal Y
Normalization	Designing a consistent database by minimising redundancy and ensuring integrity through lossless decomposition
Denormalization	Adding redundant data to avoid costly joins, applied after normalization
Database security	Protection from malicious attempts to steal or modify data
Authentication	Establishing who a user is
Authorization	Deciding what a user may do
Cursor	A pointer to the result set of a SQL query
Active set	The rows in the context area meeting the condition
View	A stored query representing data from one or more tables; stores no data

The rules, in the shortest form that is still correct

Table 35.2 — Normal forms and dependency tests.

Form	Shortest correct rule
1NF	Atomic values only
2NF	No non-prime attribute depends on part of a composite key
3NF	No non-prime attribute is transitively dependent on a key (or: X is a super key **or** A is prime)
BCNF	Every determinant of a non-trivial dependency is a super key
4NF	BCNF and no multivalued dependency
5NF	Every join dependency is implied by the candidate keys

Formulas and rules worth having at hand

Table 35.3 — The small amount of notation in these notes.

Item	Form
Functional dependency	`X → Y`
Multivalued dependency	`P →→ Q`
Join dependency	`*(A, B, C)`
Closure	`X⁺ = all attributes determined by X`
Reflexivity	`Y ⊆ X ⇒ X → Y`
Augmentation	`X → Y ⇒ XZ → YZ`
Transitivity	`X → Y, Y → Z ⇒ X → Z`
Lossless test	`(R1 ∩ R2) is a super key of R1 or of R2`
Prime attribute	Appears in some candidate key

PL/SQL essentials in six lines

Table 35.4 — The facts most likely to be asked.

Question	Answer
Which loop runs at least once?	Basic `LOOP`
Which loop may run zero times?	`WHILE` (condition tested at the top)
When is a FOR range evaluated?	Once, when the loop is first entered
Can the FOR counter be assigned?	No — read only, like a constant
What leaves a loop vs a block?	`EXIT` leaves a loop; `RETURN` leaves a block/subprogram
What is a no-op?	`NULL`

Cursors and views in eight lines

Table 35.5 — The facts most likely to be asked.

Question	Answer
Four explicit cursor steps	Declare, open, fetch, close
Implicit cursor attribute names	`SQL%FOUND`, `SQL%NOTFOUND`, `SQL%ISOPEN`, `SQL%ROWCOUNT`
Value of `SQL%ISOPEN`	Always FALSE
Correct cursor-loop order	FETCH then `EXIT WHEN c%NOTFOUND`
View storage cost	None — only the query is stored
Maximum view columns	1000
Five updatability conditions	No GROUP BY/ORDER BY; no DISTINCT; NOT NULL; not nested/complex; single table
What guards view writes?	`WITH CHECK OPTION`

Common mistakes and examination traps

THE TEN MOST COSTLY ERRORS

1. Writing “authentication controls what a user may access”. It is **authorization** that does that.
2. Saying a table can have two primary keys. It can have many candidate keys, one primary key.
3. Reversing the direction of a dependency and reading X → Y as “X depends on Y”.
4. Splitting the **left** side of a dependency (from AB → C inferring A → C). Decomposition is right-side only.
5. Treating 4NF as a replacement for BCNF. 4NF **includes** BCNF.
6. Claiming a WHILE loop always runs at least once. Its condition is at the top.
7. Writing `ELSE IF` instead of `ELSIF`.
8. Testing `%NOTFOUND` before the first `FETCH` in a cursor loop.
9. Forgetting that an unclosed cursor holds server memory.
10. Believing denormalization means skipping normalization. It is applied after it.

Frequently confused pairs

Table 35.6 — The pairs students mix up, and the one fact that separates them.

Pair	The separating fact
Super key / candidate key	Minimality: a candidate key has no removable attribute
Candidate key / primary key	Many candidate keys, exactly one primary key
Alternate key / secondary key	Alternate is a minimal identifier; secondary is only for indexing
Composite / compound key	Compound means every part is a foreign key
Primary key / unique key	Unique may be NULL; primary may not
3NF / BCNF	3NF allows A to be prime; BCNF does not
Multivalued / join dependency	Multivalued is 4NF's concern; join is 5NF's
Authentication / authorization	Who you are vs what you may do
DAC / MAC	Owner-controlled vs central authority with labels
Inference / flow control	Deducing indirectly vs information reaching the wrong place
Basic LOOP / WHILE	Body always runs once vs may run zero times
Implicit / explicit cursor	Oracle manages it vs you declare, open, fetch, close
View / base table	A view stores a query; a base table stores data

36. Source Coverage Checklist and References

Part 8 • what was covered, what was not, and where it came from

This chapter is the transparency statement. It lists every topic in the supplied source against the chapter that covers it, states plainly which course-outcome topics the source did not provide material for, and lists the references the source itself cites.

36.1 Topic-by-Topic Coverage

Table 36.1 — Coverage checklist. Every topic named in the source material maps to a chapter here.

Source topic	Covered in	Status
Super key	§2.1	**Complete**
Candidate key	§2.2	**Complete**
Primary key	§2.3	**Complete**
Alternate key	§2.4	**Complete**
Foreign key and referential integrity	§2.5	**Complete**
Partial key	§2.6	**Complete**
Composite key and compound key	§2.7	**Complete; source inconsistency flagged**
Unique key	§2.8	**Complete**
Surrogate key	§2.9	**Complete**
Secondary key	§2.10	**Complete**
Functional dependency meaning and notation	§3.1–3.3	**Complete**
Determinants and dependent attributes	§3.2	**Complete**
Trivial / non-trivial / completely non-trivial	§3.5	**Complete**
Armstrong's axioms	§4.1–4.2	**Complete**
Closure (added for reasoning)	§4.3–4.4	**Added — labelled as supplementary**
Transitive dependency	§5.1	**Complete**
Multivalued dependency	§5.2, §10.1	**Complete; source duplicate-row defect flagged**
Join dependency	§5.3, §11.2	**Complete; source over-generalisation flagged**
Normalization: purpose, features, anomalies	§6.1–6.2	**Complete**
1NF and worked example	§6.4–6.5	**Complete**
2NF, prime/non-prime, partial dependency	§7.1–7.4	**Complete**
3NF and the Employee/Zip example	§8.1–8.3	**Complete**
BCNF and the Employee/Department example	§9.1–9.4	**Complete**
4NF and both source examples	§10.1–10.4	**Complete**
5NF, two versions of agent/company/product	§11.1–11.3	**Complete**
Lossless decomposition (implied by the source's definition)	§12.1–12.4	**Made explicit**
Denormalization and the Courses/Teachers example	§14.1–14.3	**Complete**
Database security: definition and scope	§15.1–15.4	**Complete**
Types of security, levels of data security	§17.1–17.2	**Complete**
Physical / OS security	§17.3–17.4	**Complete**

Source topic	Covered in	Status
Database encryption and key management	§18.1–18.3	**Complete**
Authentication and authorization	§19.1	**Complete**
DAC, MAC, RBAC	§19.2–19.5	**Complete**
Network security: eavesdropping, masquerading, person-in-the-middle	§20.1	**Complete**
Passwords and smartcards	§20.2–20.3	**Complete**
Threats; loss of integrity, availability, confidentiality	§16.1–16.3	**Complete**
Six control methods	§21.1–21.8	**Complete**
DBA responsibilities and account privileges	§22.1–22.3	**Complete**
Conditional control: IF forms and CASE	§24.1–24.6	**Complete, all six source programs**
Iterative control: LOOP, EXIT, EXIT WHEN, WHILE, FOR, REVERSE	§25.1–25.7	**Complete, all four source programs**
Loop labels and nested loops	§25.3	**Complete**
Sequential control: GOTO, labels, restrictions	§26.1–26.3	**Complete, both source programs**
NULL statement	§26.4	**Complete, both source programs**
Cursor concept, context area, active set	§27.1–27.2	**Complete**
Implicit cursors and their attributes	§27.3–27.5	**Complete, including the source's SQL%ROWCOUNT program**
Explicit cursors: declare, open, fetch, close	§28.1–28.2	**Complete**
Complete explicit cursor example and output	§28.3	**Complete; source typos flagged**
SQL views: definition, base tables, benefits	§29.1–29.2	**Complete**
Creating views from one and many tables	§29.4–29.5	**Complete**
Querying a view	§29.6	**Complete**
Dropping a view	§30.1	**Complete**
CREATE OR REPLACE VIEW	§30.2	**Complete**
Five conditions for updating a view	§30.3	**Complete**
Inserting and deleting through a view	§30.4–30.5	**Complete**
WITH CHECK OPTION	§30.6	**Complete**
Uses of a view	§30.7	**Complete**
Course outcomes CO1–CO5	Table 0.3	**Reproduced**
Course objectives	Front matter	**Reproduced**

36.2 Documented Coverage Gaps

TOPICS THE SOURCE DOES NOT TEACH — DELIBERATELY NOT WRITTEN

The source's course-outcome slides name several topics for which the supplied PDF contains **no lecture material**. They are listed here so that the gap is visible rather than silently filled with invented content:

Topic named in the source's course outcomes	Material in the supplied PDF	Treatment here
Transaction processing (CO4)	**None**	Not written — no source material
Recovery techniques (CO4)	**None**	Not written — no source material
Triggers (CO5)	**None**	Not written — no source material
Procedures (CO5)	**None**	Not written — no source material
Packages (CO5)	**None**	Not written — no source material
DDL, DML, DCL, TCL commands (CO2)	Mentioned only in the course-outcome list	Individual statements appear where the source uses them (CREATE/DROP VIEW, INSERT, DELETE, UPDATE), but there is no systematic command chapter, because the source provides none

If your syllabus requires these topics for the examination, they must be studied from another source — this document cannot and does not claim to cover them.

36.3 How the Source's Defects Were Handled

Table 36.2 — Every point where the source needed care, and what was done.

Where	The issue in the source	What these notes do
§2.7	“Composite Key” defined with the surrogate-key wording, then redefined correctly later	Follow the later, correct definition; the copy error is flagged in a note
§3.1	“DeptId is functionally dependent on DeptName” — reversed direction	Notation retained; the inverted sentence flagged
§5.2	Duplicate rows in the “corrected” 4NF tables (Amit/Mathematics twice, Akash/Literature three times)	Corrected to true relations (rows are a set); the defect flagged
§5.3	States that the three-table decomposition “is not in 5NF” without the key-implied qualification	Presented as an illustration of what a join dependency is; the proper test explained in §11.1
§7.4	Enrolment table has no non-prime attribute, so the 2NF rule is vacuously satisfied	Explained honestly, with the source's structural point preserved
§21.5	Writes “convert channels”	Uses the standard term “covert channels” and notes the source's spelling
§28.3	`customer.name%type` (missing plural); variable-name inconsistency	Code reproduced as printed; both defects flagged with the correct form
§29.3	Sample tables shown only as images with the caption “Sample Tables:”	Rows reconstructed from the source's own reported outputs and labelled as reconstructed
Throughout	Outputs shown as “Output:” with no text	Every output derived by tracing the program, and stated as derived

36.4 References Cited by the Source

SUGGESTED BOOK REFERENCES (AS LISTED IN THE SOURCE MATERIAL)

1. J. Date, “An Introduction to Database Systems”, Addison Wesley.
2. Navathe, “Fundamentals of Database System”, The Benjamin / Cummings Publishing Co.
3. C. J. Date, “An Introduction to Database Systems”, Addison Wesley.
4. Thomas M. Connolly, Carolyn & E. Begg, “Database Systems: A Practical Approach to Design, Implementation and Management”, 5/E, University of Paisley, Addison Wesley.
5. Rob, “Database Principal Fundamental Design”, Cengage Learning.

Note: items 1 and 3 are the same book listed twice in the source, with slightly different punctuation. Both are reproduced here as the source gives them.

OTHER REFERENCES LISTED IN THE SOURCE (AS GIVEN)

- **Keys:** DBMS Keys: Candidate, Super, Primary, Foreign (Example) — guru99.com; DBMS Keys: Primary, Foreign, Candidate and Super Key — javatpoint; Types of Keys in DBMS | Definitions | Examples — Gate Vidyalay.
- **Normalization:** DBMS 1NF — javatpoint; Normalization in DBMS: 1NF, 2NF, 3NF and BCNF with Examples — hackr.io; Introduction to Database Normalization — softwaretesttips.com; Normalization in DBMS: 1NF, 2NF, 3NF and BCNF in Database — beginnersbook.com; Normalization in DBMS | 4 Useful Types of Normalization In DBMS — educba.com.
- **Dependencies:** Transitive dependency in DBMS — beginnersbook.com; Functional dependency in DBMS — tutorialspoint.com; Join dependency in DBMS — tutorialspoint.com; Multivalued dependency in DBMS — tutorialspoint.com.
- **4NF and 5NF:** Fourth Normal Form (4NF) — tutorialcup.com; DBMS — Joins — Tutorialspoint; Introduction to Database Normalization — softwaretesttips.com; Fifth Normal Form (5NF) — Computer Notes (ecomputernotes.com); Denormalization in Databases — GeeksforGeeks; Introduction of 4th and 5th Normal form in DBMS — GeeksforGeeks.
- **Control structures:** PL/SQL Control Structures — Oracle PL/SQL Tutorial | Intellipaat.com; Control Structures | DBMS | Tutorialink.com; PL/SQL Control Statement Exercises with Solution — w3resource; Writing PL/SQL Control Structures (Introduction) — relationaldbdesign.com; PL/SQL — GOTO Statement — Tutorialspoint.
- **Cursors:** PL/SQL — Cursors — Tutorialspoint; PL/SQL Cursor — javatpoint; PL/SQL Cursor — plsqltutorial.com.
- **Views:** SQL | Views — GeeksforGeeks; Oracle SQL & PL/SQL: SQL Views — sql-plsql.blogspot.com; PL/SQL Create View.

36.5 A Closing Note on Accuracy

Three commitments were made at the start of this document and are restated here, so that you can judge whether they were kept.

1. **Source fidelity.** Every definition, example, program and output traceable to the supplied UNIT 2 material has been preserved, with the source's own terminology. Where the source's examples were incomplete (images instead of tables, “Output:” instead of results), the missing content was reconstructed from the source's own text and reported outputs, and labelled as reconstructed.
2. **Honest labelling.** Additions beyond the source — the closure computation in Chapter 4, the case study in Chapter 13, the additional illustrative tables in Chapters 6, 7 and 29, and the whole question bank — are labelled as illustrative or supplementary wherever they appear. Nothing added has been presented as if it came from the source.
3. **Documented limits.** The topics named in the course outcomes for which the source supplies no material are listed in Section 36.2 and are not written here. That is a gap in this document, and it is stated rather than hidden.

HOW TO USE THE NOTES IN THE FINAL WEEK

Day 1: Chapter 31 — the comparison tables. **Day 2:** Part 3 (normalization), working every example on paper. **Day 3:** Parts 5 and 6 (PL/SQL and cursors), typing each program. **Day 4:** Parts 1 and 2 (keys and dependencies), then Chapter 32's normalization problems. **Day 5:** Parts 4 and 7 (security and views). **Day 6:** Chapter 33 MCQ and Chapter 34 viva. **Night before:** Chapter 35 only.